

For B.E. (Pokhara University), B.Sc.CSIT, BEIT

System Inception's

Insights on

Database Management System (DBMS)

Er. Bishon Lamichhane
Er. Ujawal Thapa



CamScanner

Insights on Database Management System

Published by : System Inception

Authors : Er. Bishoo Lamichhane
Er. Ujjwal Thapa

C : Publisher

All rights reserved. This book is sold subject to the condition that it shall not, by way of trade or otherwise, be lent, resold, hired out, or otherwise circulated without the authors' prior written consent in any form of binding or cover other than that in which it is published and without a similar condition including this condition being imposed on the subsequent purchaser and without limiting the rights under copyright reserved above, no part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means (electronic, mechanical, photocopying, recording or otherwise), without the prior written permission of the copyright owner of the book.

Edition : First 2024 AD

Computer : Creation Graphics
Bagbazar, Kathmandu

TABLE OF CONTENTS

Chapter - 1

INTRODUCTION

1.1	Concept and Applications	1
1.1.1	Data	1
1.1.2	Information	1
1.1.3	Database	2
1.1.4	Database Management System (DBMS)	2
1.1.5	Database Application	3
1.2	Objectives and Evolution	4
1.2.1	Objectives of DBMS	4
1.2.2	Evolution of DBMS	4
1.3	Needs of DBMS	5
1.4	Data Abstraction	6
1.5	Data Independence	8
1.6	Schema and Instances	11
1.6.1	Schema	11
1.6.2	Instance	12
1.7	Concept of DDL, DML and DCL	13
1.7.1	Data Definition Language (DDL)	13
1.7.2	Data Manipulation Language (DML)	14
1.7.3	Transaction Control Language (TCL)	14
1.8	Database Manager and Users	15
1.8.1	Database Manager (Administrator)	15
1.8.2	Database User	17
	SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS	18

Chapter - 2

DATA MODELS

2.1	Logical, Physical and Conceptual Model	23
2.1.1	Conceptual Model	23
2.1.2	Logical Model	24
2.1.3	Physical Model	25



2.2	E-R (Entity-Relationship) Model	26
2.2.1	Entity and Entity Set	27
2.2.2	Attributes and Keys	31
2.2.3	Relationship and Relationship Set	36
2.2.4	Extended ER (EER) Features	46
2.2.5	Entity - Relationship Diagram	56
2.3	Relation with UML Class Diagrams	58
2.4	Alternative Data Models	59
2.4.1	Relational Data Model	59
2.4.2	Hierarchical Data Model	61
2.4.3	Network Data Model	61
	SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS	64

Chapter - 5 RELATIONAL MODEL

3.1	Definitions and Terminology	75
3.2	Structure of Relational Databases	75
3.3	The Relational Algebra	77
3.4	Schemas and Views	85
3.5	Data Dictionary	86
	SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS	88

Chapter - 6 RELATIONAL DATABASE QUERY LANGUAGES

4.1	Introduction to SQL	101
4.1.1	Features of SQL	101
4.1.2	SQL Basic Data Types	102
4.1.3	Queries and Sub Queries	103
4.1.4	Set Operations	105
4.1.5	Joins	110
4.2	DDL and DML Queries in SQL	116
4.2.1	Data Definition Language (DDL)	116
4.2.2	Data Manipulation Language (DML)	120
4.2.3	DCL (Data Control Language)	125

4.2.4	Basic Aggregation	127
4.2.5	Aggregation with Grouping	128
4.2.6	Having Clause	130
4.2.7	Views	133
4.3	Stored Procedures	138
4.4	Query by Example (QBE)	141
	SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS	141

Chapter - 7 DATABASE CONSTRAINTS AND RELATIONAL DATABASE DESIGN

5.1	Introduction	165
5.2	Integrity Constraints	165
5.3	Referential Integrity	168
5.4	Assertions and Triggers	171
5.4.1	Assertion	171
5.4.2	Trigger	172
5.5	Functional Dependencies	177
5.6	Multivalued Dependencies	182
5.7	Normalization and Normal Forms	183
5.8	Decomposition of Relation Schemas	190
	SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS	194

Chapter - 8 SECURITY

6.1	Needs of Security	209
6.2	Security and Integrity Violations	210
6.3	Access Control	211
6.4	Authorization and Authentication	214
6.4.1	Authorization	214
6.4.2	Authentication	215
6.5	Security and Views	215
6.6	Encryption and Decryption	216
	SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS	220

Chapter - 9 QUERY PROCESSING

7.1	Introduction to Query Processing.....	236
7.2	Equivalence of Expressions.....	236
7.3	Query Cost Estimation.....	236
7.3.1	Selection Operation.....	236
7.3.2	Sorting.....	240
7.3.3	Join Operation.....	240
7.4	Query Optimization.....	246
SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS ... 246		

Chapter - 10 FILE ORGANIZATION AND INDEXING

8.1	Disks and Storage.....	257
8.2	Organization of Records into Blocks.....	259
8.2.1	Fixed Length Records.....	259
8.2.2	Variable Length Records.....	262
8.3	File Organizations.....	267
8.3.1	Sequential File Organization.....	268
8.3.2	Indexed Sequential File Organization.....	269
8.4	B+ Tree Index.....	277
8.5	Hash Index.....	279
SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS ... 279		

Chapter - 11 CRASH RECOVERY

9.1	Failure Classification.....	294
9.2	Concept of Log Based Recovery and Shadow Paging.....	295
9.2.1	Log Based Recovery.....	295
9.2.2	Shadow Paging.....	301
9.3	Data Backup/Recovery.....	304
9.3.1	Data Backup.....	304
9.3.2	Data Recovery.....	307
9.4	Remote Backup Systems.....	309
SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS ... 309		

Chapter - 12 TRANSACTION PROCESSING AND CONCURRENCY CONTROL

10.1	Introduction to Transactions.....	317
10.1.1	ACID Properties of Transaction.....	318
10.2	Schedules and Serializability.....	321
10.3	Schedule.....	324
10.3.1	Serializability.....	333
10.3.2	Concepts of Locking for Concurrency Control.....	334
10.4	Two-Phase Locking (2PL).....	336
10.4.1	Strict Two-Phase Locking (Strict-2PL).....	338
10.4.2	Deadlock.....	339
10.5	Deadlock Avoidance.....	339
10.5.1	Deadlock Detection.....	340
10.5.2	Deadlock Prevention.....	340
10.5.3	Deadlock Prevention.....	340
SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS ... 340		

Chapter - 13 ADVANCED DATABASE CONCEPTS

11.1	Object Oriented Model.....	345
11.2	Object Relational Model.....	348
11.3	Distributed Databases.....	351
11.3.1	Homogeneous Distributed Database.....	351
11.3.2	Heterogeneous Distributed Database.....	351
11.3.3	Distributed Database Storage.....	355
11.4	Concept of Data Warehouse.....	357
11.4.1	Rules of Data Warehousing.....	358
11.4.2	Need for Data Warehousing.....	358
11.4.3	Data Warehouse Architecture.....	359
11.4.4	Application of Data Warehouse.....	360
SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS ... 360		

INTRODUCTION

1.1 Concept and Applications

1.1.1 Data

Data are the raw facts and figure. In the context of databases, data refers to all the single items that are stored in a database, either individually or as a set.

Example:

In a banking application, data is the whole collection of bank account numbers, bank customer's names, addresses, ages, bank transactions and so on.

1.1.2 Information

Information is data that has been processed, organized, or interpreted to make it meaningful, relevant, and useful for decision-making or understanding.

Example:

Bank statement

Differences between Data and Information

Data refers to raw facts or figures that are collected and stored in the system, whereas information refers to the processed and analyzed data that provides meaning and context to the raw data.

For example, in a banking system, the data might include:

- Account numbers
- Transaction dates
- Amounts debited or credited
- Account holders' names
- Account types

These data can be stored in a database, but on its own, they don't provide any meaningful information. However, when these

data are processed and analyzed, they can be transformed into useful information.

For example:

- An account statement that shows a customer's transaction history and current balance.
- A monthly report that shows the total deposits and withdrawals for each account type.
- A fraud detection system that uses transaction data to flag suspicious activity.

In each of these cases, the raw data is processed and analyzed to provide meaningful information that can be used to make decisions or take action.

11.3 Database

A database is a collection of related data necessary to manage an organization. A database is usually controlled by a database management system (DBMS).

Example:

A company can have various details of employees, such as name, empID, email, blood group, salary, and so on. All these details can be stored in a database with the name "Employee" in a structured format such as tables, hierarchy, etc.

Types of databases are:

1. Relational database
2. Cloud database
3. NoSQL database
4. Graph database

11.4 Database Management System (DBMS)

Database management system (DBMS) is a system software that is used to access, create, and manage databases. With the help of DBMS, we can easily create, retrieve and update data in databases.

Type of DBMS are:

1. Object Oriented DBMS
2. Network DBMS
3. Relational DBMS
4. Hierarchical DBMS

Example of DBMS are:

MySQL, PostgreSQL, Microsoft Access, SQL Server, FileMaker, Oracle, RDBMS, dBASE, Clipper, and FoxPro.

Relation between Database and DBMS

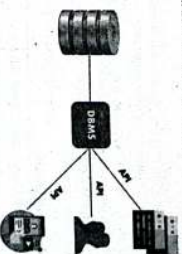


Figure: Relation between database and DBMS

In figure, API means *Application Programming Interface*; it is a software intermediary that allows two applications to talk to each other.

11.5 Database Application

Following are the applications of database:

- > Banking: Transactions
- > Airlines: Reservations, schedules
- > Universities: Registration, grades
- > Sales: Customers, products, purchases
- > Online retailers: Order tracking, customized recommendations
- > Manufacturing: Production, inventory, orders, supply chain

- **Human resources:** Employee records, salaries, tax deduction.
- **Telecommunication:** Records of calls, texts and data usage, generating monthly bills, maintaining balances on prepaid calling cards.

1.2 Objectives and Evolution

1.2.1 Objectives of DBMS

Following are the objectives of DBMS:

- A database should provide efficient storage, update, and retrieval of data.
- A database should be reliable - the stored data should have high integrity and promote user trust in that data.
- A database should be adaptable and scalable to new and unforeseen requirements and applications.
- A database should identify the existence of common data and avoid duplicate recording.

1.2.2 Evolution of DBMS

The history and evolution of DBMS is given below:

- **1960:** Charles Bachman designed first DBMS system.
- **1970:** Codd introduced IBM's Information Management System (IMS).
- **1976:** Peter Chen coined and defined the entity-relationship model also known as the ER model.
- **1980:** Relational model becomes a widely accepted database component.
- **1985:** Object-oriented DBMS develops.
- **1990s:** Incorporation of object-orientation in relational DBMS.
- **1991:** Microsoft ships MS access, a personal DBMS and that displaces all other personal DBMS products.
- **1995:** First Internet database applications.
- **1997:** XML applied to database processing. Many vendors begin to integrate XML into DBMS products.

1.3 Needs of DBMS

A file system is a process that manages how and where data on a storage disk, typically a hard disk drive (HDD), is stored, accessed and managed. It is built in OS.

Example:
NTFS (New Technology File System), EXT (Extended File System).

DBMS is needed because of the following drawbacks of file system over DBMS.

- 1. Data redundancy and inconsistency problem**
Data redundancy refers to the duplication of data. Data redundancy often leads to data inconsistency, higher storage costs and poor access time.
Data inconsistency means different and conflicting versions of same data occur at different places.
- 2. Difficult in accessing data**
We need to write a new program to carry out each new task in file system.
Example:
To obtain few KB of data from a huge TB data, we need to process all the huge data first which makes difficulty in accessing data in file system.
- 3. Data isolation problems**
File system consists of unstructured data i.e., multiple files and formats which leads to data isolation problem.
- 4. Integrity problems**
It is hard to add new constraints or change existing ones in file system. The term data integrity refers to the accuracy and consistency of data. For example, a user could accidentally try to enter a phone number into a date field. If the system enforces data integrity, it will prevent the user from making these mistakes.

5. Atomicity problems

Failures may leave database in an inconsistent state with partial updates carried out. For example: Transfer of funds from one account to another should either complete or not happen at all. It is difficult to achieve atomicity in file processing systems.

6. Concurrent access anomalies

Uncontrolled concurrent accesses in file processing system can lead to inconsistencies.

7. Security problems

Data should be secured from unauthorized access. For example a student in a college should not be able to see the payroll details of the teachers, such kind of security constraints are difficult to apply in file processing systems.

Differences between file system and DBMS:

1. A file system is a software that manages and organizes the files in a storage medium, whereas DBMS is a software application that is used for accessing, creating, and managing databases.
2. The file system doesn't have a crash recovery mechanism or the other hand, DBMS provides a crash recovery mechanism.
3. Data inconsistency is higher in the file system. On the contrary data inconsistency is low in a database management system.
4. File system does not provide support for complicated transactions, while in the DBMS system, it is easy to implement complicated transactions using SQL.
5. File system does not offer concurrency, whereas DBMS provides a concurrency facility.
6. File system are often single user oriented, whereas DBMS are often multiuser oriented.

1.4 Data Abstraction

Databases systems are made up of complex data structures. To ease the user interaction with database, the developers hide internal irrelevant details from users. This process of hiding irrelevant details from user is called data abstraction.

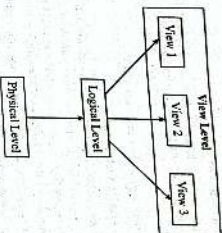


Figure: Levels of data abstraction

There are three levels of abstraction:

1. Physical Level (Internal Level)

This is the lowest level of data abstraction. It describes how data is actually stored in database. We can get the complex data structure details at this level. The access methods like sequential or random access and file organization methods like B + trees, hashing used for the same. Suppose we need to store the details of an employee, blocks of storage and the amount of memory used for these purposes is kept hidden from the user.

2. Logical Level (Conceptual Level)

This is the middle level of 3-level data abstraction architecture. It describes what data is stored in database and the relationship among the data.

Example:

name : string;
street : string;
city : integer;
ent:

type customer = record

3. View Level (External Level)

This is the highest level of data abstraction. This level describes the user interaction with database system.

Example:
Interaction of user with system using GUI.

Example of Data abstraction

Let's say we are storing customer information in a customer table. At physical level, these records can be described as blocks of storage (bytes, gigabytes, terabytes etc.) in memory. These details are often hidden from the programmers.

At logical level, these records can be described as fields and attributes along with their data types, their relationship among each other can be logically implemented. The programmers generally work at this level because they are aware of such things about database systems.

At view level, user just interact with system with the help of GUI and enter the details at the screen, they are not aware of how the data is stored and what data is stored, such details are hidden from them.

1.5 Data Independence

Data independence can be explained using the three-schema architecture. Data independence refers to characteristic of being able to modify the schema at one level of the database system, without altering the schema at the next higher level. The interfaces between the various levels and components should be well defined so that changes in some parts do not seriously influence others.

Importance of data modeling

- Data independence helps us to improve the quality of the data
- Database system maintenance becomes affordable.
- Enforcement of standards and improvement in database security
- We don't need to alter data structure in application programs

- Permit developers to focus on the general structure of the database rather than worrying about the internal implementation
- It allows us to improve state which is undamaged or undivided
- Database incompatibility is vastly reduced.

There are two types of data independence:

1. Physical Data Independence

It indicates that the physical storages structures or devices could be changed without affecting the conceptual schema. It is the ability to modify the physical schema without changing the logical schema.

Benefits of physical data Independence

- a. Performance Optimization
To increase performance without affecting the application programs, DBMS administrators might rearrange or restructure the physical storage of data. For instance, they can alter partition tables, indexes, or storage formats to improve the speed of data processing and retrieval.

b. Platform Independence

It is possible to move data between various hardware or operating systems thanks to physical data independence. The DBMS can migrate to new platforms or adapt to new storage technologies without requiring changes to the programs that use the database.

c. Data Security

Without interfering with the functioning of the current applications, changing the physical storage structure can help impose security features like encryption, access control, or data segregation.

2. Logical Data Independence

It is the capacity to change the conceptual schema without having to change the external schemas and their application programs. We may change the conceptual schema to expand

the database (by adding a record type or data item), or to reduce the database (by removing a record type or data item).

Benefits of logical data independence

a. Evolution of Data Models

Data models may need to be improved, expanded, or modified over time to meet shifting business needs. Modifications to the database's structure are possible with logical data independence without having an influence on the programs that use it. This adaptability makes it simpler to meet changing corporate needs.

b. Database Restructuring

The database schema can be reorganized to improve data integrity, normalization, or data modeling with the help of logical data independence. These changes can be implemented without impairing current applications or resulting in inconsistent data.

c. Data Integration

The seamless integration of additional data sources or databases into an existing system is supported by logical data independence. It permits the creation of new tables, relationships, or attributes without interfering with the operation of any already-running applications that are using the database.

Differences between physical and logical data independence

Logical Data Independence	Physical Data Independence
Ability to change the logical schema without changing the external schema (user view) is called logical data independence.	Ability to change the physical schema without changing the logical schema is called physical data independence.
Modification is necessary whenever the logical structure of the database is altered.	Modification is rarely done, as it is done to improve the performance of the database.

Logical Data Independence	Physical Data Independence
It is more difficult to achieve.	It is relatively easy to achieve.
Example: Consider two users A & B, both are selecting the "employee number" field. If user B adds to "D" drive will not affect the new column "salary" to the same conceptual and view level as table, it will not affect the user by the mapping technique.	Example: Changing the storage drive of the data base from "C" drive to "D" drive will not affect the new changes are absorbed by the mapping technique.

1.6 Schema and Instances

1.6.1 Schema

Design of a database including tables, fields, relationships, views and other elements is called the schema. Database schema is analogous to the variable declarations (along with associated type).

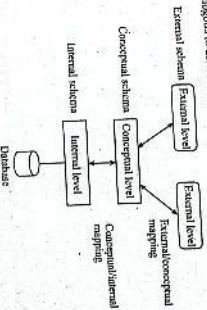


Figure: Various levels of schema

Types of schema:

1. Physical Schema

The design of a database at physical level is called physical schema; how the data stored in blocks of storage is described at this level.

2. Logical Schema

Design of database at logical level is called logical schema. programmers and database administrators work at this level.

3. View Schema

Design of database at view level is called view schema, (also called external schema); this generally describes end user interaction with database systems.

Example:

In the following diagram, we have a schema that shows the relationship between three tables: Course, Student, and Section. The diagram only shows the design of the database, it doesn't show the data present in those tables. Schema is only a structural view (design) of a database.

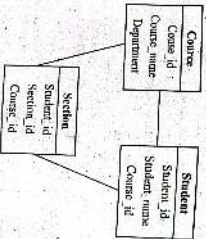


Figure: Schema of 3 tables: Course, student and section

1.6.2 Instance

The data stored in database at a particular moment of time is called instance of database. Database instance is analogous to the value of variables in a program at a point in time.

Example:

Let's say we have a single table student in the database, today the table has 100 records, so today the instance of the database has 100 records. Let's say we are going to add another 100 records in this table by tomorrow so the

instance of database tomorrow will have 200 records in table. In short, at a particular moment the data stored in database is called the instance, that changes over time when we add or delete data from the database.

Differences between schema and instance:

Schema	Instance
It is the overall description of the database.	It is the collection of information stored in a database at a particular moment.
Schema is same for whole database.	Data in instances can be changed using addition, deletion, updating.
Defines the basic structure of the database i.e. how the data will be stored in the database.	It is the set of information stored at a particular time.

Consider a database for a social media platform. The schema might include tables for users, posts, comments, and likes, as well as columns for the username, email address, and password of each user. The schema would also define the relationships between the tables, such as the fact that each user can make many posts and receive many comments and likes. The instance, in this case, would be the actual data stored in the tables at a given moment, such as a list of all the posts made by a particular user and the associated comments and likes.

1.7 Concept of DDL, DML and DCL

Database languages are the language that allow users to communicate with database. The main purpose of these language is to define the schema and then store, access and manipulate the data. This can be implemented with SQL (Structured Query language). A statement requesting the retrieval of information is called query statement.

1.7.1 Data Definition Language (DDL)

DDL deals with the description of the database schema where it creates and modifies the structure of the database objects on the database.

DDL compiler generates a set of table templates stored in a data dictionary. Data dictionary contains metadata (i.e., data about data). DDL includes:

i. Database schema

ii. Integrity constraints

iii. Authorization (who can access what)

Example:

CREATE, ALTER, DROP, TRUNCATE, COMMENT, RENAME

DDL

1.7.2 Data Manipulation Language (DML) is a language for

A data manipulation language (DML) is a language for accessing and manipulating the data organized by the appropriate data model. DML is also known as query language. It is used to

retrieve, store, modify, delete, insert and update data in database.

Example:

SELECT, INSERT, UPDATE, DELETE

DML

There are two classes of DML:

1. **Procedural:** Here user specifies what data is required and how to get those data.

Example:

C, COBOL, BASIC, ALGOL, Pascal, etc.

2. **Nonprocedural (declarative):** Here user specifies what data is required without specifying how to get those data.

Declarative DMLs are usually easier to learn and use than procedural DMLs.

Example:

SQL, Relational Calculus, LISP, Prolog.

1.7.3 Transaction Control Language (TCL)

Transaction control language (TCL) is used to manage different transactions occurring within a database.

Example:

COMMIT, ROLLBACK, SAVE TRANSACTION

1.8 Database Manager and Users

1.8.1 Database Manager (Administrator)

A person who has central control over the database management system is called database manager. Database manager is responsible for authorizing access to the database, for coordinating and monitoring its use, acquiring software and hardware resources and controlling their use and monitoring efficiency of operations.

Role of Database administrator

In order to manage and maintain the effectiveness, security, and integrity of a database system, a Database Administrator (DBA) is essential. DBAs are essential to many aspects of database management and administration during the course of a database's lifecycle. The following are some significant duties and tasks of a database administrator:

i. Database Design

DBAs collaborate closely with developers and other stakeholders to establish the database structure, schema, and relationships throughout the design phase of a database system. They make sure that the design complies with the needs of the company and industry standards for performance and scalability.

ii. Database Installation and Configuration

DBAs are in charge of setting up the database management system (DBMS) software and customizing it to the requirements of the enterprise. To maximize performance and resource usage, they configure the appropriate parameters, memory allocation, storage configuration, and other system settings.

iii. Performance Tuning and Optimization

DBAs monitor and assess the performance of databases, finding bottlenecks and improving queries and database configurations for increased effectiveness. To improve system responsiveness and throughput, they carry out

performance tuning procedures such as index optimization, query optimization, database parameter adjustments, and workload management.

iv. Backup and Recovery

To safeguard data against possible loss or corruption, DBAs create and maintain backup and recovery strategies. To ensure that data can be recovered in the event of system failures, natural disasters, or human mistakes, they design backup schedules, carry out routine backups, and test restoration procedures.

v. Security and Access Control

To protect the database and its sensitive data, DBAs put security procedures in place and make sure they are followed. They oversee user access privileges, roles, and permissions to the database, making sure that only authorized users have the proper access. To safeguard the integrity and confidentiality of data, DBAs also use security measures such as encryption and auditing tools.

vi. Database Monitoring and Maintenance

DBAs keep constant watch on the database system for problems with performance, availability, and security. To monitor resource usage, spot anomalies, and take preventative measures against possible issues, they employ monitoring tools. To maintain optimal database performance, DBAs also carry out standard maintenance procedures such as database rearrangement, statistics updating, and space management.

vii. Capacity Planning and Scalability

DBAs evaluate the database system's present and potential needs and make plans for its expansion and scalability. In order to meet rising data volumes and user needs, they keep an eye on patterns of data growth, resource use trends, and prepare for capacity improvements, storage expansion, and system innovations.

viii. Database Upgrades and Patch Management

ix. Disaster Recovery Planning

1.8.2 Database User

A person who uses or accesses the data from a database is called database user.

Types of user:

1. Application Programmer

Application programmer implements the specification generated by system analyst in some high level programming language

2. Native End User

Native end user is typically unaware of DBMS and access the database through specially written application programs. Native end user uses form based interface for querying and updating database.

3. Sophisticated End User

Sophisticated end user may use high level query language to perform any required operations. Engineers, scientists, business analyst who thoroughly familiarize themselves with the facilities of DBMS so as to implement their application to meet their complex requirement are sophisticated end user.

SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS

1. List the major steps that you would take in setting up a database for a particular enterprise. [10 Marks]

Ans Major steps involved in setting up a database for an enterprise:

i. Identify the purpose and scope of the database

Before setting up a database, it's important to understand why the enterprise needs a database and what information it needs to store and manage. This will help determine the scope of the database and guide the design process.

ii. Choose a DBMS

The next step is to choose a suitable DBMS based on the enterprise's requirements and budget. The DBMS should be able to handle the scale and complexity of the enterprise's data and provide the necessary features for data management, security, and analysis.

iii. Define the data model

The data model defines the structure of the database and how data is organized and related. This involves identifying the entities (objects or concepts) that need to be stored in the database, as well as their attributes and relationships.

iv. Design the database schema

Based on the data model, the next step is to design the database schema, which specifies the tables, columns, and constraints that make up the database. This includes defining primary and foreign keys, setting up indexes, and enforcing data integrity rules.

v. Populate the database

Once the schema is defined, the database can be populated with data. This involves entering data manually, importing data from external sources, or a combination of both.

- vi. Test the database
- vii. Deploy the database

viii. Maintain the database

The process may involve additional steps depending on the enterprise's specific needs and requirements.

2. What are the major responsibilities of Database Management System? For each responsibility, explain the problems that would arise if the responsibility were not discharged. [10 Marks]

Ans The main responsibilities of a database management system (DBMS) are listed below, along with the problems that may occur if each responsibility is not discharged:

1. Data Definition

- **Responsibility:** Defining the structure and schema of the database.
- **Problem:** Data inconsistency and ambiguity can result from improperly defined data. Users could interpret the data differently in the absence of explicit definitions, leading to inaccurate analysis, reporting, and decision-making.

ii. Data Manipulation

- **Responsibility:** Inserting, updating, deleting, and retrieving data from the database.
- **Problem:** Data integrity may be at risk if data manipulation is not done properly. Data that is inaccurate or lacking certain information may be stored, producing inaccurate results and untrustworthy information. Decision-making and business operations may be impacted by inconsistent data.

iii. Data Storage and Retrieval

- **Responsibility:** Managing the physical storage and efficient retrieval of data.
- **Problem:** Inefficient data storage can lead to poor performance and higher storage needs. Data retrieval can become time-consuming without effective indexing.

and storage optimization, which has an adverse effect on system responsiveness and user experience.

iv. Data Security

Responsibility: Enforcing security measures to protect data from unauthorized access.

- **Problem:** Sensitive data can be compromised without sufficient security, resulting in data breaches, privacy violations, and legal repercussions. Unauthorized users have the ability to access, change, or destroy data, resulting in financial losses and reputational harm to the business.

v. Data Integrity and Consistency

Responsibility: Enforcing data integrity constraints and maintaining consistency.

- **Problem:** Data that is inconsistent or wrong might produce unreliable analyses, reports, and decisions. Problems with data integrity can compromise the system's credibility and dependability as a whole.

vi. Concurrency Control

Responsibility: Managing concurrent access to the database by multiple users.

- **Problem:** Conflicts may occur if numerous users attempt to change the same piece of data at the same time without appropriate concurrency management. Inconsistent data can result in inaccurate findings and corrupted data, such as missing updates or dirty reads.

vii. Backup and Recovery

Responsibility: Creating backups and restoring data in case of failures or errors.

- **Problem:** Data loss that is permanent might be caused by insufficient or poor backups. Without proper recovery measures, system malfunctions or human mistakes might result in data being lost forever, resulting in operational hiccups and possibly large losses.

viii. Performance Optimization

Responsibility: Optimizing the performance of the database system.

- **Problem:** Without enough performance optimization, queries can be executed inefficiently, responses take a long time to process, and resources can be used up quickly. This may lead to poor system performance, disgruntled users, and reduced productivity.

ix. Data Independence

Responsibility: Providing an abstraction layer between the physical and logical data representation.

- **Problem:** Applications and the database structure may get entangled as a result of a lack of data independence. All dependent applications may need to be modified if the database schema is changed, which can complicate maintenance and limit system adaptability.

x. Data Administration

Responsibility: Managing the overall administration and maintenance of the database system.

- **Problem:** Inadequate data management can lead to underutilizing system performance, security flaws, and ineffective resource management. The database system may become unreliable and challenging to administer without sufficient monitoring, upkeep, and user control. To guarantee data reliability, security, and optimum system performance, these responsibilities must be fulfilled. Data inconsistencies, security breaches, poor performance, and operational difficulties inside the database system can result from ignoring these responsibilities.

3. Differentiate between Data abstraction and Data independence. 10M/11M

Ans: The difference between data abstraction and Data independence is given below:

• Data abstraction

Data abstraction refers to the process of hiding the complexity of the underlying data model from the users and applications that access the database. This is typically achieved through the use of abstraction layers or interfaces that provide a simplified view of the database and shield

DATA MODELS

users and applications from the underlying details. The purpose of data abstraction is to provide a simple and consistent interface for users and applications to interact with the database, regardless of the underlying complexity.

Example: In a database for an e-commerce website, the abstraction layer might provide a simple interface for users to search for products, add them to their cart, and check out. The users don't need to know about the complex data model used to store information about products, inventory levels, and customer orders. The abstraction layer hides these details and presents a simplified view of the data.

• Data independence

Data independence refers to the ability to modify the schema or data storage mechanisms without affecting the applications or users that depend on the database. There are two types of data independence: physical data independence and logical data independence.

Physical data independence allows the physical storage structure of the database to be changed without affecting the applications or users.

Example:

The underlying storage medium (such as hard disk or solid-state drive) could be changed, or the data could be partitioned across multiple disks, without affecting the applications or users.

Logical data independence allows the logical structure of the database to be changed without affecting the applications or users.

Example:

The schema could be modified to add or remove tables, columns, or constraints, without affecting the applications or users.

The purpose of data independence is to make the database more flexible and scalable, allowing it to adapt to changing requirements or technologies without requiring major changes to applications or users.



2.1 Logical, Physical and Conceptual Model

Data model is the collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints. There are various types of data models such as:

2.1.1 Conceptual Model

The conceptual data model is the very first and the most abstract data model in the data modeling process. It is a high-level diagram that we use to define, describe, organize, and present data elements and their relationships with relatively few details. We use the conceptual data model to communicate with the different business people when defining the business requirements of the database and presenting concepts (e.g. for their feedback). We do not use these models to communicate with technical teams. This model does not have technical details, such as attributes, data types, etc.

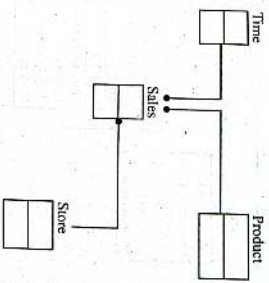


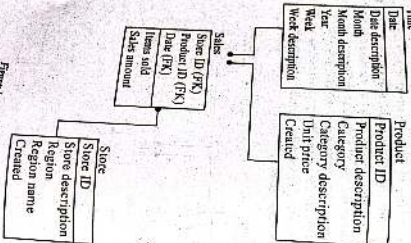
Figure: Conceptual model

In summary:

- Identifies entities and establishes their relationships.
- Simplified and highly abstract structure.
- No attribute is specified.
- No primary key is specified.

2.1.2 Logical Model

Logical models are less abstract and provide greater detail about the concepts and relationships in the domain, unlike consideration. They include descriptions of tables and columns, object-oriented classes, primary and foreign keys.



In summary:

- Adds detail information on conceptual model.
- Adds attributes, keys, and degree of relationships.
- Requires more effort and attention to build than conceptual model.

2.1.3 Physical Model

Physical data model represents how the model will be built in the database. A physical database model shows all table structures, including column name, column data type, column constraints, primary key, foreign key, and relationships between tables.

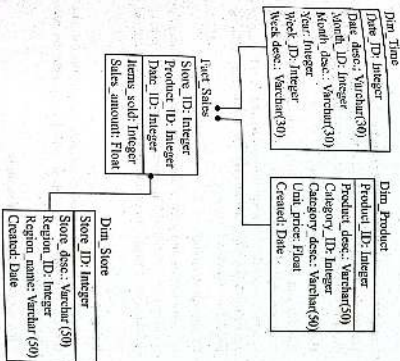


Figure 2.1.2: Logical Model

Figure 2.1.3: Physical Model

In summary:

- Refers to converting design to relation (table).
- Converts entity to table and attributes to columns.
- Converts compatible table name and column name.
- Uses database compatible table name.
- More difficult to understand.
- Require more efforts than logical model.

2.2 E-R (Entity-Relationship) Model

E-R model is defined as a conceptual data model that views the real world as entities and relationship. It was introduced by P.P. Chen in 1976. It expresses the overall logical structure of database graphically. The major advantages of E-R model are as follows:

- 1. Straightforward relation representation**
The relation representation of the database model using E-R diagram are relatively more straightforward than other models.
- 2. Mapping with relational model**
It can be easily mapped onto the relational model. The E-R diagrams used in the E-R model can easily be transformed into relational tables. The entities and attributes of E-R model can easily be transformed into relations (tables) and columns (fields) in a relational model.
- 3. Communication tool**
It is very simple and easy to understand with a minimum of training efforts required. Therefore, the model can be used by the database designer to communicate the design to the end user.
- 4. Design tool**
E-R model can also be used as a design plan by the database developer to implement a data model in specific database management software.
- 5. Easy conversion to other models**
E-R diagrams can be easily converted to a network or hierarchical data model.

6. Graphical representation

E-R model provides graphical and diagrammatical representation of various entities, their attributes and relationships between entities.

7. Easy to modify

Modifications to E-R diagram at later stage is relatively easier than in other models.

Components of E-R Model

Following figure shows various components of E-R model:

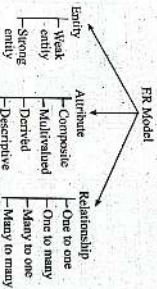


Figure: Components of ER model

2.2.1 Entity and Entity Set

Entity

An entity is a "thing" or "object" in the real world that is distinguishable from all other objects.

Example:

Each person in a university is an entity. An entity has a set of properties, and the values for some set of properties must uniquely identify an entity. For instance, a person may have a person id property whose value uniquely identifies that person. Thus, the value 677-89-9011 for person id would uniquely identify one particular person in the university. Similarly, courses can be thought of as entities, and course id uniquely identifies a course entity in the university. An entity may be concrete, such as a person or a book, or it may be abstract, such as a course, a course offering, or a flight reservation.

Entity Set

A set of entities of same type that share the same properties are referred as entity set. All entities in an entity set have the same set of attributes.

Example:

Set of all students, persons, companies, trees.

Instructor and student is show below:

Instructor (Instructor_ID, Instructor_name)		Student (Student_ID, Student_name)	
Instructor_ID	Instructor_name	Student_ID	Student_name
76766	Chick	98988	Tanaka
45565	Kaiz	12345	Shankar
10101	Srinivasan	00128	Zhang
98345	Kim	76543	Brown
76543	Singh	76653	Aoi
22222	Einstein	23121	Clavez
		44553	Peltier

Figure: Entity sets instructor and student

Following are the types of Entity sets:

1. Weak Entity Set

Entity set which does not have sufficient attributes to form a primary key is known as weak entity set. It depends on another entity and does not contain any key attribute of its own. However, it contains a partial key called as a discriminator. Discriminator is represented by underlining with a dashed line. It is partially unique and can be combined with other strong entity set to uniquely identify the tuples.

Weak entity set is represented by double rectangles and the member of weak entity set is called as subordinate entity.

The primary key of weak entity set is combination of partial key and primary key of the strong entity set. The participation of weak entity type is always total. The relationship between weak entity type and its identifying, strong entity type is called identifying relationship and it is represented by double diamond.

Example:

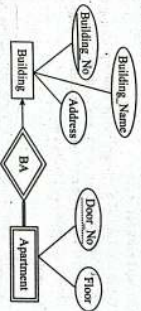


Figure: Weak entity: Apartment

In the above E-R Diagram,

- > One strong entity set "Building" and one weak entity set "Apartment" are related to each other.
- > Strong entity set "Building" has building number as its primary key.
- > Door number is the discriminator of the weak entity set "Apartment".
- > This is because door number alone cannot identify an apartment uniquely as there may be several other buildings having the same door number.
- > Double line between Apartment and relationship set signifies total participation.
- > It suggests that each apartment must be present in at least one building.
- > Single line between Building and relationship set signifies partial participation.
- > It suggests that there might exist some buildings which has no apartment.

Here to uniquely identify any apartment,

- > First, building number is required to identify the particular building.
- > Secondly, door number of the apartment is required to uniquely identify the apartment.
- > Thus, Primary key of Apartment = Primary key of Building + its own discriminator.

2. Strong Entity Set

Entity set that has a primary key is known as strong entity set. It has its own primary key and attributes and is represented by single rectangle. The member of strong entity set is called as dominant entity set.

Example:

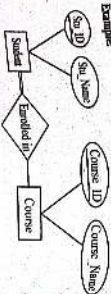


Figure: Strong entity

In the above ER diagram,

- > Two strong entity sets "Student" and "Course" are related to each other.
- > Student ID and Student name are the attributes of entity set "Student".
- > Student ID is the primary key using which any student can be identified uniquely.
- > Course ID and Course name are the attributes of entity set "Course".
- > Course ID is the primary key using which any course can be identified uniquely.
- > Double line between Student and relationship set signifies total participation.
- > It suggests that each student must be enrolled in at least one course.

- > Single line between Course and relationship set signifies partial participation.
- > It suggests that there might exist some courses for which no enrollments are made.

Comparison between weak entity set and strong entity set.

Strong Entity Set	Weak Entity Set
• Strong entity set always has primary key.	• Weak entity set has partial discriminator key.
• Strong entity set is not dependent of any other entity.	• Weak entity set is dependent on strong entity.
• Strong entity set is represented by single rectangle.	• Weak entity set is represented by double rectangle.
• Relationship between two strong entity set is represented by single diamond.	• While the relation between one strong entity set and one weak entity set is represented by double diamond.
• Strong entity set has either total participation or not.	• Weak entity set always has total participation.

2.2.2 Attributes and Keys

Attributes are the descriptive properties of an entity set. Ellipse is used to represent an attribute.

Example:

Name, BirthDate, Roll No, ID, Age, Contact Number, etc. can be attributes of a student entity set.

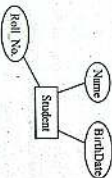


Figure: Attributes of entity student

More examples of entity and attribute:

customer (customer_id, customer_name, customer_address, customer_city)
loan (loan number, amount)
Here customer and loan are the entities whereas other are the attributes of particular entity.

Simple and Composite Attributes

- Simple or atomic attributes are those attributes which cannot be divided into subparts.

Example:

student_id, phone_no are simple attributes

- Composite attributes are those attributes which can be divided into subparts. They are composed of several basic attributes.

Example:

Name can be divided into first name and last name. So Name is composite attribute.

In the following figure Roll_No and BirthDate are simple attributes whereas Name is composite attribute.

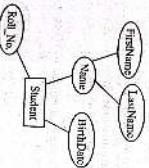


Figure: Simple and composite attributes

Single-Valued and Multi-Valued Attributes

- Singled valued attributes can have only one value. It is represented by single ellipse.

Example:

id, roll, citizenship no, etc. Most attributes are single valued for particular entity.

- Multi-valued attributes can have more than one values. It is represented by double ellipse.

Example:

phone number, college degree, address etc.

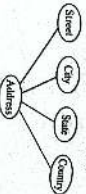


Figure: Multi-valued attribute: Address

Address is multi-valued attribute in above figure.

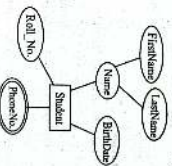


Figure: Composite, single-valued and multi-valued attribute

In above figure:

- Name is composite attribute.
- BirthDate, Roll_No, is single-valued attribute.
- PhoneNo, is multi-valued attribute.

Derived Attributes

Attributes whose values are calculated from the values of other related attributes are called derived attributes. Derived attributes are represented by dashed ellipse.

Example:
Emp, SSN, Emp_No

3. Primary Key

Primary key is a candidate key that the database designer selects while designing the database.

Example:

We can say that either (Emp, SSN) or (Emp, no) can be chosen as a primary key for above schema.

NOTE: Any entity set can have more than one candidate key but only one primary key.

4. Alternate Key

All the candidate keys other than Primary Key are known as Alternative Keys.

5. Foreign Key

It is an attribute in any entity set which is also a primary key in any other entity set.

2.2.3 Relationship and Relationship Set

A relationship is the association among several entities. It connects different entities through a meaningful relation. A relationship is represented by diamond shape in ER diagram. Relationship is classified in terms of degree, connectivity, cardinality, and existence.

Example:

44553 (Pfeifer) advisor 22222 (Einstein)
student entity relationship instructor entity

Relationship Set

A relationship set is a mathematical relation among $n \geq 2$ entities, each taken from entity sets. If $E_1, E_2, \dots, E_n$ are entity sets, then a relationship set R is a subset of:

$\{(e_1, e_2, \dots, e_n) \mid e_i \in E_i, e_i \in E_1, \dots, e_i \in E_n\}$

where $(e_1, e_2, \dots, e_n)$ is a relationship

Example:
(44553, 22222) $\in$ advisor is the relationship,
i.e. Relationship Set *advisor*

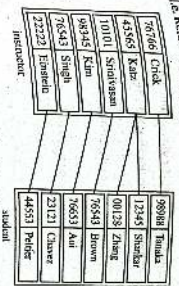


Figure 9: Relationship set *advisor*

The function that an entity plays in a relationship is called that entity's role. Since entity sets participating in a relationship are generally distinct, roles are implicit and are not usually specified. However, they are useful when the meaning of a relationship needs clarification. Such is the case when the entity sets of a relationship set are not distinct, that is, the same entity set participates in a relationship set more than once, in different roles. In this type of relationship set, sometimes called a recursive relationship set, explicit role names are necessary to specify how an entity participates in a relationship instance. For example, the employee entities *manager* and *worker* are called roles; they specify how employee entities interact via the works for relationship set. Roles are indicated in E-R diagrams by labeling the lines that connect diamonds to rectangles.

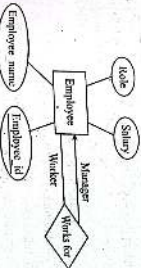


Figure 10: E-R diagram with role indicators.

Degree of Relationship Set

The degree of relationship set is defined as the number of entity sets associated with the relationship sets.

There are three types of degree of relationship:

1. Unary or Recursive Relationship Set (Degree=1)

Unary relationship set is a relationship set where only one entity set participates in a relationship set.

Example:

One person is married to only one person.

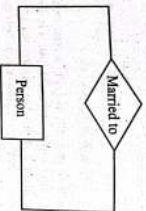


Figure: Unary relationship set

2. Binary Relationship Set (Degree=2)

Binary relationship set is a relationship set where two entity sets participate in a relationship set.

Example:

Student enrolled in Course.



Figure: Binary relationship set

3. Ternary Relationship Set (Degree=3)

Ternary relationship set is a relationship set where three entity sets participate in a relationship set.

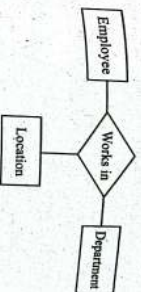


Figure: Ternary relationship set

4. n-ary Relationship Set (Degree=n)

n-ary relationship is a relationship where 'n' entities participate in a relationship.

Mapping Cardinalities

The number of times an entity of an entity set participates in a relationship is known as mapping cardinalities or cardinality ratios. It is most useful in describing binary relationships. Cardinality can be of different types:

- > One-to-one (1:1)
- > One-to-many (1:N)
- > Many-to-one (N:1)
- > Many-to-many (N:N)

One-to-One

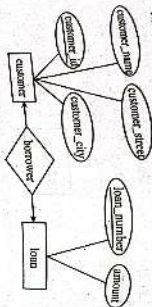
A one-to-one relationship is when at most one instance of an entity (say A) is associated with one instance of another entity (say B).



Example:

A customer is associated with at most one loan via the relationship borrower.

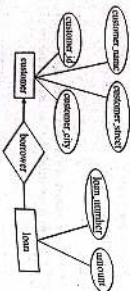
A loan is associated with at most one customer via borrower.

**One-to-Many**

A one-to-many relationship is when one instance of entity A is associated with many instances of entity B.

**Example:**

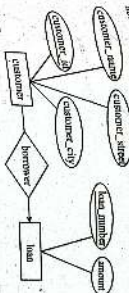
In the one-to-many relationship, a loan is associated with at most one customer via borrower, a customer is associated with several (including 0) loans via borrower.

**Many-to-One**

A many-to-one relationship is when many instances of entity A is associated with one instance of entity B.

**Example:**

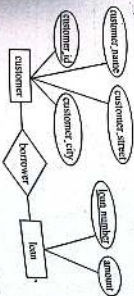
In a many-to-one relationship, a loan is associated with several (including 0) customers via borrower, a customer is associated with at most one loan via borrower.

**Many-to-Many**

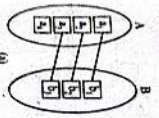
A many-to-many relationship is when one or many instances of entity A is associated with one or many instances of entity B.

**Example:**

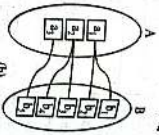
A customer is associated with several (possibly 0) loans via borrower. A loan is associated with several (possibly 0) customers via borrower.



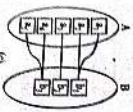
More examples of mapping cardinalities in set format



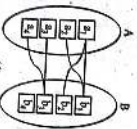
One to one



One to many



Many to one



Many to many

Note: Some elements in A and B may not be mapped to any elements in the other set

Furthermore, we can express these cardinality as following symbols:

Cardinality	Symbols
Many-to-many (*, * or m:n)	
Cardinality ratio = m : n	

Cardinality	Symbol
Many-to-one (*, 1 or m:1)	
OR	
Cardinality ratio = m:1	
One-to-many (1, * or 1:n)	
OR	
Cardinality ratio = 1:n	
One-to-one (1, 1)	
OR	
Cardinality ratio = 1:1	

Further according to Crow's feet notation, we can also express these cardinality as following:

Subclass Entry Type (Subtype)

A subclass entry type is a more specialized entry type. It has a distinct role in the organization. A subclass is a member of a superclass. It is one of the data-modeling abstractions used in ERK. A subclass may be further divided, and, in that case, it is a superclass for its subclasses.

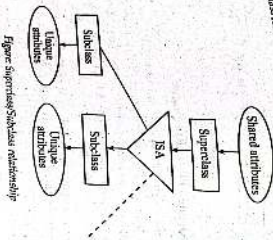


Figure: Supersclass-Subclass relationship

Specialization

Specialization and generalization are opposite to each other. Specialization is a top-down approach. In which one higher level entity can be broken down into two lower-level entities. The idea behind Specialization is to find the subsets of entities that have few distinguish attributes. It is represented by a triangle component labeled ISA, same as generalization. The ISA relationship is also referred to as superclass-subclass relationship. The circle is another symbol for ISA. While using symbol-circle we use subset symbol to denote inheritance.

Example:

Consider an entity employee which can be further classified as sub-entities Technician, Engineer & Accountant because these sub-entities have some distinguish attributes.

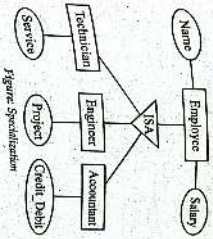


Figure: Specialization

schema

Technician (Name, Salary, Service)
Engineer (Name, Salary, Project)
Accountant (Name, Salary, Credit/Debit)
Here Employee is superclass entry type and Technician, Engineer and Accountant are subclass entry type.

More example of specialization

1. Entity type Employee before specialization

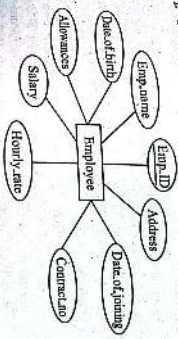


Figure: Employee before specialization

2. Entity type Employee after specialization

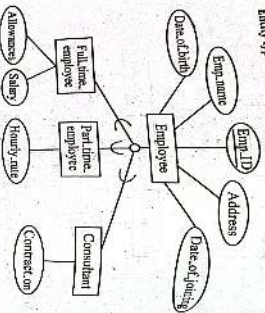


Figure: Employee after specialization

Creation of three subtypes for the EMPLOYEE supertype in above figure, is an example of specialization. The three subclasses have many attributes in common but there are also attributes that are unique to each subtype, for example SALARY and ALLOWANCES for the FULL-TIME-EMPLOYEE.

Generalization

Generalization is a bottom-up approach in which two or more lower-level entities with some common attributes combine to form a higher-level entity. It is represented by a triangle component labeled ISA (E.g., Instructor "is a person").

Example:

Let's say we have two entities Student and Teacher. Attributes of entity Student are: Name, Address & Grade whereas attributes of entity Teacher are: Name, Address & Salary. The ER diagram before generalization looks like this:

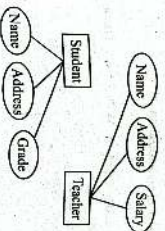


Figure: Before generalization

These two entities have two common attributes: Name and Address, we can make a generalized entity with these common attributes. Let's have a look at the ER model after generalization.

The ER diagram after generalization

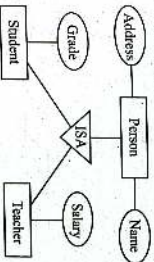


Figure: Generalization

We have created a new generalized entity Person and this entity has the common attributes of both the entities. As we can see in the above ER diagram that after the generalization process the entities Student and Teacher only has the specialized attributes Grade and Salary respectively and their common attributes (Name & Address) are now associated with a new entity Person which is in the relationship with both the entities (Student & Teacher).

More examples of generalization

Example:

(a) Three entity types of namely Car, Truck and Two-wheeler.

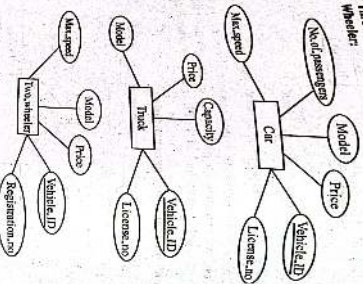


Figure (a) Generalization

(b) Generalization to Vehicle type:

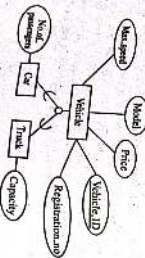


Figure (b) Generalization

Another example of generalization is shown in above figure (b). As shown in above figure (a), three entity types are defined as Car, Truck and Two-wheeler. After analysis it is observed that these entities have a number of common attributes such as REGISTRATION-NO, VEHICLE-ID, MODEL, PRICE and MAX-SPEED. This fact basically indicates that each of these three entity types is a version of a general vehicle type. Above figure(b) illustrates the generalized model of type VEHICLE together with the resulting entity type VEHICLE relationships. The entity CAR has the supertype/subtype relationships. The entity TRUCK has the specific attribute as NO-OF-PASSENGERS, while the entity TWO-WHEELER has specific attribute as CAPACITY. Thus, type TRUCK has allowed to group entity types along with their common attributes, while at the same type preserving their common attributes that are specific to each subtype.

Comparison between Generalization and Specialization

Generalization	Specialization
Generalization extracts the common features of multiple entities to form a new entity.	Specialization splits an entity to form multiple new entities that inherit some feature of the splitting entity.
It is bottom-up approach.	It is top-down approach.
The higher-level entity must have lower-level entities.	The higher-level entity may not have lower-level entities.
Generalization reduces the size of a schema.	Specialization increases the size of a schema.
Generalization is applied on group of entities.	Specialization is applied on a single entity.
Generalization results in forming a single entity from multiple entities.	Specialization results in forming the multiple entity from a single entity.

Aggregation

Aggregation is an abstraction process in which a relationship set is considered as higher-level entity set. In other words, aggregation is a process when relation between the entities is treated as a single entity.

Example:

Consider an example of ternary relationship having the entity sets Employee, Job and Branch with relationship Works-on as shown in Figure a. The information about Managers on employee, managers of particular jobs and different branches can be taken easily.

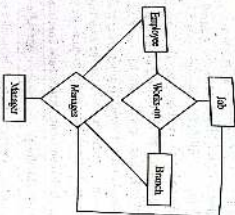


Figure (A): Before aggregation

We cannot combine Works-on and managers relationship sets because some workers are not managers. Using aggregation, Works-on relationship set acts as higher entity set and solve this drawback of E-R Model. E-R Model with aggregation is shown in Figure:

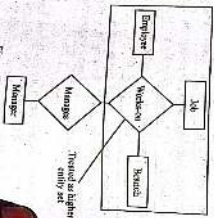


Figure (B): After aggregation

More Examples of Generalization and Specialization

Consider the example of a bank as shown in figure below in which Person entity is superclass entity type which is further divided into Employee and Customer entities. Here Employee and Customer entities are subclass entity type.

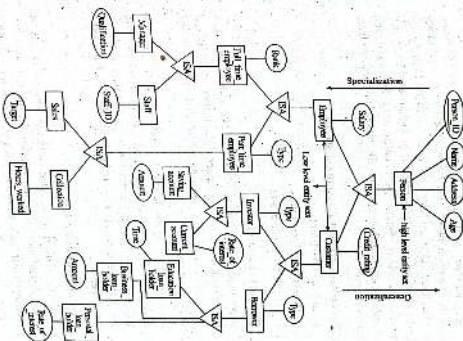


Figure: Specialization and generalization

Basic concept is that by E-R model, a person belongs to a bank is known, and by ERN how it belongs to bank is known

means, what is the exact relationship between bank and the person. A person may be employee or customer which can be further categorized into Manager, Staff or Investor, Borrower and so on.

Attribute Inheritance

Specialization and generalization lead to attribute inheritance between higher level entity set and lower level entity set. Inheritance is a process by which lower-level entity set inherits (or takes) some properties of its higher-level entity set. Consider the above figure. Here entity sets Employee and Customer inherits attributes Person ID, Name, Address, Age from Person entity set.

Specialization and generalization constraints

- Disjoint Constraints
- Overlapping Constraints
- Completeness constraint

2. Disjoint Constraints

According to disjoint constraint if the subclasses of a specialization/generalization are disjoint then an entity can be a member of only one subclass of that specialization/generalization.

Consider above figure, subclasses Full Time Employee and Part Time Employee of superclass Employee are disjoint. Suppose any employee. These classes are disjoint employees for bank that it can only belong to subclass Part Time employee.

3. Overlapping Constraints

According to overlapping constraint, the same entity may be a member of more than one subclass of the specialization. Overlap constraint is shown by placing an o in specialization circle.

Example:

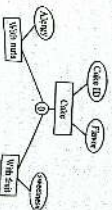


Figure: Overlapping Constraints

In a bakery, cakes can be made with nuts or with fruits. However, a cake can also be made with nuts and fruits at the same time.

c. Completeness Constraints

The completeness constraint may be either total or partial.

A total specialization constraint specifies that every entity that belongs to a superclass must belong to at least one subclass of the specialization. In EER, a total specialization is shown by using a double line that is going out of the superclass to connect the super class to the circle (this relation is similar to the notation for total participation in a relationship).

A partial specialization constraint specifies that some entities might not belong to any of the subclasses of the specialization. In EER, a single line going out of the superclass is used to display a partial specialization, meaning that an entity need not belong to any of the subclasses.

Disjoints and completeness constraints are independent, so we might have the following combinations of specializations:

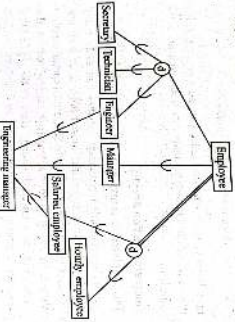
- > Disjoint, total
- > Overlapping, total
- > Overlapping, partial

If we have a subclass that has multiple super classes, then the hierarchy is called a specialization lattice.

i.e. Multiple superclass/subclass connections for the same subclass. Having more than one superclass for the same subclass is also called multiple inheritance.

Example:

Engineer manager is an engineer, a manager, and a salaried employee at the same time. Here, Engineer manager has 3 superclasses (Manager, Engineer, Salaried Employee).



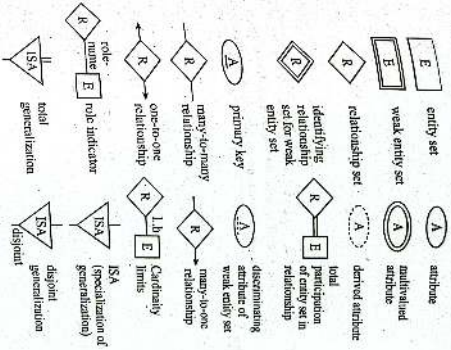
Here, we have 2 levels of superclass/subclass connectors

2.2.5 Entity – Relationship Diagram

An entity relationship diagram (ERD) shows the relationship of entity sets of a database. E-R diagrams represents the logical structure of a database. Following are the steps to create an ER diagram:

- Entity identification (Usually nouns)
- Relationship identification (usually verbs)
- Cardinality identification (1 to 1 or 1 to N)
- Identity attribute (Mention all types of attributes)
- Create ERD

Symbols used in E-R diagrams are shown below:



Example:

1. Make an E-R diagram having two entity sets, Customer and Item. Cardinality Ratio is many to many because a customer can buy any number of items and same item can be purchased by more than one customer.

Journal of the American Medical Association
 1964; 191: 100-101



... to the ...

As a result of limitations in the hierarchical model, designers developed the Network Data Model. The ability of this model to handle many to many ($N-N$) relations between its records is the

main distinguishing feature from the hierarchical model. The main model permits a child record to have more than one parent. In this model, directed graphs are used instead of tree structure. In this model, a node can have more than one parent.

Network model is based on graph structure. A network database consists of collection of records, which are connected to each other by links.

Record

A record is a collection of attributes; each contain only one data value.

Link

A link is an association between two records.

Comparison of three models

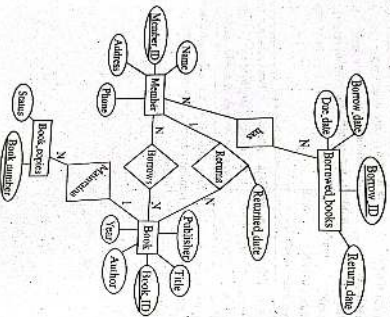
Hierarchical Model	Network Model	Relational Model
It is based on tree structure.	It is based on graph structure.	It is based on mathematical concept of relation.
Hierarchical database consists of collection of records, connected to each other by links.	Network database consists of records, connected to each other by links.	Relational database consists of tables of data and data is stored in tabular form.
It is easy to understand and more efficient than Network data model.	It offers more flexibility than hierarchical model.	It offers simplicity in representing data than network and hierarchical model.
The hierarchical structure is asymmetric and is a major drawback that leads to unnecessary complications for the user.	The network structure is more symmetric than hierarchical model.	The relational structure is more symmetric than network and hierarchical structure.

Hierarchical Model	Network Model	Relational Model
Queries are easy to write than in network model but more complicated relational model.	Queries are more complicated than hierarchical and relational model.	Queries are easy to write than other models.
Information replicated hierarchical leads to inconsistency and wastage of space.	Is offers more data in consistency than model hierarchical model.	It offers more data consistency than other two models.
It is difficult to access values at lower level.	Data accessing is more easier than navigation is easy hierarchical model.	Data accessing and navigation is easy than other models.
Structural independence is missing.	Structural independence is missing.	It offers structural independence.

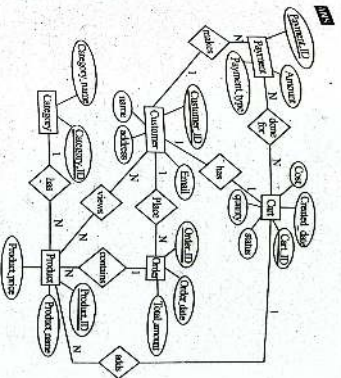
SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS

1. Construct an E-R diagram for keeping records for Library Management Systems. [2019 Spring, 2023 Spring]

Ans

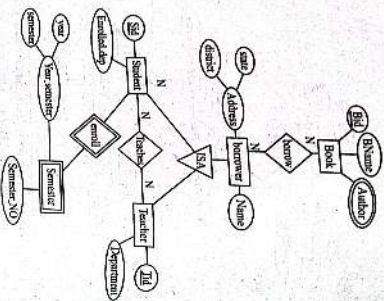


2. Draw an E-R diagram for online shop management system. Assume relevant entities and attributes for the given system. [2022 Fall]



3. Differentiate between Data Model and ER model. Draw an E-R diagram for a Library Management System including primary key, weak entity, composite attribute, derived attribute and multivalued attribute in your ER diagram. [2021 Fall]

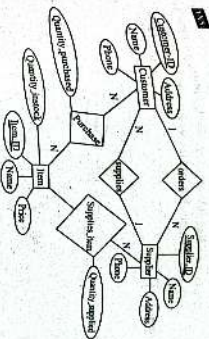
The ER model is a particular kind of data model that is concentrated on defining entities, attributes, and relationships in relational database design. A data model is a general word that incorporates numerous sorts of models used to describe data.



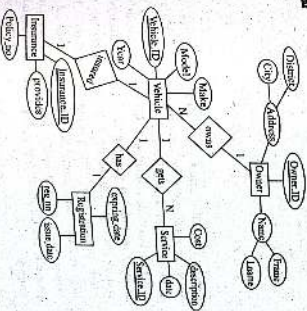
4. Draw an ER diagram for a library system. Assume the entities: student, teacher, book and semester. In the diagram illustrate the concept of strong entity, weak entity, composite attributes, multivalued attributes and derived attributes.

Refer Q.No. 3 Solution.

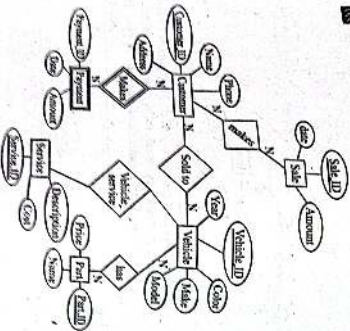
5. Draw E-R diagram for a departmental store that keeps the information about customer, supplier and item. [2011 Spring]



6. Draw an E-R Diagram for a Vehicle Management System including primary key, weak entity, composite attribute, derived attribute and multivalued attributes in your ER Diagram.

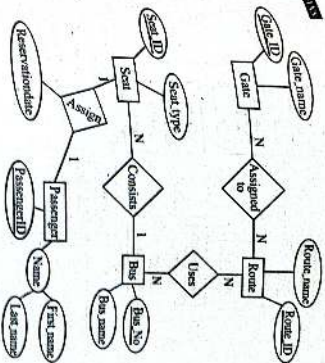


7. Draw an E-R diagram for a Gandaki Auto Vehicle Shop system including primary key, weak entity, composite attribute and multivalued attributes in your ER diagram. [10]



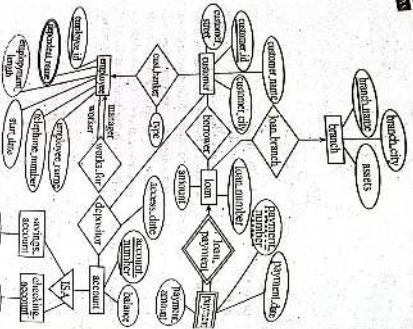
8. Construct an ER diagram for a Metropolitan Bus Park. There are many gates for entering, bus park. Different gates are assigned to different routes. A route uses different buses. Bus consists of different seats which are assigned to different passengers. Frequent travelers are also in passenger. Associate a log of reservation date two attributes first name & last name. Each of the entities must have primary key attribute as far as possible. The cardinality mappings should be explained properly. [10]

10



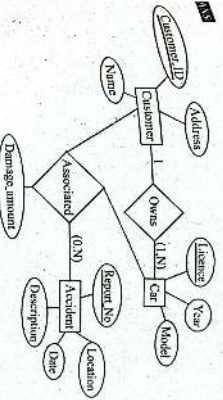
9. Consider the following relations for a database that keeps track of student enrollment in courses and the books adopted for each course:
 STUDENT(SSN, Name, Major, Bdate)
 COURSE(Course#, Cname, Dept)
 ENROLL(SSN, Course#, Quarter, Grade)
 BOOK ADOPTION(Course#, Quarter, Book ISBN)
 TEXT(Book ISBN, Book Title, Publisher, Author)
 Draw a relational schema diagram specifying the foreign keys for this schema. [10]

2018-11-14



13. Construct E-R model for a car insurance company whose customers own one or more cars each. Each car has associated with it zero to any number of recorded accidents. Also design relational database corresponding to the E-R diagram.

14. Suppose you are given the following requirements for a simple database for the National Cricket League (NCL):



Construct a clean and concise ER diagram for the NCL database. List your assumptions and clearly indicate the cardinality mappings as well as any role indicators in your ER diagram.

RELATIONAL MODEL

3.1 Definitions and Terminology

Relation

In a relational database, the table is known as relation because it stores the relation between data in row-column format.

Attribute

The columns are the table's attributes while the rows represent the data records.

Tuple

A single row is known as a tuple to database designers.

Base Relation

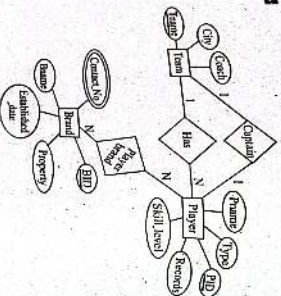
Relation that are physically stored on database are called base relations.

Derived Relation

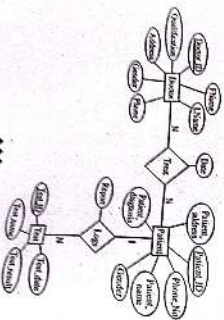
Some relations do not physically exist on database but provide different ways of looking at the base relations known as derived relations. They are also called virtual relation.

3.2 Structure of Relational Databases

A relational database is made up of a number of tables, each of which has a unique name. For example, consider the instructor table of Figure a, which stores information about instructors. The table has four column headers: ID, name, dept name, and salary. Each row of this table records information about an instructor, consisting of the instructor's ID, name, dept name, and salary. Similarly, the course table of Figure b stores information about courses, consisting of a course id, title, dept name, and credits, for each course. Note that each instructor is identified by the value of the column ID, while each course is identified by the value of the column course id.



15. Draw an ER diagram for the database of a hospital with a set of patients and a set of medical doctors. With each patient a log of the various tests conducted is also associated. Make your assumption if necessary. [20/11/19]



ID	name	dept_name	salary
110	Sqjan	Comp. Sci.	68000
121	Sqjan	Finance	80000
133	Madhav	Music	50000

Figure a: The instructor relation.

course_id	title	dept_name	credits
BI0678	Intro to Biology	Biology	4
BI0680	Genetics	Biology	4
CS234	Intro. To Comp. Sci.	Comp. Sci.	4
CS240	Image Processing	Comp. Sci.	3
FIN987	Financial Accounting	Finance	3

Figure b: The course relation.

Figure c shows a third table, prereq, which stores the prerequisite courses for each course. The table has two columns: course_id and prereq_id. Each row consists of a pair of course identifiers such that the second course is a prerequisite for the first course. Thus, a row in the prereq table indicates that two courses are related in the sense that one course is a prerequisite for the other. As another example, when we consider the table instructor, a row in the table can be thought of as representing the relationship between a specified ID and the corresponding values for name, dept_name, and salary values.

course_id	prereq_id
BI0680	BI078
CS240	CS234

Figure c: The prereq relation.

In general, a row in a table represents a relationship among a set of values. We can see that the relation instructor has four attributes:

ID, name, dept_name, and salary.

We use the term relation instance to refer to a specific instance of a relation, that is, containing a specific set of rows.

The instance of instructor shown in Figure a has 3 tuples, corresponding to 3 instructors.

The order in which tuples appear in a relation is irrelevant. The order in which attributes appear in a relation is irrelevant, since a relation is a set of tuples. Thus, whether the tuples of a relation are listed in sorted order or are unsorted, as does not matter: the relations in the both case are the same, since both contain the same set of tuples.

For each attribute of a relation, there is a set of permitted values, called the domain of that attribute. Thus, the domain of the salary attribute of the instructor relation is the set of all possible salary values, while the domain of the name attribute is the set of all possible instructor names.

3.3 The Relational Algebra

The relational algebra is a procedural query language. It consists of a set of operations that take one or two relations as input and produce a new relation as their result. The fundamental operations in the relational algebra are: select, project, union, set difference, cartesian product, rename, select, project, and rename operations are called unary operations, because they operate on one relation. Union, set difference and Cartesian product operate on pairs of relations and are, therefore, called binary operations. Operations such as set intersection, natural join, and assignment are defined in terms of fundamental operations.

Schema of University Database

Classroom (building, room number, capacity)

department (dept_name, building, budget)

course (course_id, title, dept_name, credits)

instructor (ID, name, dept_name, salary)

teaches (ID, course_id, sec_id, semester, year)
 section (course_id, sec_id, semester, year, building, room, number, time_slot_id)

Schema of Northwind Database

dept(deptname, d_head)
 emp(emp_id, deptno, empname, position, salary)

SELECT Operation

The select operation selects tuples that satisfy a θ predicate. Lowercase greek letter sigma (σ) is used to denote selection. The predicate appears as a subscript to σ . The argument relation is in parentheses after the σ .

Syntax:

$\sigma_p(r)$

σ denotes selection 'p' denotes predicate 'r' denoting relation/table

Example:

1. Find the record of IT department

$\sigma_{deptname = 'IT'}(dept)$

Here, deptname = 'IT' is a predicate (p) and dept is a relation (r)
 50000

$\sigma_{deptname = 'emp'}(emp)$

In general, we allow comparisons using =, <, <=, >, >=, and > in the selection predicate. Furthermore, we can combine several predicates into a larger predicate by using the connectives AND ($\wedge$), OR ($\vee$), and NOT ($\neg$).

2. Find the record of employees who works in department 1 and salary is above 50000

$\sigma_{deptname = 1 \wedge salary > 50000}(emp)$

Project Operation

Project operation is denoted by the uppercase greek letter pi (π). This operator helps to keep specific columns from a relation

and discards the other columns. Desired column list appears in the subscript to π . Project operation is used to filter columns from a table.

Syntax:

$\pi_{A_1, A_2, A_3, \dots, A_n}(R)$, where A_1, A_2, A_3 are columns of relation R

Example:

1. Find id, name and salary of employee from all the tuples of emp table.

$\pi_{emp_id, emp_name, salary}(emp)$

2. Find name and salary of employees who work in department no. 1

$\pi_{emp_name, salary}(\sigma_{deptname = 1}(emp))$

Rename Operation

Rename operation is denoted with small greek letter rho (ρ). The results of relational algebra operation are also relations but without any name. The rename operation allows to rename result of such expression, $\rho_A(E)$, where the result of expression E is saved with name of A. It is also used to rename existing relation, $\rho_A(r)$, where r is an existing relation on database. It is also used to rename attributes of a result.

Syntax:

$\rho_{A_1, A_2, A_3, \dots, A_n}(E)$

Returns the result of expression E under name X and attributes renamed to A1, A2, A3, ... An

Union Operation

It performs binary union between two given relations and defined as:

$r \cup s = \{t \mid t \in r \vee t \in s\}$

Notation:

$r \cup s$

Where r and s are either database relations or relation result set (temporary relation).

For a union operation to be valid, the following conditions must hold:

- > r and s must have same number of attributes.
- > Attribute domains must be compatible.
- > Duplicate tuples are automatically eliminated.

Example:

Find the set of all courses taught in the Fall 2009 semester, the Spring 2010 semester, or both.

Step 1:

To find the set of all courses taught in the Fall 2009 semester, we can write:

$\Pi_{course_id} (\sigma_{semester = 'Fall' \wedge year = 2009} (section))$

Step 2:

To find the set of all courses taught in the Spring 2010 semester, we write:

$\Pi_{course_id} (\sigma_{semester = 'Spring' \wedge year = 2010} (section))$

To answer the above query, we need the union of these two sets.

Step 3:

$\Pi_{course_id} (\sigma_{semester = 'Fall' \wedge year = 2009} (section)) \cup$

$\Pi_{course_id} (\sigma_{semester = 'Spring' \wedge year = 2010} (section))$

Set Difference Operation

The set difference operation is denoted by Minus (-). It allows to find tuples in one relation but not in another relation. The expression $r - s$ produces a relation containing those tuples in r but not in s .

Syntax:

r - s

Where r and s are either database relations or relation result set (temporary relation).

For a union operation to be valid, the following conditions must hold:

- > The relations r and s must be union-compatible
- > The schema of the result is defined to be identical to the schema of R .

Example:

Provide the name of authors who have written books but not articles.

1. $\pi_{author} (Books) - \pi_{author} (Articles)$

2. Find the courses offered in the Fall 2009 semester but not in Spring 2010 semester.

We can find all the courses taught in the Fall 2009 semester but not in Spring 2010 semester by writing:

$\Pi_{course_id} (\sigma_{semester = 'Fall' \wedge year = 2009} (section)) -$

$\Pi_{course_id} (\sigma_{semester = 'Spring' \wedge year = 2010} (section))$

Notation:

$r \times s$

Where r and s are relations and their output will be defined as

$r \times s = \{ \langle t \mid t \in r \text{ and } t \in s \rangle \}$

Example:

1. Find tuples from table dept and emp

dept X emp

2. Find name of employee from table dept and emp where department name is IT

$\Pi_{name} (\sigma_{deptname = 'IT'} (dept \times emp))$

Additional operation in RA

Set Intersection ($\cap$)

An intersection is defined by the symbol $\cap$. $r \cap s$ defines a relation consisting of a set of all tuple that are in both r and s . For set intersection to be valid, r and s must be union-compatible.

Example:
Find the set of all courses taught in both the Fall 2009 and the Spring 2010 semesters, using set Intersection.

$\Pi_{course}(\sigma_{semester = 'Fall' \wedge year = 2009}(Section)) \cap$
 $\Pi_{course}(\sigma_{semester = 'Spring' \wedge year = 2010}(Section))$

Note: we can rewrite any relational-algebra expression that uses set intersection by replacing the intersection operation with a pair of set-difference operations as $r \cap s = r - (r - s)$.

Thus, set intersection is not a fundamental operation and does not add any power to the relational algebra. It is simply more convenient to write $r \cap s$ than to write $r - (r - s)$.

Natural Join

The natural join is a binary operation that allows us to combine certain selection operations and a cartesian product into one operation. It is denoted by the join symbol $\bowtie$. The natural-join operation forms a cartesian product of two relations, performs a selection based on equality of those attributes that appear in both the relations, and finally removes duplicate attributes. Natural join can only be performed if there is a common attribute (column) between the relations. The name and type of the attribute must be same.

Example:

1. Find the names of all instructors together with the course id of all courses they taught.

$\Pi_{name, nameid}(\text{instructor} \bowtie \text{teaches})$

2. Find the names of all instructors in the Comp. Sci. department together with the course titles of all the courses that the instructors teach.

$\Pi_{name, title}(\sigma_{deptname = 'Comp. Sci.'}(\text{instructor} \bowtie \text{teaches} \bowtie \text{course}))$

Theta Join Operation

The theta join operation is a variant of the natural-join operation. It allows to combine a selection and a Cartesian product into a single operation. Consider relations r and s , and let

θ be a predicate on attributes in $R \cup S$. The theta join operation between r and s is defined as follows:
 $r \bowtie_{\theta} s = \sigma_{\theta}(r \times s)$

Example:
Find the tuple from dept and emp where department is "IT" and salary is higher than 50000.

$\text{dept} \bowtie_{\theta} \text{emp} \text{ where } \theta = \text{"IT" } \wedge \text{ emp.salary} > 50000$
is equivalent to:

$\sigma_{deptname = \text{"IT"} \wedge emp.salary > 50000}(\text{dept} \times \text{emp})$

Outer Join

Left Outer Join

All the tuples from left relation R are included in the resulting relation. If there are tuples in R without any matching tuple in right relation S then S -attributes of the resulting relation are made NULL.

The left outer join between r and s is denoted by $r \bowtie_{\leftarrow} s$.

Example:

$\text{dept} \bowtie_{\leftarrow} \text{emp}$

Right Outer Join

All the tuples from right relation S are included in the resulting relation. If there are tuples in S without any matching tuple in left relation R then R -attributes of the resulting relation are made NULL.

The right outer join between r and s is denoted by $r \bowtie_{\rightarrow} s$.

Example:

$\text{dept} \bowtie_{\rightarrow} \text{emp}$

Full Outer Join

All tuples from both relations are included in the result irrespective of the matching condition.

The full outer join between r and s is denoted by $r \bowtie_{\leftarrow \rightarrow} s$.

Example:

$\text{dept} \bowtie_{\leftarrow \rightarrow} \text{emp}$

Extended Relational Algebra Operations

Operations that provide the ability to write queries cannot be expressed using the basic relational-algebra operations. These operations are called extended relational-algebra operations.

Generalized Projection

Generalized-projection operation extends the projection operation by allowing operations such as arithmetic and set operations to be used in the projection list. The generalized-projection operation has the form:

$$\pi_{f_1, f_2, \dots, f_n}(E)$$

where E is any relational-algebra expression, and each of $f_1, f_2, \dots, f_n$ is an arithmetic expression involving constants and attributes in the schema of E .

In general, an expression can use arithmetic operations such as $+$, $-$, $*$, and $/$ on numeric valued attributes, numeric constants, and on expressions that generate a numeric result.

Example:

$$\Pi_{empid, salary, avg_sal(emp)}$$

gives the emp_id, eqname and the monthly salary of each instructor

Aggregate Functions:

Aggregate functions take a collection of values and return a single value as a result. Aggregate operation in relational algebra is denoted by G (allgraphic G)

Some example of Aggregate Functions are:

1. SUM

SUM takes a collection of values and returns the sum of the values.

$$G_{sum}(salary)(emp)$$

2. AVG

AVG returns the average of the values.

$$G_{avg}(salary)(emp)$$

3. COUNT

COUNT returns the number of the elements in the collection including duplicate elements.

$$G_{count}(emp)(emp)$$

4. COUNT-DISTINCT

COUNT-DISTINCT Returns distinct no of elements removing duplicate elements.

$$G_{count-distinct}(emp)(emp)$$

5. MIN

MIN return the minimum values in a collection.

$$G_{min}(salary)(emp)$$

6. MAX

MAX return the maximum values in a collection.

$$G_{max}(salary)(emp)$$

Group by Function

Group by function is used to apply aggregate function on group of sets of tuples.

General form:

$$G_{agg}(G_{agg}(A_1)(\Pi(A_1, A_2, \dots, A_n)(\sigma(P(T_1 \times T_2 \times \dots \times T_n))))$$

Example:

1. find the average salary in each department (group by department name)

$$avg_name(G_{avg}(salary)(emp))$$

3.4 Schema and Views

Relation Schema

The name of a relation and set of attributes for a relation is called a schema. The schema for a relation consists of relation name followed by parenthesized list of its attributes.

$\{A_1, A_2, A_3, \dots, A_n\}$ An are attributes then $R = (A_1, A_2, A_3, \dots, A_n)$ is a relation schema.

Example:
cust_schema(customer_name, customer_street, customer_city)

r(R) denotes a relation r on relation schema R.

Example:
customer(cust_schema)

Views:

A view is a tailored presentation of the data contained in one or more tables. In other words, a view is a logical or virtual table, which does not exist physically in the database. Views are also called as dictionary objects. It takes the output of a query and treats it as a table.

3.5 Data Dictionary

A data dictionary is a centralized repository of information about data that is used within an organization. It provides a detailed description of the data elements that are used in a database or other data storage system, including their structure, format, and relationships to other data elements.

In general, a data dictionary contains the following types of information:

- **Data element names and definitions**
This includes the names of all data elements used in the system, as well as their definitions and descriptions.
- **Data element attributes**
This includes any attributes associated with each data element, such as data type, length, format, and validation rules.
- **Data element relationships**
This includes any relationships between data elements, such as foreign keys and other constraints.
- **Metadata**
This includes any metadata associated with the data elements, such as data source, date of creation, and ownership.

Data element usage

This includes any rules or guidelines for using each data element, such as naming conventions and data entry guidelines.

A data dictionary is an important tool for managing and maintaining data within an organization. It ensures that all stakeholders have a consistent understanding of the data being used, which can help to prevent errors and inconsistencies. Additionally, it can be used to automate data integration and transformation processes, which can save time and reduce errors.

SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS

1. Suppose we have the following relation

Employee(person_name, company_name, salary)

Write(person_name, company_name, city)

Company(company_name, city)

Write relational algebraic expressions for the following queries:

- i. List the name and city of employees who work in "Yokhart" and have salary greater than Rs.50,000
- ii. Find the names of all employees who work for "ABC bank".
- iii. Delete all employees who come from "Chitwan"
- iv. Increase salary of all employee by 15% [20 Marks]

Ans

- i. $\sigma_{\text{company_name} = \text{'Yokhart'} \wedge \text{salary} > 50000}(\text{Employee} \bowtie \text{Works})$
- ii. $\Pi_{\text{person_name}}(\sigma_{\text{company_name} = \text{'ABC bank'}}(\text{Works}))$
- iii. $\text{Employee} \leftarrow \text{Employee} - \sigma_{\text{city} = \text{'Chitwan'}}(\text{Employee})$
- iv. $\text{Employee} \leftarrow \text{Employee} \bowtie \sigma_{\text{salary} < 0.15 \cdot \text{salary}}(\text{Employee})$

2.

Suppose we have the following relation:

Employee(person_name, street, city)

Works(person_name, company_name, salary) Company(company_name, city)

Write relational algebra expressions for the following queries:

- i. Find names of all employees who live in "Birtwal"
- ii. Find the names of all employees who work for "Nabil Bank Limited".
- iii. Find the names and cities of residence of all employees who work for "Global bank".
- iv. Update the salary of all employees by 10%. [20 Marks]

Ans

- i. $\sigma_{\text{city} = \text{'Birtwal'}}(\text{Employee} \bowtie \text{Works})$
- ii. $\Pi_{\text{person_name}}(\sigma_{\text{company_name} = \text{'Nabil Bank'}}(\text{Works} \bowtie \text{Employee}))$
- iii. $\Pi_{\text{person_name, city}}(\sigma_{\text{company_name} = \text{'Global Bank'}}(\text{Works}))$
- iv. $\text{Works} \leftarrow \Pi_{\text{person_name, company_name, salary} \cdot 1.10}(\text{Works})$

3. Considering the following schemas:

Sailors(sid, sname, rating, age)

Boats(bid, bname, color)

Reserves(resid, bid, day)

Write relational algebra expressions for the following queries:

- i. Find the records of sailors who have reserved boat number 103 (bid=103).
- ii. Update the color of the boat, where bid is 104, into green.
- iii. Find the names of sailors who have reserved a red or green boat.
- iv. Find the names of sailors who have reserved boat number 103 on day 5.
- v. Find the names of sailors whose name is not "Ram". [20 Marks]
- vi. Find the names of all boats.

Ans

- i. $\sigma_{\text{bid} = 103}(\text{Reserves}) \bowtie \text{Sailors}$
- ii. $\text{Boats} \leftarrow \sigma_{\text{bid} = 104}(\text{Boats}) \bowtie \Pi_{\text{bid, bname, green}}(\sigma_{\text{bid} = 104}(\text{Boats}))$
- iii. $\Pi_{\text{sname}}((\text{Sailors} \bowtie \text{Reserves} \bowtie \sigma_{\text{color} = \text{'red'} \vee \text{'green'}}(\text{Boats}))$
- iv. $\Pi_{\text{sname}}((\text{Sailors} \bowtie \text{Reserves} \bowtie \sigma_{\text{bid} = 103 \wedge \text{day} = 5}(\text{Reserves}))$
- v. $\sigma_{\text{sname} \neq \text{'Ram'}}(\text{Sailors})$
- vi. $\Pi_{\text{sname}}(\text{Boats})$

4. Consider following relations, where the primary keys are underlined. Give an expression in the relational algebra to express each of the following queries:

Doctor (SSN, FirstName, LastName, Specialty, YearsOfExperience, PhoneNum)

Patient (SSN, FirstName, LastName, Address, DOB, PrimaryDoctor SSN)

Medicine (TradeName, UnitPrice, GenericFlag)

Prescription (Id, Date, Doctor SSN, Patient SSN, Prescription_Medicine (Prescription_Id, TradeName, NumOfUnits))

- List the trade name of generic medicine with unit price less than \$50.
- List the first and last name of patients whose primary doctor named John Smith.
- List the first and last name of doctors whose primary doctors to any patient.
- List the SSN of distinct patients who have Aspirin prescribed to them by doctor named John Smith.

Ans

i. $\Pi_{TradeName}(\sigma_{GenericFlag=1} Medicine < 50 (Medicine))$

ii. $\Pi_{FirstName, LastName}((Patient \bowtie_{Doctor \bowtie \text{John Smith}} Doctor) \bowtie_{PrimaryDoctor} Doctor)$

iii. $\Pi_{SSN, LastName} (Doctor) - \Pi_{SSN, LastName, Medicine} (Prescription \bowtie_{Doctor \bowtie \text{John Smith}} Doctor) \bowtie_{Medicine} Medicine)$

iv. $\Pi_{SSN} \left(\sigma_{Prescription_Medicine = Aspirin} (Prescription \bowtie_{Doctor \bowtie \text{John Smith}} Doctor) \right)$

v. $\Pi_{SSN} (Patient \bowtie_{PrimaryDoctor \bowtie \text{John Smith}} Doctor)$

vi. $\Pi_{SSN} \left(\sigma_{Prescription_Medicine = Aspirin} (Prescription \bowtie_{Doctor \bowtie \text{John Smith}} Doctor) \right)$

vii. $\Pi_{SSN} (Patient \bowtie_{PrimaryDoctor \bowtie \text{John Smith}} Doctor)$

viii. $\Pi_{SSN} (Patient \bowtie_{PrimaryDoctor \bowtie \text{John Smith}} Doctor)$

ix. $\Pi_{SSN} (Patient \bowtie_{PrimaryDoctor \bowtie \text{John Smith}} Doctor)$

x. $\Pi_{SSN} (Patient \bowtie_{PrimaryDoctor \bowtie \text{John Smith}} Doctor)$

xi. $\Pi_{SSN} (Patient \bowtie_{PrimaryDoctor \bowtie \text{John Smith}} Doctor)$

xii. $\Pi_{SSN} (Patient \bowtie_{PrimaryDoctor \bowtie \text{John Smith}} Doctor)$

xiii. $\Pi_{SSN} (Patient \bowtie_{PrimaryDoctor \bowtie \text{John Smith}} Doctor)$

5. Write relational algebra statement for the following schemas (underline indicates primary key):

Employee (Emp No, Name, Address)

Project (Proj No, Name, Address)

Workon (Emp No, Proj No)

| 90 | Insights on Database Management Systems

Part (Partno, Part name, Qty on hand)
Use (Emp No, PNo, Parting Number)

- List all the employee details who are not working yet.
- List the partname and Quantity on hand those were used in DBMS project.
- List the name of the projects that are used by employee from London.
- Modify the database so that Jones now lives in USA.
- Update address of an employee Japan to USA.

Ans

a. $\Pi_{Emp No, Name, Address} (Employee - \Pi_{Emp No, PNo, Address} (Workon \bowtie_{Employee} Employee))$

b. $\Pi_{Partname, Qty on hand} ((Part \bowtie_{Use} Use) \bowtie_{Project} Project)$

c. $\Pi_{name} ((Project \bowtie_{Use} Use) \bowtie_{Address = London} Employee)$

d. $Employee \leftarrow (Employee - \sigma_{Name = Jones} (Employee)) \cup \Pi_{Emp No, Name, Address} (\sigma_{Name = Jones} (Employee)) \cup \Pi_{Emp No, Name, Address} (\sigma_{Address = Japan} (Employee))$

e. $Employee \leftarrow (Employee - \sigma_{Address = Japan} (Employee)) \cup \Pi_{Emp No, Name, Address} (\sigma_{Address = Japan} (Employee))$

f. $Employee \leftarrow (Employee - \sigma_{Address = Japan} (Employee)) \cup \Pi_{Emp No, Name, Address} (\sigma_{Address = Japan} (Employee))$

g. $Employee \leftarrow (Employee - \sigma_{Address = Japan} (Employee)) \cup \Pi_{Emp No, Name, Address} (\sigma_{Address = Japan} (Employee))$

h. $Employee \leftarrow (Employee - \sigma_{Address = Japan} (Employee)) \cup \Pi_{Emp No, Name, Address} (\sigma_{Address = Japan} (Employee))$

i. $Employee \leftarrow (Employee - \sigma_{Address = Japan} (Employee)) \cup \Pi_{Emp No, Name, Address} (\sigma_{Address = Japan} (Employee))$

j. $Employee \leftarrow (Employee - \sigma_{Address = Japan} (Employee)) \cup \Pi_{Emp No, Name, Address} (\sigma_{Address = Japan} (Employee))$

k. $Employee \leftarrow (Employee - \sigma_{Address = Japan} (Employee)) \cup \Pi_{Emp No, Name, Address} (\sigma_{Address = Japan} (Employee))$

l. $Employee \leftarrow (Employee - \sigma_{Address = Japan} (Employee)) \cup \Pi_{Emp No, Name, Address} (\sigma_{Address = Japan} (Employee))$

m. $Employee \leftarrow (Employee - \sigma_{Address = Japan} (Employee)) \cup \Pi_{Emp No, Name, Address} (\sigma_{Address = Japan} (Employee))$

n. $Employee \leftarrow (Employee - \sigma_{Address = Japan} (Employee)) \cup \Pi_{Emp No, Name, Address} (\sigma_{Address = Japan} (Employee))$

o. $Employee \leftarrow (Employee - \sigma_{Address = Japan} (Employee)) \cup \Pi_{Emp No, Name, Address} (\sigma_{Address = Japan} (Employee))$

p. $Employee \leftarrow (Employee - \sigma_{Address = Japan} (Employee)) \cup \Pi_{Emp No, Name, Address} (\sigma_{Address = Japan} (Employee))$

q. $Employee \leftarrow (Employee - \sigma_{Address = Japan} (Employee)) \cup \Pi_{Emp No, Name, Address} (\sigma_{Address = Japan} (Employee))$

r. $Employee \leftarrow (Employee - \sigma_{Address = Japan} (Employee)) \cup \Pi_{Emp No, Name, Address} (\sigma_{Address = Japan} (Employee))$

s. $Employee \leftarrow (Employee - \sigma_{Address = Japan} (Employee)) \cup \Pi_{Emp No, Name, Address} (\sigma_{Address = Japan} (Employee))$

- i. Π Express, Π Express (EMPLOYEE $\bowtie$ σ Projection Use by (PROJECT) $\bowtie$ ASSIGNED_TO)))
- ii. Π Express, Π Express (EMPLOYEE $\bowtie$ ASSIGNED_TO $\bowtie$ ProjectionManager="Rohan" (PROJECT)))
- iii. Π Express, Π Express (EMPLOYEE - Π Express, Π Express (ASSIGNED_TO $\bowtie$ EMPLOYEE))
- iv. Π Express, Π Express ((σ Count(Projection) > 1 (Π Empman, Projection (ASSIGNED_TO))) $\bowtie$ EMPLOYEE)

7. Compare and contrast relational algebra with relational calculus.

[2018 Fall, 2014 Fall]

ANS Relational database management systems (RDBMS) use formal languages called relational algebra and relational calculus to express and manipulate data operations and queries. They differ from one another, nevertheless, in approach, syntax, and level of abstraction. Compare and contrast them as follows:

Relational Algebra

- i. **Syntax:** Relational algebra is a procedural query language that manipulates relations through the use of operators like selection, projection, union, intersection, difference, join, and division. It concentrates on the procedure or series of steps.
- ii. **Expressiveness:** Compared to relational calculus, relational algebra is a more expressive language. It may describe different operations and transformations on relations and supports complicated queries.
- iii. **Implementation:** The actual physical implementation of a database is more similar to relational algebra. It can be converted into a series of fundamental activities that a database management system can effectively carry out.
- iv. **Query Optimization:** The clear foundation provided by relational algebra makes query optimization possible. Its optimization strategies including performing selection

and projection operations early in the query evaluation process and reordering operations.

Ease of Use: In terms of understanding and application, relational algebra is simpler to formulate and optimize queries. It is frequently used in relational algebra-based query languages like SQL.

Relational Calculus

i. Syntax: Relational calculus is a non-procedural query language that indicates what information should be retrieved from a database but not how. To express the needed facts, it makes use of quantifiers and mathematical predicates.

ii. Expressiveness: Less expressive than relational algebra is relational calculus. It emphasizes the selection and filtering of tuples based on conditions and focuses on the logical characteristics and restrictions of the data.

iii. Implementation: Database systems do not directly implement relational calculus. In addition to providing a theoretical framework for query languages, it offers a framework for comprehending the semantics of queries.

iv. Query Optimization: There is no direct foundation for query optimization in relational calculus. The optimization methods are often used at the query execution level or higher-level query language level.

v. Ease of Use: Relational calculus is better suited for expressing sophisticated constraints or logical characteristics of data and is more appropriate for theoretical or academic uses. It may not be as natural for the formulation of practical queries as relational algebra, and it necessitates a solid understanding of mathematical logic.

Relational algebra and relational calculus are two distinct approaches to querying and manipulating relational databases. Relational algebra is procedural, expressive, and closer to the physical implementation, making it more suitable for practical database management systems. On the

other hand, relational calculus is non-procedural, emphasising logical properties, and serves as a theoretical foundation for query languages.

8. Define TRC and DRC.

[2 Marks]

Ans Tuple Relational Calculus (TRC) and Domain Relational Calculus (DRC) are the two basic subtypes of relational calculus.

Tuple Relational Calculus (TRC)

- A non-procedural query language called Tuple Relational Calculus describes a collection of tuples that meet a given predicate in order to provide the intended result.
- The formulas are constructed using variables, constants, logical operators, and quantifiers (existential and universal).
- TRC focuses on choosing tuples from a relation that meet a predicate or condition that is given.
- It is rarely utilized as a practical query language and is mostly used for the theoretical research of relational databases.

Domain Relational Calculus (DRC)

- Another non-procedural query language that works with single values rather than tuples is Domain Relational Calculus.
- DRC describes a collection of values for the attributes of the result relation in order to specify the desired result.
- To express the conditions that must be met for the desired values, it uses formulas with variables, constants, logical operators, and quantifiers.
- DRC focuses on selecting individual values from the domains of the attributes in a relation.
- Similar to TRC, DRC is primarily used for theoretical purposes and is not widely used as a practical query language.

9. Consider the relational database model:

Users (uid, cname, city)

Items (itemid, itemname, city, quantity, price)

Manager (mid, aname, city)

Query (queryno, uid, mid, itemid, query_details, hitratio)

Write the relational algebraic expression for the following tasks:

- i. Find all (queryno, uid) pairs for query with a hitratio value greater than 30.
- ii. Find all item names of items in Pokhara ordered with query_details as pokhara_details.
- iii. Find itemids of items ordered through manager 35 but not through manager 27. [2 Marks]

Ans

i. $\Pi_{queryno, uid} (\sigma_{hitratio > 30} (Query))$

ii. $\Pi_{itemname} (\sigma_{city = 'Pokhara'} \Join query_details = 'pokhara_details' (Items \Join Query))$

iii. $\Pi_{itemid} (\sigma_{mid = 35 \wedge mid \neq 27} (Query) \Join Items)$

10. Consider the following relational Schemas:

Department (DepartmentID, DepartmentName)

Designation (DesignationID, DesignationName, Salary)

Allowance (AllowanceID, AllowanceName)

Employee (EmpID, empName, Gender, DesignationID, DepartmentID)

DetailID, EmpID, AllowanceID, Amount)

Write the relational algebraic expression for the following task:

- i. Find the number of employees department-wise.
- ii. List the employee details whose total salary is above Rs. 50000.
- iii. List the employee those who are getting house allowance. [2 Marks]

- i. Count the number of student registered subject in year 2015 gender wise.
- ii. Show student details taught by teacher Rishi Sharma.
- iii. Delete student information taught by teacher N. Mathema.

14. Consider the relational database of Figure below, where the primary keys are underlined. Give an expression in the relational algebra to express each of the following queries:

- i. $\sigma_{\text{Teacher} = \text{N. Mathema}} (\text{Student} \bowtie \text{Registration} \bowtie \text{Course})$
- ii. $\sigma_{\text{Teacher} = \text{N. Mathema}} (\text{Student} \bowtie \text{Registration} \bowtie \text{Course})$
- iii. $\text{Student} \leftarrow \text{Student} \bowtie \sigma_{\text{Teacher} = \text{N. Mathema}} (\text{Student} \bowtie \text{Registration} \bowtie \text{Course})$

15. Consider the relational database of Figure below, where the primary keys are underlined. Give an expression in the relational algebra to express each of the following queries:

- i. Find the names of all employees in this database who live in the same city as the company for which they work.
- ii. Give all employees of First Bank Corporation a 10 percent salary raise.
- iii. Modify the database so that Hrushikesh now lives in Bangalore.
- iv. Delete all tuples in works relation for employee of First Bank Corporation.

16. Consider the relational database of Figure below, where the primary keys are underlined. Give an expression in the relational algebra to express each of the following queries:

- i. $\Pi_{\text{person-name}} (\sigma_{\text{employee} = \text{Hrushikesh}} (\text{employee} \bowtie \text{works} \bowtie \text{bank}))$
- ii. $\text{Works} \leftarrow (\text{Works} \bowtie \sigma_{\text{bank-name} = \text{Nepal Rastra Bank}}) \cup (\text{person-name} \bowtie \text{salary}) \cup (\text{bank-name} \bowtie \text{works})$

- iii. $\Pi_{\text{person-name, salary}} (\sigma_{\text{person-name} = \text{Hrushikesh}} (\text{employee} \bowtie \text{works}))$
- iv. $\text{works} \leftarrow \text{works} \bowtie \sigma_{\text{bank-name} = \text{First Bank Corporation}} (\text{works})$

17. Consider the relational database of Figure below, where the primary keys are underlined. Give an expression in the relational algebra to express each of the following queries:

- i. $\Pi_{\text{person-name, street, city}} (\sigma_{\text{employee} = \text{Hrushikesh}} (\text{employee} \bowtie \text{works} \bowtie \text{bank}))$
- ii. $\Pi_{\text{person-name, salary}} (\sigma_{\text{employee} = \text{Hrushikesh}} (\text{employee} \bowtie \text{works} \bowtie \text{bank}))$

- i. Find the names of all employees who work for Nepal Rastra Bank and Salary greater than \$10,000.
- ii. Find the names and cities of residence of all employees who work for Nepal Rastra Bank.
- iii. Find the names, street address, and cities of residence of all employees who work for Nepal Rastra Bank Corporation and earn more than \$10,000 per annum.
- iv. Delete all tuples in works relation for employee of Nepal Rastra Bank.

18. Consider the relational database of Figure below, where the primary keys are underlined. Give an expression in the relational algebra to express each of the following queries:

- i. $\Pi_{\text{person-name}} (\sigma_{\text{bank-name} = \text{Nepal Rastra Bank}} (\text{works}))$
- ii. $\Pi_{\text{person-name, salary}} (\sigma_{\text{employee} = \text{Hrushikesh}} (\text{employee} \bowtie \text{works} \bowtie \text{bank}))$
- iii. $\Pi_{\text{person-name, salary}} (\sigma_{\text{employee} = \text{Hrushikesh}} (\text{employee} \bowtie \text{works} \bowtie \text{bank}))$
- iv. $\text{works} \leftarrow \text{works} \bowtie \sigma_{\text{bank-name} = \text{Nepal Rastra Bank}} (\text{works})$

19. Consider the relational database of Figure below, where the primary keys are underlined. Give an expression in the relational algebra to express each of the following queries:

- i. $\Pi_{\text{person-name, street, city}} (\sigma_{\text{employee} = \text{Hrushikesh}} (\text{employee} \bowtie \text{works} \bowtie \text{bank}))$
- ii. $\Pi_{\text{person-name, salary}} (\sigma_{\text{employee} = \text{Hrushikesh}} (\text{employee} \bowtie \text{works} \bowtie \text{bank}))$

Bank (bankname city)
Manager (personidname manager_name)

- i. Find the total salary sum of all the banks.
- ii. Modify the database so that Ram now lives in Kathmandu.
- iii. Find the name, street address, and cities of residents of all employees who work for Nepal, World Bank Corporation and earn more than \$10,000 per annum.
- iv. Order all tuples in works relation for employees of South Bank Corporation.

5.2

1. `SELECT SUM(salary) FROM Works`

ii. `Employee -> (Employee -> personidname (Employees))`
`personidname (personidname (Employees))`

iii. `personidname (personidname (Employees))`
`personidname (personidname (Employees))`

iv. `works -> works -> personidname (personidname (works))`

♦♦♦

RELATIONAL DATABASE QUERY LANGUAGES

4.1 Introduction to SQL

SQL stands for Structured Query Language pronounced as "See-Que-El". SQL is the standard language for query or statements as "See-Que-El". SQL is the standard language for accessing and manipulating Relational Databases. It enables a user to create, read, update and delete relational databases and tables. It is a practical implementation of relational algebra and relational calculus. SQL became a standard of the American National Standards Institute (ANSI) in 1986, and of the International Organization for Standardization (ISO) in 1987. Although SQL is an ANSI/ISO standard, there are different versions of the SQL language. Relational databases like MySQL, Database, Oracle, MS SQL Server, Sybase, etc. use ANSI SQL.

Example of SQL:

1. Find the name of the customer with customer_id 7465 from customer table.

```
SQL SELECT *
FROM customer
WHERE customer_id = 7465;
```

```
MySQL SELECT *
FROM customer
WHERE customer_id = 7465;
```

2. Find the balances of all accounts held by the customer with customer_id 7465

```
SQL SELECT accountbalance
FROM depositor accountnumber = accountnumber
WHERE depositorcustomer_id = 7465;
```

4.1.1 Features of SQL

- Following are the features of SQL:
 - SQL is easy to learn.
 - The SQL keywords are case insensitive (SELECT, FROM, WHERE, etc.) but are often written in all caps.

SQL is used to access data from relational database management systems.

SQL can execute queries against the database.

SQL is used to define the data in the database and manipulate it when needed.

SQL is used to create and drop the database and table.

SQL is used to create a view, stored procedure, function, etc.

SQL allows users to set permissions on tables, procedures and views.

4.1.2 SQL Basic Data Types

• **char(n)**

A fixed-length character string with user-specified length. A full-form character(n) can be used instead. Maximum length is 8000 characters.

• **varchar(n)**

A variable-length character string with user-specified maximum length. n: The full form, character(n), varying is equivalent. Maximum length is 8000 characters.

• **int**

An integer is the subset of the integers that is machine dependent. The full form, integer, is equivalent.

• **smallint**

Allows whole numbers between -32768 and 32768.

• **float(p)**

A floating-point number, with precision of at least n digits.

• **numeric(p, d)**

A fixed-point number with user-specified precision. The number consists of digits (data type), and d of the p digits are to the right of the decimal point. Thus, numeric(13,1) allows 14.5 to be stored exactly; the number 454.5 or 0.332 can be stored exactly as a third of this type.

• **decimal(p, d)**

Fixed precision and scale numbers. Allows numbers from $10^p \times 38 + 1$ to $10^{-38} - 1$. p must be a value from 1 to 38. Default value of p is 18. The d parameter indicates the maximum number of digits stored to the right of the decimal point. d must be a value from 0 to p. Default value is 0.

4.1.3 Queries and Sub-Queries

A query is a statement requesting the retrieval of information. The portion of a DML that involves information retrieval is called a query language. The categories of query language are:

1. **Procedural**

User tells system what and how to do.

2. **Nonprocedural**

User tells system what to do.
Relational algebra, tuple relational calculus, domain relational calculus are the four languages that form underlying basis of query languages such as SQL.

Basic Structures of SQL Queries

Queries written in SQL (Structured Query Language) comply to a particular syntax. Multiple clauses are joined to form the functional structure of SQL queries, which are used to obtain data from databases. An outline of the typical structure is provided here:

• **SELECT:** The columns or expressions to obtain from the database table are specified in this clause. It chooses the information that the query will return.

• **FROM:** The table or tables from which the data will be fetched are specified in this clause. It indicates where the data come from.

• **WHERE:** We may specify conditions in this optional clause to filter the data based on certain criteria. By utilizing logical expressions and comparison operators, it assists in reducing the number of outcomes.

1. Union
2. Union All
3. Intersect
4. Minus/Except/Difference

1. Union

It is used to combine the result of two or more SQL **SELECT** queries. To perform this operation, all the number of datatype and columns must be same in both the tables on which **UNION** operation is being applied. It eliminates the duplicate rows from the combined result set.

Syntax:

SELECT column_name **FROM** table1

UNION

SELECT column_name **FROM** table2;

Representation:



Figure: Visual representation of **UNION** operation

Example:

SELECT * FROM sales2005;

UNION

SELECT * FROM sales2006;

sales2005

Person	Amount
Joe	1000
Alex	2000
Bob	5000

sales2006

Person	Amount
Joe	2000
Alex	2000
Zach	35000

Result	
Person	Amount
Joe	1000
Alex	2000
Bob	5000
Joe	2000
Zach	35000

2. Union All

It is almost similar to **UNION** operator except it preserves the duplicate rows from the result set.

Syntax:

SELECT column_name **FROM** table1

UNION ALL

SELECT column_name **FROM** table2;

Representation:



Figure: Visual representation of **UNION ALL** operation

Example:

SELECT * FROM sales2005

UNION ALL

SELECT * FROM sales2006;

sales 2005

Person	Amount
Kamal	1000
Bipin	2000
Arun	5000

sales 2006

Person	Amount
Kamal	2000
Bipin	2000
Dilnath	35000

Result:

Person	Amount
Kamal	1000
Kamal	2000
Bipin	2000
Bipin	2000
Ajayn	5000
Diladeh	35000

Here in this example, Alex 2000 are the duplicate rows.

3. Intersect

The intersect operation returns the common rows from both the SELECT statements. The number of datatype and columns must be the same. Here duplicate rows are removed, and result set is arranged in ascending order by default.

Syntax:

SELECT column_name FROM table1

INTERSECT

SELECT column_name FROM table2;

Representation:

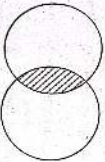


Figure: Pictorial representation of INTERSECT operation

Example:

SELECT * FROM Orders

INTERSECT

SELECT * FROM Sales;

Orders

Id	Items
1	Phone
2	Laptop
3	Camera

Sales

Id	Items
2	Laptop
3	Camera
4	Watch

Result

Id	Items
2	Laptop
3	Camera

4. Minus/ Except/ Difference

Minus operator is used to display the rows which are present in the first query but absent in the second query. It has no duplicates and data arranged in ascending order by default.

Syntax:

SELECT column_name FROM table1

MINUS

SELECT column_name FROM table2;

Representation:

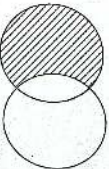


Figure: Pictorial representation of MINUS operation

Example:

SELECT * FROM Orders

MINUS

SELECT * FROM Sales;

Orders

Id	Items
1	Phone
2	Laptop
3	Camera

Sales

Id	Items
2	Laptop
3	Camera
4	Watch

Result

Id	Items
1	Phone

4.1.5 joins

A SQL join statement is used to combine data or rows from two or more tables based on a common field between them. Different types of SQL joins are:

1. INNER JOIN
2. OUTER JOIN
 - a. LEFT JOIN
 - b. RIGHT JOIN
 - c. FULL JOIN

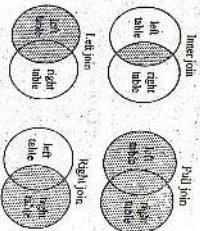


Figure: Different types of SQL join

1. Inner join

Inner join keeps only the information from both tables that's related to each other. The INNER JOIN creates a new result table by combining column values of two tables (table1 and table2) based upon the join condition.

Syntax:

```
SELECT table1.column1, table2.column2 ...
FROM table1 INNER JOIN table2
ON
```

table1.common field = table2.common field;

Example:

```
SELECT *
FROM table1 t1 INNER JOIN table2 t2
ON
t1.value1 = t2.value2;
```

table1		table2	
Key1	Value1	Key2	Value2
1	a	5	a
2	b	6	b
3	c	7	c
4	d	8	d
5	f	9	e

Result			
Key1	Value1	Key2	Value2
1	a	5	a
2	b	6	b
3	c	7	c
4	d	8	d

Self join

It involves the join operation with the table itself. It is basically used where there is any relationship between rows stored in the same table.

Example:

```
SELECT e1Name, e2Name AS Boss
FROM emp e1 INNER JOIN emp e2
```

ON

e1.Boss_id = e2.id;

Emp

Id	Name	Boss Id
1	Manish	3
2	Manoj	1
3	Sushil	2

Result

Name	Boss
Manish	Sushil
Manoj	Manish
Sushil	Manoj

2. Outer join

An outer keeps the information that is not related to the other table in the resulting table. Following are the types of outer join.

a. Right (Outer) join

It selects records from the second (right-hand) table with matching left table records.

Syntax:

SELECT table1.column1, table2.column2 ...
FROM table1 RIGHT JOIN table2

ON

table2.common_field = table2.common_field;

Example:

SELECT *

FROM table1 RIGHT JOIN table2

ON

table1.value1 = table2.value2;

table1

Key1	Value1
1	a
2	b
3	c
4	d
5	f

table2

Key2	Value2
5	c
8	a
7	b
9	d
6	e

[112] [Impdb on Database Management Systems](#)

Result

Key1	Value1	Key2	Value2
1	a	5	a
2	b	6	b
3	c	7	c
4	d	8	d
(NULL)	(NULL)	9	e

b. Left (Outer) join

It selects all the records from left table with matched records from right table.

Syntax:

SELECT table1.column1, table2.column2 ...

FROM table1 LEFT JOIN table2

ON

table1.common_field = table2.common_field;

Example:

SELECT *

FROM table1 LEFT JOIN table2

ON

table1.value1 = table2.value2;

table1

Key1	Value1
1	a
2	b
3	c
4	d
5	f

table2

Key2	Value2
5	a
6	b
7	c
8	d
9	e

[Database System Query Language](#) [113]

Result	Key1	Value1	Key2	Value2
1	a	5	a	
2	b	6	b	
3	c	7	c	
4	d	8	d	
5	e	(NULL)	(NULL)	

c. Full (Outer) join

It selects all records that match either left or right table records. MySQL does not support FULL JOIN, we can use this by using UNION of LEFT and RIGHT join.

Syntax:

SELECT table1.column1, table2.column2 ...

FROM table1 FULL JOIN table2

ON

table1.common_field = table2.common_field;

Example:

SELECT *

FROM table1 FULL JOIN table2

ON

table1.value1 = table2.value2;

table1

Key1	Value1
1	a
2	b
3	c
4	d
5	e

table2

Key2	Value2
5	a
6	b
7	c
8	d
9	e

Result	Key1	Value1	Key2	Value2
1	a	5	A	
2	b	6	B	
3	c	7	C	
4	d	8	D	
5	e	(NULL)	(NULL)	
(NULL)	(NULL)	9	E	

Furthermore, there are more joins which are described below:

1. Natural join

Natural join joins two tables based on same column name and datatypes. Main difference between inner join and natural join is the number of columns returned. The resulting table will contain all the columns of both the table but keeps only one copy common column.

Syntax:

SELECT *

FROM table1 NATURAL JOIN table2;

If several columns have the same names but the data types do not match, the NATURAL JOIN clause can be modified with the USING clause.

Example:

SELECT *

FROM table1 NATURAL JOIN table2;

table1

Key1	Value1
1	a
2	b
3	c
4	d
5	e

table2

Key2	Value1
5	a
6	b
7	c
8	d
9	e

Result

Value1	Key1	Key2
a	1	5
b	2	6
d	4	8
e	3	9

Note: Above Natural join syntax is not valid in Microsoft SQL Server (T-SQL) while PostgreSQL, MySQL and Oracle support natural joins.

2. Cartesian Join

The CARTESIAN JOIN or CROSS JOIN returns the cartesian product of the sets of records from two or more joined tables.

Syntax:

SELECT column_name

FROM table1 CROSS JOIN table2;

Example:

Orders

id	item
1	Phone
2	Laptop

Sales

id	items
2	Laptop
3	Camera

Result

id	item	id	items
1	Phone	2	Laptop
1	Phone	3	Camera
2	Laptop	2	Laptop
2	Laptop	3	Camera

Total no of rows in result:

Orders_row * Sales_row = 2 * 2 = 4

4.2. DDL and DML Queries in SQL

4.2.1 Data Definition Language (DDL)

DDL is used for specifying the database schema. It is used for creating tables, schema, indexes, constraints etc. in database.

> CREATE: To create the database instance / object

> ALTER: To alter the structure of database

> DROP: To drop database instances and tables

> TRUNCATE: To delete tables data in a database instance

> RENAME: To rename database instances

1. Create Operation

a. Create Database

Syntax:

CREATE DATABASE database_name;

Example:

CREATE DATABASE Nepal;

b. Create Table:

Syntax:

CREATE TABLE table_name(

column1 datatype,

column2 datatype,

column3 datatype,

)

Example:

CREATE TABLE student (

s_id INT,

s_name VARCHAR(20),

s_address VARCHAR(20));

2. Alter Operation

Alter operation is used to modify attributes of database objects such as tables. It is used to add, delete or modify the column from table. It is also used to add or drop constraints from tables.

a. Add new column

Syntax:

ALTER TABLE table_name

ADD column_name datatype;

Example:

```
ALTER TABLE student  
ADD date_of_birth DATE;
```

b. Delete existing Column

Syntax:

```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

Example:

```
ALTER TABLE student  
DROP COLUMN date_of_birth;
```

c. Change data type of column:

Syntax:

```
ALTER TABLE table_name  
ALTER COLUMN column_name datatype;
```

Example:

```
ALTER TABLE student  
ALTER COLUMN s_id VARCHAR(20);
```

d. Add constraint

Example:

```
ALTER TABLE student  
ADD CONSTRAINT PK_student PRIMARY KEY (s_id);
```

e. Drop constraint

Example:

```
ALTER TABLE student  
DROP CONSTRAINT PK_student;
```

3. Drop Operation

It is used to delete database instance and database objects like Table, View, Index etc.

a. Drop Database

Syntax:

```
DROP DATABASE database_name;
```

Example:

```
DROP DATABASE Nepal;
```

b. Drop Table

Syntax:

```
DROP TABLE table_name;  
  
Example:  
DROP TABLE Student;
```

4. Truncate Operation

It is used to delete the data from table without deleting the table structure.

Syntax:

```
TRUNCATE TABLE table_name;
```

Example:

```
TRUNCATE TABLE student;
```

5. Rename Operation

It is used to rename database instance and Objects.

a. Rename Database:

Syntax:

```
ALTER DATABASE current_database_name MODIFY  
NAME = new_database_name;
```

Example:

```
ALTER DATABASE Nepal MODIFY NAME = 'Nepal college';
```

b. Rename Table:

Syntax:

```
EXEC sp_rename 'old_table_name', 'new_table_name';
```

Example:

```
In MYSQL:  
RENAME TABLE 'emp' TO 'employee';  
  
In SQL SERVER:  
EXEC sp_rename 'emp', 'employee';
```

4.2.2 Data Manipulation Language (DML)

DML is used to accessing and manipulating data in table. DML is also known as query language. The following operations on database comes under DML:

- INSERT: To insert record(s) into the table(s)
- SELECT: To read records from table(s)
- UPDATE: Update the data in table(s)
- DELETE: Delete all the records from the table

1. Insert Operation

a. Inserting into single row:

Syntax:

```
INSERT INTO table_name(column1, column2, column3,...)
VALUES (value1, value2, value3,...);
```

Example:

```
INSERT INTO student(s_id, s_name, s_address)
VALUES (6, 'Shrey', 'Pohchang');
```

Remember:

- There is no need to mention column name and value if it is auto increment type.

Example: If s_id is auto increment type then

```
INSERT INTO student(s_name, s_address);
```

VALUES ('Shrey', 'Pohchang');

- If all the column from table are to be filled with values then so need to mention columns name.

Example:

```
INSERT INTO student;
```

```
VALUES ('Shrey', 'Pohchang');
```

b. Inserting into multiple rows

Syntax:

```
INSERT INTO table_name(column1, column2, column3,...)
```

```
VALUES (value1, value2, value3,...);
```

```
VALUES (value1, value2, value3,...);
```

```
VALUES (value1, value2, value3,...);
```

Example:

```
INSERT INTO student(s_id, s_name, s_address)
```

```
VALUES (2, 'Rishabh', 'Dharwad');
```

```
VALUES (3, 'Ajay', 'Bapalgaon');
```

```
VALUES (4, 'Mehar', 'Pulchani');
```

2. Select Operation

a. Selects all rows from the table

Syntax:

```
SELECT *
```

```
FROM table_name;
```

Example:

```
SELECT *
```

```
FROM emp;
```

b. Select specific columns from a table:

Syntax:

```
SELECT column1, column2, column3
```

```
FROM table_name;
```

Example:

```
SELECT s_id, s_name, s_address;
```

```
FROM emp;
```

c. Select command with where clause

To select specific rows from a table we include 'where' clause in the select command. It can appear only after the 'from' clause.

Syntax:

```
SELECT column_name1, ..., column_nameN
```

```
FROM table_name
```

```
WHERE condition;
```

Example:

```
SELECT empno, empname
```

```
FROM emp
```

```
WHERE sal > 4000;
```


d. Select command with order by clause

In order to display the tuples in ascending or descending order, ORDER BY statement is used.

Syntax:

```
SELECT column_name1, ..., column_name
FROM table_name
WHERE condition ORDER BY column_name;
```

We may specify **desc** for descending order or **asc** for ascending order for each attribute, ascending order is by default.

Example:

List in alphabetical order the names of all instructors.

```
SELECT name
FROM instructor
ORDER BY name;
```

List names of all instructors to descending order.

```
SELECT name
FROM instructor
ORDER BY name DESC;
```

e. Select command to insert records:

Syntax:

```
INSERT INTO table_name ( SELECT columns
existing (tablename);
```

Example:

```
INSERT INTO emp1 (SELECT * from emp);
```

f. Selecting column using alias
Alias is a temporary name given to table or columns of a user friendly. It exists only for the duration of the query.
Syntax: (table alias)
SELECT column_name(s)
FROM table_name AS alias_name;

Syntax: (column alias)

```
SELECT column_name AS alias_name
FROM table_name;
```

Example:

```
SELECT emp_id AS id, ename AS Name
FROM emp AS e;
```

g. Like operator in SELECT

It is used to find pattern on column values. It is mostly used with wildcards. Wildcards are used with LIKE operator to find pattern on data value.

wildcard in SQL server

Symbol	Description	Example
%	Represents zero or more characters	man% finds man, manager, mango
_	Represents a single character	c_t finds cat, cup and cut
[]	Represents any single character within the brackets	[pou] finds pot and put but not pit
^	Represents any character not in the brackets	not b[oa] finds bit but not hot and hat
-	Represents a range of characters	a range of [a-b] finds cat and cbt

All the wildcards can be used in combination.

Example:

i. Employee whose position ends with 'engineer'.

```
SELECT *
FROM emp
```

WHERE position like '%engineer';

ii. Employee whose position starts with 'Associate'.

```
SELECT *
FROM emp
```

WHERE position like 'Associate%';

iii. Employee whose position contains word 'engineer'.
SELECT *

FROM emp

WHERE position like '%engineer%';

iv. Employee whose position starts with 'X' and ends with 'Y'.
SELECT *

FROM emp

WHERE position like 'X%Y';

v. Select the record of employee whose position has second letter 'X'.
SELECT *

FROM emp

WHERE position LIKE '%X%';

vi. Select the record of employee whose position has 'r' in third last letter.
SELECT *

FROM emp

WHERE position LIKE '%_r_';

vii. Select the department head whose name starts with 'r' and has at least 3 letters.
SELECT *

FROM dept

WHERE d.head like 'r_';

3. Update Operation

a. Update single column:

Syntax:

UPDATE table_name SET column_name = new_value
WHERE condition;
Example:
UPDATE emp SET salary = 100000 WHERE emp_id = 5;

b. Update multiple column:

Syntax:

UPDATE table_name SET column1 = value1, column2 = value2, ...
WHERE condition;

Example:

UPDATE emp SET ename = 'Jigney', address = 'Kolkata' WHERE emp_id = 5;

c. Update using CASE

Example:

UPDATE table SET sal = CASE
WHEN id = 1 THEN 2952
WHEN id = 2 THEN 4925
WHEN id = 3 THEN 1592
ELSE sal
END
WHERE id in (1,2,3);

4. Delete operation

Syntax:

DELETE FROM table_name
WHERE condition;

Example:

DELETE FROM emp

WHERE sal = 5;

Note: While using delete operation, it is very much important to take care of where clause. If where clause is absent in delete statement, it deletes all the records from table. In above example if where clause is removed then it deletes all the records from 'emp' table. In DELETE FROM emp

4.2.3 DCL (Data Control Language)

DCL (Data Control Language) includes commands like GRANT and REVOKE, which are useful to give rights & permissions. The authorities or actions that a user is permitted to carry out on database objects like tables, views, procedures, or functions are

determined by these privileges. The ability to grant and revoke privileges is a crucial component of database security because it enables administrators to manage access and limit specific actions to authorized users or roles.

- **GRANT Command**

GRANT is used to grant specific privileges or rights to users or user groups.

Syntax:

GRANT ON TO (WITH GRANT OPTION);

Here the following privileges can be specified:

ALTER, DELETE, INDEX, INSERT, SELECT, UPDATE

Example:

Give the user rabin permission to only view and modify records in the table employees.

GRANT SELECT, UPDATE ON employees TO rabin;

By default, a user granted privileges is not allowed to grant these privileges to other users. To allow this, we append the term WITH GRANT OPTION clause to the appropriate grant command.

Example:

GRANT SELECT, UPDATE ON employees TO rabin WITH GRANT OPTION;

- How user rabin can grant select and update privileges to all other users. Similarly, to revoke a privilege we use the REVOKE clause is used. This is very much like GRANT.

REVOKE Command

Revoking rights or privileges is done using the REVOKE statement.

Syntax:

REVOKE <privilege list> ON <relation or view name> FROM <user list>

Example:

REVOKE INSERT, SELECT ON employees FROM rabin

There is another keyword: ROLE which is the collection of privileges or access rights. When there are many users in a db it becomes difficult to grant or revoke privileges to users. Therefore, if you define roles, you can grant or revoke privileges to users. ROLE command syntax:

CREATE ROLE <rolename>

Example:

To grant CREATE TABLE privilege to a user by creating a testing role: CREATE ROLE testing GRANT CREATE TABLE TO testing GRANT testing TO rabin; -- Grant the role to a user.

rabin

REVOKE CREATE TABLE FROM testing --To revoke a CREATE TABLE privilege from testing ROLE

4.2.4 Basic Aggregation

Aggregate functions return a single value after performing calculations on a group of values. Following are some of the frequently used Aggregate functions:

1. **AVG() Function**

average returns average value after calculating it from values in a numeric column.

General syntax:

SELECT AVG(column_name) FROM table_name;

2. **COUNT() Function**

Count returns the number of rows present in the table either based on some condition or without condition.

General syntax:

SELECT COUNT(column_name) FROM table_name;

3. **MAX() Function**

MAX function returns maximum value from selected column of the table.

General syntax:

SELECT MAX(column_name) FROM table_name;

4. MIN() Function

MIN function returns minimum value from a selected column of the table.

General syntax:

SELECT MIN(column_name) FROM table_name;

5. SUM() Function

SUM function returns total sum of a selected column's numeric values.

General syntax:

SELECT SUM(column_name) FROM table_name;

Statements

4.25 Aggregation with Grouping

The group by statement groups the rows that have the same values in the summary rows like "find the number of customers in each city". The group by statement is often used with aggregate functions to group the result set by one or more columns.

GROUP BY Syntax:

SELECT column_name(s)

FROM table_name

WHERE condition

GROUP BY column_name(s)

ORDER BY column_name;

Example

1. Consider the table customers as:

custID	CustName	ContactName	Country
1	Chennai Store	Vishnuresh	India
2	Pakistan Sals	Javed	Pakistan
3	Zahid	Suorano	Bangladesh
4	Bahin Store	Rehbe	Nepal
5	Dell's Express	Ahmal	India

128 | Aspects on Database Management Systems

SELECT COUNT(Cust_ID) Country
FROM Customers
GROUP BY Country;

output:

Country
1 Bangladesh
2 India
1 Nepal
1 Pakistan

The following statement lists the number of customers in each country, sorted high to low.

SELECT COUNT(Cust_ID) Country
FROM Customers

GROUP BY Country

ORDER BY COUNT(Cust_ID) DESC;

2. Considers the tables orders and shippers as:

OrderID	CustomerID	EmployeeID	ShipperID
10248	90	5	3
10249	81	6	1
10250	34	4	2
10251	29	3	2

Shippers

ShipperID	ShipperName
1	Speedy Express
2	United Package
3	Federal Shipping

SELECT

Shippers.ShipperName, COUNT(Orders.OrderID)
NumberOrder

FROM Orders LEFT JOIN Shippers

ON

Orders.ShipperID=Shippers.ShipperID

GROUP BY ShipperName;

Output:

ShipperName	NumberOrder
FederalShipping	1
Speedy Express	1
United Package	2

4.2.6 Having Clause

The having clause was added to SQL because the WHERE keyword could not be used with aggregate functions.

General Syntax:

SELECT Column Name(s)

FROM table_name

WHERE Condition

GROUP BY column_name

HAVING Condition;

Example:

SELECT COUNT(Cust.ID) Country

FROM Customers

GROUP BY country

HAVING COUNT(Cust.ID)>1;

Output:

Country
2 India

4.2.7 Views

A view is a tailored presentation of the data contained in one or more tables. In other words, a view is a logical or virtual table, which does not exist physically in the database. Views are also called as dictionary objects. It takes the output of a query and uses it as a table.

To create view we have a following syntax:

CreateView <View_Name> as <Query>;

Following are the advantages of views:

- ▶ They provide an additional level of table security by restricting access to a predetermined set of rows and/or columns of a table.
 - ▶ They hide data complexity.
 - ▶ Simplifies commands for the user because they allow them to select information from multiple tables without actually knowing how to perform a join.
 - ▶ They isolate application from changes in definitions of base tables.
 - ▶ They provide data in a different perspective than that of a base table by retaining columns without affecting the base table.
 - ▶ Although there are many advantages to views, the main disadvantage to using views rather than real tables is performance degradation.
- Views can be classified into two categories on the way they are created.

1. Simple View

If the view is created from a single table, it is called as a simple view.

Example:

A view of instructors without their salary.

CREATE VIEW faculty AS
SELECT ID, name, dept_name
FROM instructor;

2. Complex View

If the view is created on multiple tables, it is called a complex view.

Example:

A view 'student_course' having the attributes `cm`, `sec_id`, `poos`, `course_name`, `enroll_data`.

CREATE VIEW student_course **AS**

```
SELECT scm, scname, sphone, secname, enroll_data  
FROM student_s, course_c, enroll_e  
WHERE scm=ccm AND ccourseid=c.courseid;
```

A view can be updated with the **CREATE OR REPLACE VIEW** statement as follows:

CREATE OR REPLACE VIEW view_name AS

SELECT column1, column2, ..

FROM table_name

WHERE condition;

A view is deleted with the **DROP VIEW** statement as:
DROP VIEW view_name;

Materialized Views

Certain database systems allow view relations to be stored but they make sure that if the actual relations used in the view definition change, the view is kept up-to-date. Such views are called materialized views.

Example:

Consider the view departments_sal salary. If that view is materialized, its results would be stored in the database. Later by using the precompiled view result, instead of recompiling it.

However, if an instructor tuple is added to or deleted from the instructor relation, the result of the query defining the view would change, and as a result the materialized view's contents must be updated.

Similarly, if an instructor's salary is updated, the tuple in department's total salary corresponding to that instructor's department must be updated.

View Maintenance

The process of keeping the materialized view up-to-date is called materialized view maintenance [or often, just view maintenance].

4.3 Stored Procedures

A stored procedure is a collection of precompiled SQL statements and procedural logic that are stored and executed on a database server. This allows us to encapsulate a set of database requests into reusable, well-defined routines. Stored procedures are commonly used in database applications to improve performance, improve security, and promote code reuse.

Syntax for creating stored procedures in MySQL:

CREATE PROCEDURE procedure_name [(parameter_list)]

BEGIN

– (procedure body (SQL statements))

END;

Here's an example of a simple stored procedure without parameters:

DELIMITER //

CREATE PROCEDURE HelloWorld()

BEGIN

SELECT Hello, World!;

END //

DELIMITER //

And here's an example of a stored procedure with input parameters:

DELIMITER //

CREATE PROCEDURE GetCustomerByCountry(IN

country_name VARCHAR(50))

BEGIN

```
SELECT * FROM Customers WHERE Customers.Country =  
country_name;
```

END //

DELIMITER;

To create a stored procedure, we can use the **CALL** statement:

```
CALL procedure_name(argument_list);
```

For example, to call the `GetCustomerByCountry` stored procedure with the argument `USA`:

```
CALL GetCustomerByCountry('USA');
```

Here are some of the key features and benefits of using stored procedures:

1. Code Encapsulation

A stored procedure encapsulates a set of SQL statements and business logic into a single entity. This makes managing and maintaining complex database operations easier.

2. Performance Optimization

By precompiling SQL statements within stored procedures, the database server can execute the SQL statements more efficiently. This improves performance, especially for frequently performed operations.

3. Code Reusability

Stored procedures can be reused for multiple applications and database operations. This promotes code reusability and reduces code duplication.

4. Security

Stored procedures can provide an additional layer of security by allowing controlled access to the database. Users can be granted privileges to execute specific stored procedures without restricting direct access to the underlying tables.

5. Transaction Management

We can use stored procedures to define transaction boundaries and ensure that a set of database operations succeeds

or fails simultaneously. This helps maintain data integrity and consistency.

Modularity and Maintainability

Stored procedures promote modular programming by breaking complex operations into smaller, more manageable blocks. This makes the code base easier to understand, debug, and maintain.

Reduced Network Traffic

By executing a stored procedure, we can simply send the procedure name and arguments to the database server, instead of sending the full SQL statements. This can reduce network traffic between our application and our database server.

Disadvantages of Stored Procedures

1. Portability: Because stored procedures are frequently database-specific, they could be difficult to adapt to other database management systems (DBMS). It may be necessary to rewrite or adjust our stored procedures to operate with the new system if we need to migrate your application to a different DBMS, which can be time-consuming and error-prone.

2. Limited Debugging and Testing: Comparatively speaking to other application code, debugging and testing stored procedures can be more difficult. The procedure is often executed in its entirety or in part within the database environment. This may make it more difficult to identify and troubleshoot problems, particularly if the stored procedure interacts with complex database structures or external components.

3. Versioning and Maintenance: When numerous versions of variables are in use, managing stored procedures can become challenging. If there are dependencies on the procedure's operation from other parts of the system, updating many places may be necessary if changes to a stored procedure involve changing the existing code. This may make versioning, deployment, and maintenance procedures more

- **Performance Optimization:** Performance can be increased with stored procedures by streamlining query execution and lowering network traffic. The necessity for repetitive queries and optimization is removed by stored procedures by precompiling and storing the SQL statements. Faster query execution results from this, especially for complicated or often executed processes.

4.4 Query By Example (QBE)

QBE is a graphical query language based on domain relational calculus. It's a query language used in relational databases that allows users to search for information in tables and fields by providing a simple user interface where the user will be able to input an example of the data that he or she wants to access. It is merely an abstraction between the user and the real query that the database system will receive. In the background, the user's query is transformed into a database manipulation language form such as SQL, and it is this SQL statement that will be executed in the background.

QBE Skeleton Table for the Bank Example:

branch	branch_name	branch_city	assets
customer	customer_name	customer_street	customer_city
loan	loan_number	branch_name	amount
borrower	customer_name	loan_number	
account	account_number	branch_name	balance

depositor	customer_name	account number

example:
find all loan numbers at the Perryridge branch.

loan	loan_number	branch_name	amount
	P.X	Perryridge	

x is a variable (optional; can be omitted in above query)
P.x means print(display)

Duplicates are removed by default
to return duplicates we can use P.ALL

loan	loan_number	branch_name	amount
	P.ALL	Perryridge	

2. display/full details of all loans.

loan	loan_number	branch_name	amount
	P.X	P.Y	P.Z

Method 2: Shorthand notation

loan	loan_number	branch_name	amount
	P.		

3. Find the loan number of all loans with a loan amount more than \$700.

loan	loan_number	branch_name	amount
	P.	P.	>700

4. Find names of all branches that are not located in Brooklyn.

branch	branch_name	branch_city	assets
	P.	-Brooklyn	

5. Find the names of all customers who have a loan from Perryridge branch.

loan	loan number	branch name	amount
	x	Perryridge	
borrower	customer name	loan number	
	P _{xy}	x	

The Condition Box

Allows the expression of constraints on domain variables like are either invariant or impossible to express within the set of values in condition boxes we can use complex conditions.

Example:

Find the loan numbers of all loans made to Smith, to Jones, or to the both.

borrower	customer name	loan number
	l	P _{xy}
	Conditions	
	Smith or Jones	

[140] Rights as follows are reserved by the authors.

SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS

1. Write SQL statements for the following queries using the given Employees relation.

E_id	Prname	lastame	Department	Salary	Hire Date
01	Rajou	Bastiyal	Sales	20000	2023-08-08
02	Jaganu	Pandey	IT	50000	2022-01-01
03	Biru	B.K.	Sales	40000	2021-02-10
04	Hiru	Dharmala	HR	35000	2023-12-16
05	Biren	Chudka	IT	60000	2012-10-22

- Create a database named company and Employee relation
- Create a view that shows the E_id, Department and Hire Date of all employees
- Modify the table such that the Department of Biren is HR now
- Delete the record of employees whose Ename is 'pandey'.

[ans smith

[141]

- CREATE DATABASE company;
- USE company;
- CREATE TABLE Employee (

E_id INT PRIMARY KEY,
Prname VARCHAR(25),
lastame VARCHAR(25),
Department VARCHAR(25),
Salary INT,
Hire Date DATE

);

— Insert data into the Employee table

Source: Interview Questions | 141 |

INSERT INTO Employee (E_id, Fname, Lname,
Department, Salary, Hire_Date) VALUES

(01, 'Ramu', 'Rashtyal', 'Sales', 20000, '2023-06-08'),

(02, 'Danu', 'Pandey', 'IT', 50000, '2022-01-01'),

(03, 'Biju', 'B.K.', 'Sales', 40000, '2021-02-10'),

(04, 'Hru', 'Dhamale', 'HR', 35000, '2023-12-18'),

(05, 'Biju', 'Khadka', 'IT', 60000, '2012-10-22').

ii. CREATE VIEW Employee_View AS

SELECT E_id, Department, Hire_Date FROM Employee;

iii. UPDATE Employee

SET Department = 'HR'

WHERE Fname = 'Biju' and Lname = 'Khadka';

iv. DELETE FROM Employee WHERE Lname = 'Pandey';

2. Consider the relation Actress_Details and write the SQL statements for the following queries:

Players	id	Address	Debut	Recent	fee	Address
1	Keou	2010	Samay	40000		
2	Sita	2022	Radhya	30000		
3	Geeta	2001	Maio	60000		
4	Ambha	1990	Man	70000		
5	Karishma	1989	Prem	100000		

i. Create the table Actress_Details relation.

ii. Delete the data of actress whose recent release is Prem.

iii. Modify the database so that Ramu's new release is 'Win the Race' film.

iv. Insert a new record in the above table.

1. CREATE TABLE Actress_Details ([2023-07-07]
Players_id INT,
Actress_fname VARCHAR(50),
Debut_year INT,
Recent_release VARCHAR(50).

Actress_fee DECIMAL(10,2)

);

ii. DELETE FROM Actress_Details

WHERE Recent_release = 'Prem';

iii. UPDATE Actress_Details

SET Recent_release = 'Win the Race'

WHERE Actress_name = 'Ramu';

iv. INSERT INTO Actress_Details (Players_id, Actress_name,
Debut_year, Recent_release, Actress_fee)

VALUES (6, 'Priya', 2018, 'Journey', 50000);

3. Considering the following schemas:

Sailors (sid, sname, rating, age)

Boats (bid, bname, color)

Reserves (sid, bid, day)

Write SQL expressions for the following queries:

i. Find the records of sailors who have reserved boat number 103 (bid=103).

ii. Update the color of the boat where bid is 104, into green.

iii. Find the names of sailors who have reserved a red or green boat.

iv. Find the names of sailors who have reserved boat number 103 on day 5.

v. Find the names of sailors whose name is not 'Ramu'

vi. Find the names of all boats. [2023-07-07]

1. SELECT *

FROM Sailors

JOIN Reserves ON Sailors.sid = Reserves.sid

JOIN Boats ON Reserves.bid = Boats.bid

WHERE Boats.bid = 103;

ii. UPDATE Boats

SET color = 'green'

WHERE bid = 104;

iii. SELECT DISTINCT Sailors.sname

FROM Sailors

JOIN Reserves ON Sailors.sid = Reserves.sid

JOIN Boats ON Reserves.bid = Boats.bid

WHERE Boats.color = 'red' OR Boats.color = 'green';

iv. SELECT Sailors.sname

FROM Sailors

JOIN Reserves ON Sailors.sid = Reserves.sid

JOIN Boats ON Reserves.bid = Boats.bid

WHERE Boats.bid = 103 AND Reserves.day = 5;

v. SELECT name

FROM Sailors

WHERE sname != 'Ram';

vi. SELECT sname

FROM Boats;

4. Write SQL statements for following:

i. Create a table named Chef with chef license as primary key and following attributes:

chef_license, c_fname, c_lname, c_dob, c_gender, c_experience_hours, c_photograph

ii. Enter a full detailed information of a chef.

iii. Change chiefs experience hours of a chef.

iv. Remove all chef records whose name contains character 'r' in second position in his first name.

v. Display the total experience hours of all chef.

vi. Create a view from above table.

vii. Display details of chef ordering on descending manner in last name and by first name when last name matches.

Ans 1.

CREATE TABLE Chef (

chef_license INT PRIMARY KEY,

c_fname VARCHAR(50),

c_lname VARCHAR(50),

c_dob DATE,

c_gender CHAR(1),

c_experience_hours INT,

c_photograph BLOB

);

ii. INSERT INTO Chef (chef_license, c_fname, c_lname, c_dob,

c_gender, c_experience_hours, c_photograph)

VALUES (1234, 'John', 'Doe', '1985-06-15', 'M', 2000,

NULL);

iii. UPDATE Chef SET c_experience_hours = 2500 WHERE

chef_license = 1234;

iv. DELETE FROM Chef WHERE c_fname LIKE 'J%';

v. SELECT SUM(c_experience_hours) as

total_experience_hours FROM Chef;

vi. CREATE VIEW Chef_View AS

SELECT chef_license, c_fname, c_lname, c_dob, c_gender,

c_experience_hours

FROM Chef;

vii. SELECT * FROM Chef

ORDER BY c_lname DESC, c_fname;

5. Write SQL statement for the following schemas

(underline indicates primary key):

Employee (Emp_No, Name, Address)

Project (P_No, Pname)

Workon (Emp_No, P_No, P_No)

Part (Parting_Part_name, Qty_on_hand)

Use (Emp_No, P_No, Parting_Part_name)

a. Listing all the employee details who are not working yet.

b. Listing partname and Quantity on hand those were used in DBMS project.

c. List the name of the projects that are used by employee from London.

d. Modify the database so that Jones now lives in USA.

e. Update address of an employee Japan to USA.

pg 144

146 a. SELECT *

FROM Employee
WHERE Emp_No NOT IN (SELECT Emp_No FROM Workon);

b. SELECT Part,PartName,PartQty, on, Hand FROM Part

JOIN Use ON Use.Partno = Part.Partno

JOIN Project ON Project.Pno = Use.Pno

WHERE Project.Pname = 'DBMS';

c. SELECT DISTINCT Project.Pname FROM Project

JOIN Workon ON Workon.Pno = Project.Pno

JOIN Employee ON Employee.Emp_No = Workon.Emp_No

WHERE Employee.Address = 'London';

d. UPDATE Employee

SET Address = 'USA';

WHERE Name = 'Jones';

e. UPDATE Employee

SET Address = 'USA';

WHERE Address = 'Japan';

6. Write SQL statements for following:

i. Create a table named Automotor with

chassis_number as primary key and following

attributes:

veh_brand, veh_name, veh_model, veh_year,

veh_cost, veh_color, veh_weight

ii. Enter a full detailed information of an automotor.

iii. Change any Automotor's year to 2019.

iv. Remove all Automotor records whose model

contains character 'r' in last position.

v. Display the total cost of all vehicles of the table Automotor.

147 Create a view from above table having vehicles only

red color.

vi. Display details of Automotor ordering on

displaying manner by brand name and by

ascending on model when brand matches.

vii. Change data type of color so that it only takes one

character.

1. CREATE TABLE Automotor (

chassis_number INTEGER PRIMARY KEY,

veh_brand VARCHAR(30),

veh_name VARCHAR(30),

veh_model VARCHAR(30),

veh_year INTEGER,

veh_cost REAL,

veh_color VARCHAR(10),

veh_weight REAL

);

ii. INSERT INTO Automotor VALUES (1, 'Toyota', 'Corolla',

'SE, 2020, 25000', 'Black, 1200');

iii. UPDATE Automotor SET veh_year = 2019 WHERE

chassis_number = 1;

iv. DELETE FROM Automotor WHERE veh_model LIKE '%K';

v. SELECT SUM(veh_cost) AS total_cost FROM Automotor

vi. CREATE VIEW Red_Vehicles AS

SELECT * FROM Automotor WHERE veh_color = 'Red';

vii. SELECT * FROM Automotor ORDER BY veh_brand DESC;

viii. SELECT * FROM Automotor WHERE veh_model ASC;

ix. ALTER TABLE Automotor

ALTER COLUMN veh_color CHAR(1);

x. Differentiate between join and sub query

148 SQL uses joins and subqueries, two distinct methods, to

query and retrieve data from relational databases. These are

their differences:

146 | Insights on Database Management Systems

147 | Relational Database Query Languages

148 |

149 |

150 |

151 |

152 |

153 |

154 |

Join:

- When combining rows from two or more tables, joins in similar columns, a join is utilized.
- By including the condition that establishes the relationship between the tables in the join clause, we can combine data from many tables.
- By matching values in the specified columns, joins can be used to select columns from many tables, including foreign data.

The way the data from the joined tables is matched is referred to as the type of join used. Unions, Joins, and Joins.

Subquery:

- A subquery is a query that is nested inside of another query, also referred to as a nested query or inner query.
- It enables us to manipulate the inner query's output into the outer query.
- Subqueries may be used in the SELECT, FROM, WHERE, or HAVING clauses, among other portions of a query.
- Using operations on a subset of data, a subquery, which is surrounded in parentheses, can be used to filter or manipulate data.
- The outer query uses a subquery's output to further refine the set of records that it returns.

Write the SQL statements for the following queries by reference of hotel details relation

1	Hotel name	City	Year hotel started	Hotel worth
1	Hyatt	2007	Five	15M
2	Hotel Ram	2013	Three	5M
3	Pullmant	2018	Five	20M
4	Wald and Vail	2012	Four	10M
5	Hotel Chiverna	2015	Three	7M

148 | Chapter 10 Database Management Systems

Create a database named hotel & table relation.

- Create a view named Price which shows hotel name & its worth.
- Modify the data so that Hotel Chiverna is now four star hotel.
- Delete the records of all hotels having worth more than 90M.

6. CREATE DATABASE hotel;

USE hotel;

```
CREATE TABLE hotel_details (
    hotel_id INT NOT NULL PRIMARY KEY,
    hotel_name VARCHAR(50) NOT NULL,
    city VARCHAR(10) NOT NULL,
    hotel_worth FLOAT NOT NULL
);
```

```
INSERT INTO hotel_details (hotel_id, hotel_name,
city, year, hotel_start, hotel_worth)
VALUES (1, 'Hyatt', '2007', 'Five', '15000000');
2. 'Hotel Ram', '2013', 'Three', '5000000';
3. 'Pullmant', '2018', 'Five', '20000000';
4. 'Wald and Vail', '2012', 'Four', '10000000';
5. 'Hotel Chiverna', '2015', 'Three', '7000000';
```

7. CREATE VIEW Price AS

```
SELECT hotel_name, hotel_worth FROM hotel_details
WHERE hotel_worth > 90000000;
```

```
SET local_name = 'Four'
UPDATE hotel_details SET hotel_worth = 'Four'
WHERE hotel_worth > 90000000;
```

149 | Chapter 10 Database Management Systems

9. Consider a simple relational database of Hospital Management System. (Underlined attributes represent Primary key attributes)

Doctors (DoctorID, DoctorName, Department, Address, Salary)
 Patients (PatientID, Patient Name, Address, Age, Gender)
 Hospitals (PatientID, Doctor ID, HospitalName, Location)

Write down the SQL statement for the following:

- Display ID of Patient admitted to hospital at Polkara and whose name ends with 's'.
- Delete the record of Doctors whose salary is greater than average salary of doctors.
- Increase the salary of doctors by 18.5% who works in OPD department.
- Find the average salary of Doctors for each address who have average salary more than 55K.

10. SQL27 p.PatientID

PHOM Patients P

JOIN Hospitalas H ON P.PatientID = H.PatientID

WHERE H.Location = 'Polkara' AND P.PatientName LIKE

'%s';

11. DELETE FROM Doctors

WHERE Salary > (SELECT AVG(Salary)

FROM Doctors)

);

12. UPDATE Doctors

SET Salary = Salary * 1.18;

WHERE Department = 'OPD';

13. SELECT Address, AVG(Salary) AS AvgSalary

FROM Doctors

GROUP BY Address

HAVING AVG(Salary) > 55000;

130 | Imparts on Database Management Systems

Consider the relational database model:

11. Create field, create, city)
 Item (ItemID, Itemname, city, quantity, price)
 Item (ItemID, Itemname, city)
 Manager (ManagerID, Itemname, city)
 Query (QueryID, ItemID, Itemname, query details, history)

Write SQL statements for following:

- Create a table named Vehicle with veh_number as primary key and following attributes:
 veh_type, veh_brand, veh_year, veh_mileage, veh_color, veh_photo, veh_price
 Enter a full detailed information of a vehicle.
- Increment vehicle's price by 10,000.
- Remove all vehicle's records whose brand contains character 'V' in second position.
- Display the total price of all vehicles.
- Create a view from above table.
- Display details of vehicles ordering on descending manner in brand and by mileage when brand matches.
- Change data type of year to date-time.

131 | Database Management Systems

12. CREATE TABLE Vehicle (

veh_number INT PRIMARY KEY,

veh_type VARCHAR(20),

veh_brand VARCHAR(50),

veh_year INT,

veh_mileage INT,

veh_color VARCHAR(50),

veh_photo BLOB,

veh_price DECIMAL(10,2)

);

13. INSERT INTO Vehicles (veh_number, veh_type, veh_brand,

veh_year, veh_mileage, veh_color, veh_price, veh_photo)

VALUES ('1', 'Car', 'Toyota', 2020, 5000, 'John Doe',

'toyota.jpg', 200000.00);

132 | Database Management Systems


```

iii. UPDATE Vehicle
SET veh_price = veh_price + 10000;

iv. DELETE FROM Vehicle
WHERE SUBSTRING(veh_brand, 2, 1) = 'v';

v. SELECT SUM(veh_price) AS TotalPrice
FROM Vehicle;

vi. CREATE VIEW VehicleView AS
SELECT *
FROM Vehicle;

vii. SELECT * FROM Vehicle
ORDER BY veh_brand DESC, veh_price DESC;

viii. ALTER TABLE Vehicle
ALTER COLUMN veh_year DATETIME;

```

11. Consider the following three relations.

```

Doctor(No, name, age, address)
Workshop(No, depart, no, salary)
Department(Dept no, department, floor, room)

```

Write down the SQL statement for the following.

- Display the name of doctor who do not work in any department.
- Modify the database so that Dr. Hari lives in Pochana.
- Delete all record of Doctor working in OPD department.
- Display the name of Doctors who work in at least two departments.

```

SQL:
FROM Doctor
WHERE Name NOT IN (
SELECT Name
FROM Work
)

```

```

UPDATE Doctor
SET address = 'Pochana';

WHERE Name = 'Dr. Hari';

ii. DELETE FROM Works
WHERE Dept no IN (
SELECT Dept no
FROM Department
WHERE department = 'OPD'
)

```

12. Differentiate between SQL and MS-SQL. Why access to database from general purpose programming language is required? Explain.

```

SQL:
SELECT Name
FROM Works
GROUP BY Name
HAVING COUNT(DISTINCT DeptNo) >= 2;

```

13. In the field of databases, SQL (Structured Query Language) and MySQL are similar but different ideas. There are four differences:

SQL:

- > For handling relational databases, SQL is a standardized language.
- > For defining, modifying and querying data in a relational database management system (RDBMS), it offers a set of commands and syntax.
- > SQL is supported by a wide range of relational databases, including MySQL, Oracle, Microsoft SQL Server, PostgreSQL, and others. SQL is not specific to any one particular database system.
- > It is a declarative language, so we only specify the data we need, and the database system will take care of the specifics of retrieving or changing the data.

MySQL:

- > A relational database management system (RDBMS) that employs the SQL language is specifically implemented by MySQL.
- > It is a popular open-source database management system that is used for a variety of applications, from medium personal projects to extensive business systems.
- > MySQL supports the SQL language and provides additional features and optimizations specific to the MySQL database system.
- > It offers strong data storage, retrieval, and management features, including support with transactions, including user administration, and more.
- > Let's talk about the need for database access from a general purpose programming language now:

Flexibility and Control:

General-purpose programming languages like Python, Java, or C++ give program logic critical and a wide range of functionalities.

Developers can do table-level, built, complicated business logic, modify data, etc. manage data transactions that go beyond what SQL by itself is capable of by directly accessing a database from a computer language.

Application Development:

General-purpose programming languages allow developers to build entire applications by combining the power of programming constructs with the data storage capabilities provided by databases.

Integration and Interoperability:

Many applications require complex business logic, user interfaces, integration with external systems, and more. In addition to data storage and retrieval.

Connecting databases to general-purpose programming languages enables seamless integration with other systems and technologies.

By accessing the database from a programming language, developers can interact with APIs, web services, external libraries, and frameworks, enabling data-driven applications and system integrations.

Data Processing and Analysis:

Data processing and tools are available for data processing such libraries and tools are available for data processing analysis, and visualization in general-purpose programming languages.

Developers can use these tools to perform complex computations, statistical analysis, machine learning, and generate reports based on the data stored in the database by accessing the database from a computer language.

13. Write the SQL statements for the following queries by reference of Liquors_Info relation:

Social No	Liquors	Start year	Bottles	Ready year
1	fortkna	1997	10	1998
2	Dyfine	1998	5	2000
3	Old Durbur	1997	12	2001
4	Rthkurf	1991	10	1992
5	Xing	1994	5	1995

1. Create the Liquors_Info relation.
2. Insert the records in Liquors_Info as above.
3. List all the records which were ready by 2000.
4. Remove all records from data base that required more than 2 years to get ready.

14. CREATE TABLE Liquors_Info (

Social No INT PRIMARY KEY,

Liquors VARCHAR(20) NOT NULL,

Start Year INT,

Bottles INT,

Ready Year INT,

INDEX (Liquors),

INDEX (Ready Year))

155 | **Relational Database Query Language**

Liquors VARCHAR(50),

Start_Year INT,

Bottles INT,

Ready_Year INT);

ii. INSERT INTO Liquors_Info (Serial_No, Liquors, Start_Year,

Bottles, Ready_Year) VALUES

(1, 'Gordon', 1997, 10, 1998),

(2, 'Dvine Wine', 1998, 5, 2000),

(3, 'Old Durnar', 1997, 12, 2001),

(4, 'Kinkut Rum', 1991, 10, 1992),

(5, 'Mig', 1994, 5, 1995);

iii. SELECT *

FROM Liquors_Info

WHERE Ready_Year <= 2000;

iv. DELETE FROM Liquors_Info

WHERE (Ready_Year - Start_Year) > 2;

14. What is the difference between where and having clause?

Ans: In SQL queries, the WHERE and HAVING clauses are both employed to filter and restrict data according to predefined criteria. They are utilized in various parts of a query and have different functions. The where clause and the having clause differ in the following ways:

[20/11/21]

WHERE Clause:
Rows from a table are filtered using the WHERE clause according to a criteria or group of conditions. SELECT, frequently utilized, and DELETE commands are where it is most conditions) to each individual row to filter it.

Syntax:
SELECT column1, column2, ...
FROM table

[15/6] Insights on Database Management Systems

WHERE condition;

Example:

SELECT * FROM employees

WHERE department = 'Sales';

This query retrieves all the rows from the 'employees' table where the department is 'Sales'.

HAVING Clause:

After data has been grouped and aggregated, rows are filtered based on conditions using the HAVING clause. It frequently goes along with the GROUP BY clause. When applied to grouped data, the HAVING clause filters the groups according to the supplied condition(s). Conditions can be applied to the output of aggregate functions like COUNT, SUM, AVG, etc.

Syntax:

SELECT column1, column2, ...

FROM table

GROUP BY column1, column2, ...

HAVING condition;

Example:

SELECT department, COUNT(*) as total_employees

FROM employees

GROUP BY department

HAVING COUNT(*) > 5;

This query retrieves the departments from the 'employees' table along with the count of employees in each department. However, it only includes departments that have more than 5 employees in the result.

15. Consider a simple relational database of Hospital

Management System.

(Underlined attributes represent Primary key attributes)

Doctors (DoctorID, DoctorName, Department, Address, Salary)

Patients (PatientID, PatientName, Address, Age, Gender)
Hospitals (HospitalID, Doctor ID, HospitalName, Location)

Write Down the SQL statement for the following.

- Display ID of Patient admitted to hospital Polkara and whose name ends with 'v'.
- Delete the record of Doctors whose salary is greater than average salary of doctors.
- Increase the salary of doctors by 18.5% who work in OPD department.
- Find the average salary of Doctors for each address who have average salary more than 55K.

150 i. SELECT PatientID FROM Hospital

WHERE Location = 'Polkara' AND PatientID IN

(SELECT PatientID

FROM Patients

WHERE PatientName LIKE '%a%');

ii. DELETE FROM Doctors

WHERE Salary > (SELECT AVG(Salary) FROM Doctors);

iii. UPDATE Doctors

SET Salary = Salary * (Salary * 0.185)

WHERE Department = 'OPD';

iv. SELECT Address, AVG(Salary)

FROM Doctors

GROUP BY Address

HAVING AVG(Salary) > 55000;

151 k. How does a view differ with relation?

Ans: A view and a relation (table) in a database differ in the following ways:

i. Data Storage:

Relation: Stores data directly in a physical structure (table) within the database.

152

l. Explain on Database Management Systems

view: Provides a virtual representation of data drawn from underlying tables but does not actually store the data itself.

Data Modification:

Relation: Allows direct modifications (insert, update, delete) to the stored data.

View: Generally used for data retrieval and has limited or restricted modifications.

Data Representation:

Relation: Represents a complete set of data with all attributes and constraints.

View: Represents a subset or customized representation of data from one or more relations.

Schema:

Relation: Has its own defined schema specifying attributes, data types, and constraints.

View: Inherits the schema from the underlying relation(s) it is based on.

Data Abstraction:

Relation: Provides a concrete representation of data without abstraction.

View: Offers a level of abstraction by presenting a tailored, filtered, or aggregated view of the data.

Query Simplification:

Relation: Requires complex SQL statements for joins, aggregations, and subqueries.

View: Can simplify queries by encapsulating complex operations into a single view definition.

17. Consider a relational Schema:

Teacher(TeacherID, TeacherName, Office)

Write SQL statement for the following task:

- To create a table from a table.
- To eliminate duplicate rows.
- To add a new column Gender to the table.

Answer: Teacher(TeacherID, TeacherName, Office) | 159 |

- iv. To sort data in a table.
- v. To delete rows.
- vi. Count number of rows based on Office.

Ans

i. CREATE TABLE Teacher_Details AS

Bharu

- ii. SELECT DISTINCT * FROM Teacher;
- iii. ALTER TABLE Teacher ADD Gender VARCHAR(10);
- iv. SELECT * FROM Teacher ORDER BY TeacherName ASC;
- v. DELETE FROM Teacher;
- vi. SELECT Office, COUNT(*) AS NumberOfTeachers FROM Teacher GROUP BY Office;

18.

Suppose we are given the following table definitions with certain records in each table.

EMPLOYEE (EID, NAME, POST, AGE)
 POST (POST_TITLE, SALARY)
 PROJECT (PID, PNAME, DURATION, BUDGET)
 WORK_IN (PID, EID, JOIN_DATE)

- i. List the name of employees whose age is greater than the average age of all employees.
- ii. Display all employee numbers of those employee who are not working in any project
- iii. List name of employee and their salary who are working in the project 'DBMS'.
- iv. Update the database so that 'Rishab' now lives in 'Bharu'.

Ans

i. SELECT NAME FROM EMPLOYEE WHERE AGE > (SELECT AVG(AGE) FROM EMPLOYEE);

ii. SELECT EID FROM EMPLOYEE WHERE EID NOT IN (SELECT EID FROM WORK_IN);

iii. SELECT NAME, SALARY FROM EMPLOYEE WHERE PROJECT = 'DBMS';

160

Insights on Database Management Systems

iii. SELECT E.NAME, P.SALARY FROM EMPLOYEE E JOIN POST P ON E.POST = P.POST_TITLE JOIN WORK_IN W ON E.EID = W.EID JOIN PROJECT PJ ON W.PID = PJ.PID WHERE PJ.PNAME = 'DBMS';

iv. ALTER TABLE EMPLOYEE ADD ADDRESS VARCHAR(100);

UPDATE EMPLOYEE SET ADDRESS = 'Bharu' WHERE NAME = 'Rishab';

19. Write SQL statements for the following queries in reference to relation Emp_time provided.

Eid*	Name	Start time	End time
E101	Hari	10:15	16:00
E102	Malvi	8:00	15:30
E103	Kalyan	9:30	17:00

- i. Create the table 'Eid*' as primary key and insert the values provided.
- ii. Display the name of the employee whose name start from letter 'M' and who work for more than seven hours.
- iii. Delete the entire contents of the table so that new data can be inserted.

Ans

i. CREATE TABLE Emp_time (Eid VARCHAR(10) PRIMARY KEY, Name VARCHAR(50), Start_time TIME, End_time TIME);

INSERT INTO Emp_time (End, Name, Start_time, End_time)
VALUES ('E101', 'Hart', '10:15', '18:00'),
('E102', 'Maid', '8:00', '15:30'),
('E103', 'Kalyan', '9:30', '17:00');

iii. SELECT Name

FROM Emp_time

WHERE Name LIKE 'M%' AND TIMEID(Find time,

Start time) > 7:00;

iii. DELETE FROM Emp_time;

20. Consider the employee database of figure given below, where primary keys are underlined. Give an expression in SQL for each of following queries.

Employees (employee_name, street, city)

Works (employee_name, company_name, salary)

manages (employee_name, city)

manages (employee_name, manager_name)

i. Modify the database so that Kam now lives in Kadambari.

ii. Give all employees of First Bank Corporation a 10 percent raise.

iii. Give all managers of First Bank Corporation a 10 percent raise.

iv. Delete all tuples in the works relation for employees of small bank Corporation.

v. Find all employees who earn more than the average salary of all employees of bank company. SQL query

1. UPDATE Employee

SET city = 'Kadambari';

ii. WHERE employee_name = 'Kam';

SET salary = salary * 1.1

WHERE company_name = 'First Bank Corporation';

iii. UPDATE Works
SET salary = salary * 1.1

WHERE company_name = 'First Bank Corporation' AND
employee_name IN (SELECT manager_name FROM

Manages);

iv. DELETE FROM Works

WHERE company_name = 'Small Bank Corporation';

v. SELECT e.employee_name

FROM Employee e

INNER JOIN Works w ON e.employee_name =

w.employee_name

INNER JOIN (

SELECT company_name, AVG(salary) AS avg_salary

FROM Works

GROUP BY company_name

) AS avg_salary ON w.company_name =

company_avg.avg_salary;

WHERE w.salary > company_avg.avg_salary;

21. Consider following relations:

employee (emp_name, street, city)

works (emp_name, company_name, salary)

manages (emp_name, city)

manages (emp_name, manager_name)

Write SQL statements for:

i. Find employee names that lives in the city same as the company city.

ii. List all employee details who earn more than 2500.

iii. Update address of an employee 'Shravan' to 'Pebkani'.

iv. Create a view for which employee earns Rs. 20,000 or more.

v. Delete all employee from the table employee.

1. SELECT emp_name

FROM employee e, company c

WHERE e.city = c.city;

ii. SELECT e.*

FROM employee e, works w

WHERE emp_name = w.emp_name AND w.salary > 25000;

iii. UPDATE employee

SET city = 'Pahara'

WHERE emp_name = 'Sriyash';

iv. CREATE VIEW high_earners AS
SELECT e.*

FROM employee e, works w

WHERE emp_name = w.emp_name AND w.salary >= 20000;

v. DELETE FROM employee

◆◆◆

DATABASE CONSTRAINTS AND RELATIONAL DATABASE DESIGN

CHAPTER - 5

5.1 Introduction

A database constraint is a rule that governs the data that can be stored in a database. Constraints are used to ensure the accuracy, consistency, and integrity of the data in the database. Constraints are typically defined at the time the database schema is created and can be enforced by the database management system.

5.2 Integrity Constraints

Integrity constraints are the set of rules which are enforced to maintain the quality of information. Constraints are used to limit the type, length and range of data that can go into the table, accuracy and consistency in the database can be maintained with the help of constraints. If there is any violation between the constraint and the data action, the action is aborted. Thus, integrity constraints help to guard against accidental damage to the database.

Constraints can be specified when the table is created with the CREATE TABLE statement, or after the table is altered with the ALTER TABLE statement.

a. Constraints on a Single Relation

The create table command may also include integrity-constraint statements. In addition to the primary-key constraint, there are a number of other ones that can be included in the create table command. The allowed integrity constraints include

- i. not null
- ii. unique
- iii. check(predicate)
- iv. Primary Key Constraint

L. Not Null Constraint

The null value is a member of all domains, and as a 'true' certain attribute for every attribute in SQL, by default, for inappropriate. The not null constraint prohibits the insertion of a null value for the attribute. Consider the relation in the student relation where name is null. Such a tuple does not contain useful information for an unknown student, thus, we don't want the department budget to be null. In cases such as this, we wish to forbid null values, and we can do so by restricting the domain of the attributes name and budget to exclude null values, by declaring it as follows:

budget numeric(12,2) NOT NULL.

Example:

Employee (EmpId, EmpName, Salary)

EmpId	EmpName	Salary
123	Sudip	50000
142	Kundan	60000
164	Mithun	20000
NULL	Yinod	27000

NULL is not allowed in EmpId as Primary key, contains a NULL value.

II. Unique Constraint

SQL also supports an integrity constraint: UNIQUE (A1, A2, ..., An).

The unique specification says that attributes A1, A2, ..., An form a superkey; that is, no two tuples in the relation can be equal on all the listed attributes. However, attributes declared as unique are permitted to be null unless they have explicitly been declared to be not null.

III. The Check Clause

When applied to a relation declaration, the clause check(P) specifies a predicate P that must be satisfied by every tuple in a relation.

A common use of the check clause is to ensure that attribute values satisfy specified conditions, in effect creating a powerful type system. For instance, a clause check (budget > 0) in the create table command for relation department would ensure that the value of budget is nonnegative.

As another example, consider the following:

CREATE TABLE section

(course_id VARCHAR (8),

Sec_id VARCHAR (8),

semester VARCHAR (6),

year NUMERIC (4,0),

building VARCHAR (15),

room number VARCHAR (7),

time_slot_id VARCHAR (4),

PRIMARY KEY (course_id, sec_id, semester, year),

CHECK (semester IN ('Fall', 'Winter', 'Spring', 'Summer'));

Here, we use the check clause to simulate an enumerated type by specifying that semester must be one of 'Fall', 'Winter', 'Spring', or 'Summer'. Thus, the check clause permits attribute domains to be restricted in powerful ways that most programming language type systems do not permit.

IV. Primary Key Constraint

Keys are the attributes that are used to identify an entity within its entity set uniquely. An entity set can have multiple keys, but one key will be primary key. A primary key can contain a unique value in the relational table.

Example:

ID	Name	Semester	Age
1000	Prabesh	I	17
1001	Sudhir	II	24
1002	Akhil	V	21
1003	Ashul	III	19
1002	Jay	VIII	22

Not allowed because all row must be unique

To define a PRIMARY KEY constraint on single column
CREATE TABLE Persons (

ID INT NOT NULL, PRIMARY KEY,

} Here, defining 'Int' Null is optional

Lastname VARCHAR(50)

NOT NULL,

Firstname VARCHAR(50),

Age INT);

To define a PRIMARY KEY constraint on multiple columns with constraint name
CREATE TABLE Persons (

ID INT NOT NULL,

Lastname VARCHAR(50) NOT NULL,

Firstname VARCHAR(50),

Age INT;

CONSTRAINT PK_Person PRIMARY KEY (ID, Lastname));

Here PK_Person is a name of primary key constraint.

5.3 Referential Integrity

Often, we wish to ensure that a value that appears in one relation (the referencing relation) for a given set of attributes also appears for a certain set of attributes in another relation (the

referenced relation). Such conditions are called referential integrity constraints and foreign keys are a form of a referential integrity constraint where the referenced attributes form a primary key of the referenced relation. A referential integrity constraint is specified between two tables. If a foreign key in table1 refers to the primary key of Table2, then every value of the foreign key in Table1 must be NULL or available in Table2.

Table 1

D.No	D.Location
11	Mudtnath
24	Isahat
13	Budhahare

primary Key

Table 2

Emp.ID	Name	Age	D.No
1	Pravin	20	11
2	Suman	40	24
3	Nahin	27	18
4	Balkrishna	38	13

Foreign Key

Not allowed

18 is not allowed because it's not present in Table 1.

Example:

1. To define foreign key on single column

CREATE TABLE Orders (

OrderID INT PRIMARY KEY,

OrderNumber INT NOT NULL,

PersonID INT FOREIGN KEY REFERENCES Persons (PersonID));

2. To define foreign key on multiple column with constraint name

CREATE TABLE Orders (

OrderID INT PRIMARY KEY,

OrderNumber INT NOT NULL,

PersonID INT,

LastName VARCHAR(50),

CONSTRAINT FK_PersonOrder FOREIGN KEY (PersonID, LastName)

REFERENCES Persons(PersonID, LastName);

Here 'FK_PersonOrder' is a name of foreign key constraint.

Cascading in Referential Integrity

When a referenced record in a parent table is modified or removed, cascading, a characteristic of referential integrity in database systems, decides what should happen. When cascade is enabled for a foreign key constraint, changes made to the parent table are automatically propagated to any associated child tables. There are two common types of cascading actions in referential integrity:

I. Cascading Updates

Cascading updates automatically propagate changes to the matching foreign key values in the child table(s) when a referenced record in the parent database is updated. By doing this, it is made sure that the relationships' integrity is upheld. For example, if we update the primary key value of a customer record in the parent table, the cascading update would automatically update the foreign key values in the orders table that reference that customer.

II. Cascading Deletes

Cascading deletes automatically remove the associated child records in the child table(s) when a referenced record in the parent table is deleted. By deleting any children records in the child table, the parent record that was deleted, this guarantees that the database will stay consistent. For example, if you delete a department record from the parent table, the cascading delete would automatically remove all the associated employee records from the child table.

it's important to remember that enabling cascading actions should be done cautiously because they can significantly affect the database's consistency and integrity. If cascading updates or deletes are to be enabled, careful study, understanding of the data integrity, and business requirements are required.

By using cascading actions in referential integrity, database administrators can automate the maintenance of data relationships and ensure that changes to the parent table are properly reflected in the child tables, simplifying the management of the database and reducing the risk of inconsistent data.

5.4 Assertions and Triggers

5.4.1 Assertion

An assertion is a predicate expressing a condition that we with the database always to satisfy. Domain constraints and referential integrity constraints are special forms of assertions. There are many constraints that we cannot express by using only these special forms.

Two example of such constraints are:

- I. The sum of all loan amounts for each branch must be less than the sum of all account balances at the branch
- II. Every loan has at least one customer who maintains an account with a minimum balance of 100000

Syntax:

Create assertion <assertion_name> check <predicate>;

Predicate must return true or false.

Example:

```
Creating a table
CREATE TABLE Table_1 (column_1 INT, column_2
VARCHAR(4));
Creating an assertion
CREATE ASSERTION constraint_1
CHECK ((SELECT AVG(column_1) FROM Table_1) > 50)
```


Suppose column_1 has single row with value 52.
Inserting Value in column_1:

```
INSERT INTO Table_1(column_1)  
VALUES (46);
```

This statement will violate the constraint 1 because the condition inside check clause returns false value i.e. 46>50 is false. So, the value wouldn't get inserted.

When an assertion is created, the system tests it for validity. If the assertion is valid, any further modification to the database, allowed only if it does not cause that assertion to be violated. In testing may introduce a significant amount of overhead. These assertions should be used with great care.

5.4.2 Trigger

A trigger is a statement that is executed automatically by the system as a side effect of a modification to the database.

To design a trigger mechanism, we must meet two requirements:

- I. Specify the conditions under which the trigger is to be executed. This is broken up into an event that causes the trigger to be checked and a condition that must be satisfied for trigger execution to proceed.
- II. Specify the actions to be taken when the trigger executes.

There are basically three parts of a trigger:

- I. Event
- II. Condition (optional)
If the condition is true, the trigger executes, otherwise it is skipped.
- III. Action

The actions performed by the trigger

Syntax of Triggers:
CREATE TRIGGER
trigger_name

172 | *Insights on Database Management Systems*

```
trigger_time  
trigger_event  
ON  
table_name FOR EACH ROW  
BEGIN
```

```
END;  
trigger_time----> BEFORE, AFTER  
trigger_event----> INSERT, UPDATE, DELETE  
trigger_event----> INSERT, UPDATE, DELETE
```

Example:

```
1. BEFORE INSERT Trigger  
#First of all we create a table named customers and then we  
defined trigger named age_verify to that table.  
CREATE TABLE customers(cust_id INT, age INT, name  
VARCHAR(30));
```

```
define trigger  
DELIMITER //  
CREATE TRIGGER age_verify  
BEFORE INSERT ON customers  
FOR EACH ROW  
IF new.age < 0 THEN SET new.age = 0;  
END IF; //
```

```
#Now let's insert the values to the customers  
INSERT INTO customers VALUES (101, 27, 'James'),  
(102, 40, 'Amy'),  
(103, -32, 'Ben'),  
(103, 39, 'Angela');  
In the customers table, for those customers: 102 and 103, the  
trigger automatically set the age to 0 since their age is less  
than 0. This is how before insert trigger works.
```

2. AFTER INSERT Trigger

#First of all we create a table named customers1 as message then we defined trigger named check_null_dob % that table.

CREATE TABLE customers1 (

id INT AUTO INCREMENT PRIMARY KEY,

name VARCHAR(40) NOT NULL,

email VARCHAR (30),

birthdate DATE;

CREATE TABLE message (

id INT AUTO INCREMENT,

messageid INT,

message VARCHAR(300) NOT NULL,

PRIMARY KEY (messageid);

DELIMITER //

CREATE TRIGGER check_null_dob

AFTER INSERT

ON customers1

FOR EACH ROW

BEGIN

If new.birthdate IS NULL, THEN

INSERT INTO message(messageid, message)

VALUES(new.id, concat('If: new name, Please update your

DOB!;

END//

INSERT INTO

customers1 (name, email, birthdate)

VALUES

(John Doe, 'john.doe@example.com', '2000-01-01')

| 374 | *Integrating Business Management Systems*

(John Doe, 'john.doe@example.com', NULL),
(Jane Doe, 'jane.doe@example.com', '2000-01-01');

SELECT *

FROM customers1;

id	Name	Email	birthdate
1	John Doe	john.doe@example.com	(NULL)
2	Jane Doe	jane.doe@example.com	2000-01-01

SELECT *

FROM message;

id	messageid	Message
1	1	Hi John Doe please update your Date of Birth

3. BEFORE UPDATE Trigger

Acresse employees table

CREATE TABLE employees(emp_id INT AUTO INCREMENT

PRIMARY KEY,

emp_name VARCHAR(25),

age INT,

salary FLOAT);

Insert some values in employees table

INSERT INTO employees VALUE

1,

1,

Fulltime Times insert default values

trigger trigger name: updat_trigger

DELIMITER //

CREATE TRIGGER updat_trigger

BEFORE UPDATE

ON employees

FOR EACH ROW

DECLARE CURRENT_DATE AND CURRENT_TIMESTAMP CURRENT_TIMESTAMP(1) 375 |

```

BEGIN
SET new_salary=CASE
WHEN new_salary > 10000 THEN 8500
WHEN new_salary < 10000 THEN 7200
END;
END
//

#Now perform update command
UPDATE employees
SET salary = 80000;

```

We can see that new salary is 80000 which is greater than 10000, the trigger updates the salary as 85000

FROM employees;

SELECT *

emp_id	emp_name	age	salary
1	NULL	NULL	85000
2	NULL	NULL	85000

4. BEFORE DELETE Trigger

#create a table named salarydel to store deleted details

```

CREATE TABLE salarydel(emp_id INT,
emp_name VARCHAR(25),
age INT,
salary FLOAT);

```

#create trigger named: salary_delete

```

DELIMITER //
CREATE TRIGGER salary_delete
BEFORE DELETE
ON employees
FOR EACH ROW

```

176 | Insights on Database Management Systems

```

BEGIN
INSERT INTO salarydel(emp_id, emp_name, age, salary)
VALUES( old.emp_id, old.emp_name, old.age, old.salary);
END //

#Now delete emp_id = 1 from employees
DELETE FROM employees
WHERE emp_id = 1;

```

#successfully deleted

SELECT *

FROM employees;

emp_id	emp_name	age	salary
2	NULL	NULL	85000

SELECT *

FROM salarydel;

emp_id	emp_name	age	salary
1	NULL	NULL	85000

5.5 Functional Dependencies

Functional dependency is a relationship that exists when one attribute uniquely determines another attribute. If R is a relation with attributes X and Y, a functional dependency between the attributes is represented as X → Y, which specifies Y is functionally dependent on X. Here X is a determinant set and Y is value attribute. Each value of X is associated with precisely one value. Functional dependency in a database serves as a constraint between two sets of attributes.

Defining functional dependency is an important part of relational database design and contributes to aspect normalization. Functional dependency typically exists between the primary key and non-key attribute within a table.

$X \rightarrow Y$

The left side of FD is known as determinant, the right side of the production is known as dependent

Example:

Assume we have an employee table with attributes: Emp_Name, Emp_Address

Let employee(Emp_Id, Emp_Name, Emp_Address)

Emp_Id	Emp_Name	Emp_Address
1234	Bishon	Parbat
4567	Nabin	Birtwal

Here Emp_Id attribute can uniquely identify the Emp_Name attribute of employee table because if we know the Emp_Id we can tell the employee's name associated with it.

Functional dependency can be written as:

Emp_Id $\rightarrow$ Emp_Name
We can say that Emp_Name is functionally dependent on Emp_Id

Compound Determinants
If more than one attributes are necessary to determine another attribute in an entity, then such determinant is termed as compound or composite determinant.

Full Functional Dependency
This is the situation when it is necessary to use all the attributes of composite determinant to identify its object uniquely. A and B are attributes of a relation, B is fully functionally dependent on A if B is functionally dependent on A but not on any proper subset of A.

Partial Functional Dependency

This is the situation that exist if it is necessary to only use a subset of the attributes of the composite determinant to identify its object uniquely. A functional dependency $A \rightarrow B$ is partially dependent if there are some attributes that can be removed from A and yet the dependency holds.

Types of Functional Dependency:

Trivial functional dependency
Non-trivial functional dependency

Trivial Functional Dependency

1. $A \rightarrow B$ has trivial dependency if B is a subset of A. The following dependencies are also trivial:
 $A \rightarrow A$, $B \rightarrow B$

Example:
Consider a table with two columns Employee_Id and Employee_Name

[Employee_Id, Employee_Name] $\rightarrow$ Employee_Id is a trivial functional dependency as:

Employee_Id is subset of [Employee_Id, Employee_Name]

Also,

Employee_Id $\rightarrow$ Employee_Id
Employee_Name $\rightarrow$ Employee_Name are trivial dependencies too.

Non-trivial Functional Dependency

1. $A \rightarrow B$ has non trivial functional dependency if B is not a subset of A.
When A intersection B is NULL, then $A \rightarrow B$ is called as complete non-trivial.

Company	CEO	Age
Microsoft	Satya Nadella	51
Google	Sunder Pichai	46
Apple	Tim Cook	58
Amazon	Andy Jassy	35

Example:

ID $\rightarrow$ Name

Name $\rightarrow$ DOB

Company $\rightarrow$ CEO

But FD is not a subset of company and hence Armstrong's functional dependency.

Armstrong's Axioms property of functional dependency

Armstrong's Axioms property was developed by William Armstrong in 1974 to reason out about functional dependencies.

i. Transitivity

If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$, i.e. transitive relation.

ii. Reflexivity

$A \rightarrow B$, if B is a subset of A.

iii. Augmentation

The last rule suggests: $AC \rightarrow BC$, if $A \rightarrow B$.

Characteristics of Functional Dependency

There is one-to-one relationship between the attributes on the left-hand side (determinant) and those on the right-hand side (determinands) have the minimal number of attributes necessary to maintain the dependency with the attributes on right hand side.

Closure of a set of Functional Dependencies

A closure is a set of all possible FDs that can be derived from given set of FDs. Closure set is also referred as a complete set of FDs. If F is used to denote the set of FDs for relation R, then a closure of a set of FDs implied by F is denoted by F^+ . Let's $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$.

From F it is possible to derive the following dependencies:

- $A \rightarrow A$
- $A \rightarrow B$
- $A \rightarrow C$
- $A \rightarrow D$
- $A \rightarrow ABCD$
- $A \rightarrow ABCD$ (Union)
- (Closure of A under F)

All such type of FDs derived from each FD of F form closure of F.

Additional Rules

Union Rule

1. If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds then $\alpha \rightarrow \beta\gamma$ holds

Decomposition Rule

ii. If $\alpha \rightarrow \beta\gamma$ holds then $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$ holds

Pseudo-transitivity Rule

iii. If $\alpha \rightarrow \beta$ holds and $\beta\gamma \rightarrow \delta$ holds then $\alpha \rightarrow \gamma\delta$ holds

Steps to determine F^+

1. Determine each set of attributes X that appears as a left hand side of some FD in F.

2. Determine the set X^+ of all attributes that are dependent on X.

3. All such set of X^+ . In combine form closure of F.

Example:

For given FD, find all possible F^+

$K = \{A, B, C, G, H, I\}$

$F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, G \rightarrow I, B \rightarrow H, CG \rightarrow I\}$

Some members of F^+

$A \rightarrow \{I\}$ from $A \rightarrow B$ and $B \rightarrow H$ by transitivity

$AG \rightarrow \{I\}$ by augmenting $A \rightarrow C$ with G to get $AG \rightarrow CG$ and then transitivity with $CG \rightarrow I$

$CG \rightarrow \{H, I\}$ by augmentation

Transitive Dependency

A functional dependency is said to be transitive if it is indirectly formed by two functional dependencies.

Example:

$X \rightarrow Z$ is a transitive dependency if the following three functional dependencies hold true:

$X \rightarrow Y$

Y does not $\rightarrow$ X
Y $\rightarrow$ Z

A transitive dependency can only occur in a relation of three or more attributes.

Book	Author	Author Age
Game Of Thrones	George R. R. Martin	72
Harry Potter	J. K. Rowling	55
Dying Of The Light	George R. R. Martin	72

{Book} $\rightarrow$ {Author} (if we know the book, we know the author name)

{Author} does not $\rightarrow$ {Book}

{Author} $\rightarrow$ {Author Age}

Therefore as per the rule of transitive dependency:
{Book} $\rightarrow$ {Author Age} should hold, that makes sense because if we know the book name we can know the author's age.

5.6 Multivalued Dependencies

	A	B	C
Name	Project	Hubby	
T1	Yuvraj	MS	Reading
T2	Yuvraj	Oracle	Music
T3	Yuvraj	MS	Music
T4	Yuvraj	Oracle	Reading

Here project and hubby are multivalued attributes because they contain the different values for the same name (Yuvraj).

Attributes (Columns): A, B, C

Tuples (Rows): T1, T2, T3, T4

R = set of attributes, r = relation

We can say that multivalued dependency exists if the following conditions are met

Any attribute say 'A' multiple define another attribute B; if any legal relation $r(R)$, for all pairs of tuples T1 and T2 in r, such that

$r1[A] = T2[A]$

Then there exists t3 and t4 in r such that

$t1[A] = T2[A] = T3[A] = T4[A]$

$t1[B] = T3[B]$

$t3[R-B] = T2[R-B]$

$t2[B] = T4[B]$

$t4[R-B] = T1[R-B]$

Then multivalued (MVD) dependency exists.

To check the MVD in given table, we apply the conditions stated above and we check it with the values in the given table.

$T1[A] = T2[A] = T3[A] = T4[A] = \text{Yuvraj}$

$T1[B] = T3[B] = \text{MS}$

$T3[R-B] = T2[R-B] = \text{Yuvraj, Music}$

$T2[B] = T4[B] = \text{Oracle}$

$T4[R-B] = T1[R-B] = \text{Yuvraj, Reading}$

Hence, we know that MVD exists in the above table and it can be stated by,

name $\rightarrow \rightarrow$ project

name $\rightarrow \rightarrow$ hobby

5.7 Normalization and Normal Forms

Normalization is the process of restructuring the logical data model of a database to eliminate redundancy, organize data efficiently and reduce repeating data and to reduce the potential for anomalies during data operations.

Database Normalization is a technique of organizing the data in the database. Normalization generally involves splitting existing tables into multiple ones which must be joined or link each time a query is issued. Normalization is multi-step process. Normalization is the process of efficiently organizing data in database with two goals:

1. Eliminate redundant data

i.e. Ensure data dependencies make sense

Data normalization also may improve data consistency by simplifying future extension of the logical data model. The formal classifications used for describing a relational database's level of normalization are called normal forms. Normal Forms is abbreviated as NF.

What happens without Normalization?

- > A non-normalized database can suffer from data anomalies.
- > A non-normalized database may store data representing a particular recurrent in multiple locations.
- > An update to such data in some but not all of those locations results in an update anomaly, yielding inconsistent data.
- > A non-normalized database may have inappropriate dependencies, i.e. relationships between data with no functional dependencies.

Example

Roll No	Name	Branch	HOD	Office Tel
1	Jagdish	BCT	Hari	01-5337
2	Yyaga	BCT	Hari	01-5337
3	Suman	BCT	Hari	01-5337
4	Shankar	BCT	Hari	01-5337

Insertion Anomaly

Suppose for a new admission unit and unless a student opts have to see the branch information as NULL.

If we have to insert data of 100 student of same branch then the branch information will be repeated for all those 100 students.

Update Anomaly

If Hari is no longer the HOD of computer department or what if har leaves the college?

Redundancy

In our student information and branch information.

at the end of the academic year if student records are deleted we will also lose the branch information. This is deletion anomaly.

Advantages of Normalization

- > Elimination of redundant data storage.
- > Closed modeling of real world entities, processes and their relationship.
- > Structuring of data so that model is flexible.
- > Less storage space.
- > Easier to add data and avoid certain updating anomalies.
- > Facilitate the enforcement of data constraints.

Normalized Database

Normalized databases have a design that reflects the true dependencies between tracked quantities, allowing quick updates to data with little risk of introducing inconsistencies. Instead of attempting to lump all information into one table, data is spread out logically into many tables.

Normalizing the data is decomposing a single relation into a set of smaller relations which satisfy the constraints of the original relation. Redundancy can be solved by decomposing the tables. However certain new problems are caused by decomposition. Normalization helps us to make a rational decision to avoid redundancy keeping the pros and cons in mind.

One can only describe a database as having a normal form if the relationships between quantities have been rigorously defined. It is possible to use set theory to express this knowledge once a problem domain has been fully understood, but most database designers model the relationships in terms of an "idealized schema". [The mathematical support came back into play in proofs regarding the process of transforming from one

form to another.) The transformation of conceptual model to computer representation format is known as normalization.

Stages of Normalization

First Normal Form (1NF)

A relational schema R is in first Normal form if the domains of all attributes of R are atomic. Non-atomic values complicate storage and encourage redundant(repeated) storage of data. To transform the un normalized table(a table that contains one or more repeating groups) to first normal form, identify and remove the repeating groups within the table, (ie multi valued attributes, composite attributes and their combinations).

Un Normalized Table

Emp_id	Ename	Project
1	Subodh	A, B
2	Anuj	C, D

Table in 1 NF

Emp_id	Ename	Project
1	Subodh	A
2	Subodh	B
3	Anuj	C
4	Anuj	D

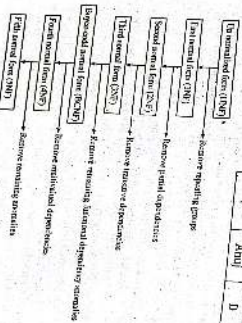


Figure: Stages of Normalization

second Normal Form (2NF)

A table is in 2NF if and only if it is in 1NF and has no attribute which require only part of the key to uniquely identify them. For a table to be in 2NF we need to remove partial dependency. Where a key has more than one attribute, check that each non key attribute depends on the whole key and not part of the key. For each subset of the key which determines an attribute or group of attributes create new form. Move the dependant attributes to the new form. Add the part key to new form, making it the primary key. Mark the part key as a foreign key in the original form.

Example:

Department(Project Name, Employee Name, Emp_Hire Date, Project_Manager)

Project Name and Employee Name are the candidate key for this table. Emp_Hire Date and Project_Manager are partially depend on the Employee Name, but not depend on the project Name. Therefore, this table will not satisfy the 2NF

In Order to satisfy the second Normal form, We need to put the Emp_Hire Date and Project Manager to other tables. We can put the Emp_Hire Date to the Employee table and put the project_Manager to the Project table.

So now we have three tables:

Department(Project Name, Employee Name)
 Project(Project_ID, Project_Name, Project_Manager)
 Employee(Employee_ID, Employee Name, Employee_Hire Date)

Now, the Department table will only have the candidate key left.

Emp_id	Proj_id	Ename	Pname
1	Emp_100	Subodh	HRMIS
1	Emp_200	Subodh	ETRMIS
2	Emp_100	Anuj	HRMIS
2	Emp_200	Anuj	ETRMIS

Here, Pname can be uniquely determined by ProjId only as pname is partially dependent on ProjId.

Converting into 2nd normal form | Removing Pname dependency.

EmpId	ProjId	Pname
1	Emp_100	Subodh
1	Emp_200	Subodh
2	Emp_100	Anuj
2	Emp_200	Anuj

ProjId	Pname
Emp_100	HRMS
Emp_200	ETRMS

Third Normal Form (3NF)

- A relation is in third normal form if and only if it is:
 - In second normal form
 - Every non key attribute is non transitively dependent on the primary key.

As attribute C is transitively dependent on attribute A if there exists an attribute B such that A → B and B → C, then A → C.

EmpId	Pname	Position	PnameDept
1	Subodh	Software Engineer	IT
2	Rajesh	System Admin	SA

Deptname and Deptid are separated and a new table is made and new column deptno is added to it.

EmpId	Pname	Position	Deptno
1	Subodh	Software Engineer	1
2	Rajesh	System Admin	2

3rd Codd Normal Form (BCNF)

- A relation is in BCNF if for every FD A → B either
 - A is contained in A (the FD is trivial), or
 - B is contained in A (the FD is a candidate key of the relation)

A contains a candidate key of the relation

in other words:

Every determinant in a non-trivial dependency is a key. If there is only one candidate key then 3NF and BCNF are the same.

EmpId	Project	phred
1	HRMS	Manju
1	ETRMS	Bidur
2	CTRMS	Rajesh
2	FARRMS	Deva

Here, empId and project together makes a primary key. This table satisfies 1st, 2nd and 3rd normal form but it doesn't satisfy BCNF.

Why the table doesn't satisfy BCNF?

To satisfy BCNF, table must be in 3rd normal form and for all FD: X → Y within the table X must be super key (prime attribute). It satisfies 1st condition but not the second one.

Since, empId and project together makes a primary key empId and project both are prime attributes.

In the table, phred attribute determines project since each project head leads only one project at a time, i.e. (1-1 relationship). As project head is a non prime attribute it fails the 2nd condition.

EmpId	Project	phred
1	1	1
1	2	2
2	3	3
2	4	4

Project	phred
1	Manju
2	Bidur
3	Rajesh
4	Deva

Fourth Normal Form (4NF)

For a table to satisfy fourth normal form it should satisfy following two conditions:

1. It should be in BCNF
2. And the table should not have any non-trivial dependency

S.Id	Course	Hobby
1	Science	Cricket
	Maths	Hockey
2	CP	Cricket
	Phy	Football

S.Id 1 has opted for two courses, science and Maths and has two hobbies cricket and hockey which will lead to the following table.

S.Id	Course	Hobby
1	Science	Cricket
	Maths	Hockey
1	Science	Hockey
	Maths	Cricket

Here certain attributes are independent of each other

S.Id	Course	Hobby
1	Science	Cricket
1	Maths	Hockey
2	CP	Hockey
2	Phy	Cricket

5.8 Decomposition of Relation Schemes

When a relation in the relational model is not in appropriate normal form then the decomposition of relation is required. If the relation has no proper decomposition then it may lead to

like loss of information. Decomposition is used to prevent some of the problems of bad design anomalies, redundancies and redundancy

Types of Decomposition

1. Lossless Decomposition
2. Lossy Decomposition

Lossless Decomposition

1. Let R be a relation schema, and let F be a set of functional dependencies on R . Let R_1 and R_2 form a decomposition of R . We say that the decomposition is a 'lossless decomposition' if there is no loss of information by replacing R with two relation schemes R_1 and R_2 .

The decomposition is lossless if, for all legal database instances (that is, database instances that satisfy the specified functional dependencies and other constraints), relation r contains the same set of tuples as the result of the following SQL query:

SELECT *

FROM (SELECT R1 FROM r_1

NATURAL JOIN (SELECT R2 FROM r_2))

This is stated more succinctly in the relational algebra as:

$$R \bowtie (r_1 \bowtie r_2) = R$$

In other words, if we project r into R_1 and R_2 , and compute the natural join of the projection results, we get back exactly r . To check for lossless join decomposition using FD set, following conditions must hold:

1. Union of Attributes of R_1 and R_2 must be equal to attribute of R . Each attribute of R must be either in R_1 or in R_2 .
2. $Att(R_1) \cap Att(R_2) = Att(K)$
3. $Att(R_1) \cap Att(R_2) \neq \emptyset$
4. Common attributes must be a key for at least one relation (R_1 or R_2)

$\text{Att}(R_1) \cap \text{Att}(R_2) \rightarrow \text{Att}(R_1) \cup \text{Att}(R_2)$
 $\text{Att}(R_2)$

Example:

A relation $R(A, B, C, D)$ with FD set $\{A \rightarrow BC\}$ is decomposed into $R_1(ABC)$ and $R_2(AD)$ which is a lossy decomposition as:

First condition holds true as:

$$\text{Att}(R_1) \cup \text{Att}(R_2) = (ABC) \cup (AD) = (ABCD) = \text{Att}(R)$$

Second condition holds true as:

$$\text{Att}(R_1) \cap \text{Att}(R_2) = (ABC) \cap (AD) \neq \emptyset$$

Third condition holds true as:

$\text{Att}(R_1) \cap \text{Att}(R_2) = A$ is a key of $R_1(ABC)$ because $A \rightarrow BC$ is given.

Decomposition of $R = (A, B, C)$

$$R_1 = (A, B)$$

$$R_2 = (B, C)$$

A	B	C
a	1	A
b	2	B

A	B
a	1
b	2

B	C
1	A
2	B

A	B	C
a	1	A
b	2	B

$$R_1 \cup R_2 = R$$

A	B	C
a	1	A
b	2	B

2. Lossy Decomposition

As the name suggests, when a relation is decomposed into two or more relational schemas, the loss of information is unavoidable when the original relation is retrieved.

Consider there is a relation R which is decomposed into relations $R_1, R_2, \dots, R_n$.

This decomposition is called lossy join decomposition when the join of the sub relations does not result in the same relation R that was decomposed.

The natural join of the sub relations is always found to have some extraneous tuples.

For lossy join decomposition, we always have:
 $R_1 \bowtie R_2 \bowtie R_3 \dots \bowtie R_n \supset R$

As an example of a lossy decomposition, the decomposition of the employee schema into employee1 and employee2

employee1 (ID, name, street, city, salary)

employee2 (name, street, city, salary)

The result of employee1 $\bowtie$ employee2 is a superset of the original relation employee, but the decomposition is lossy since the join result has lost information about which employee identifiers correspond to which addresses and salaries, in the case where two or more employees have the same name.

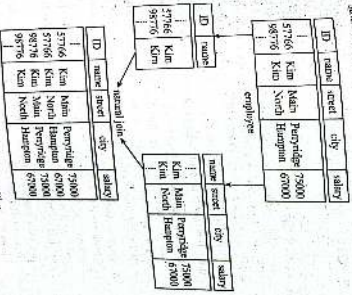


Figure: Lossy decomposition

SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS

1. Consider the following relation where (M_ID and P_ID) are primary keys. State in which normal form the relation is. What anomalies can occur in this relation? How can these anomalies be removed?

M_ID	M_Date	P_ID	Quantity
M11	16 June, 2022	11	20
M11	26 June, 2022	16	30
M22	3 September, 2022	15	20
M22	13 September, 2022	16	60
M22	23 September, 2022	12	35

- Def 4.1**
In each attribute has atomic values and there are no repeated groups, this relation is currently in the First Normal Form. To satisfy the Second Normal Form (2NF), i.e., M_Date does not fully depend on (M_ID, P_ID).

Anomalies that can occur in this relation include:

- Insertion Anomaly:** If we don't have all the necessary data, we can't insert a new row. For instance, we cannot add a row if we wish to introduce a new manufacturer (M_ID) but don't know the corresponding purchase (P_ID).
- Update Anomaly:** Multiple rows must be updated whenever the value of an attribute is changed. For instance, updating all the rows concerning a P_ID is required if we wish to change the quantity for a certain product (P_ID).
- Deletion Anomaly:** unintentionally deleting a row could result in the loss of data that is required for other purposes. For instance, if we remove the purchase data

entry with P_ID = 15, the data regarding the manufacturer made on September 3, 2022 (M_ID = M22) is lost.

To remove these anomalies, we can normalize the relation further. Here's one possible approach:

Create a separate table for "Manufacturer Details" with attributes: M_ID and M_Date and (M_ID, M_Date) as primary key.

Manufacturer Details Table:

M_ID	M_Date
M11	16 June, 2022
M11	26 June, 2022
M22	3 September, 2022
M22	13 September, 2022
M22	23 September, 2022

Create another table for "Purchase Details" with attributes: M_ID (foreign key referencing Manufacturer_Details table), P_ID (foreign key referencing the relevant products table), and Quantity. Primary key would be (M_ID, P_ID).

Purchase Details Table:

M_ID	P_ID	Quantity
M11	11	20
M11	16	30
M22	15	20
M22	16	60
M22	12	35

By separating the data into two tables, we eliminate the partial dependencies and achieve Second Normal Form (2NF). This structure allows us to insert, update, and delete data without considering the aforementioned anomalies.

2. When do you use triggers?

Ans Triggers in the database are used to automatically execute a set of actions or instructions in response to certain events, or changes in the database. Triggers are typically used in the following scenarios:

1. Enforcing Data Integrity

Triggers can be used to enforce complex business rules and data validation constraints that cannot be easily expressed using standard constraints and rules. For example, we can create a trigger that checks that certain conditions are met before allowing an insert or update operation.

II. Auditing and Logging

For auditing purposes, triggers can be used to record changes made to particular tables or columns. They can automatically log information about modifications, including who made them, when they happened, and what was modified.

III. Synchronization of Data

Data synchronization between various tables or databases is possible with the help of triggers. To preserve data consistency, for instance, a trigger can be used to automatically update related tables when a change is made in one table.

IV. Business Logic Enforcement

Specific business logic or rules that are difficult to define through constraints can be enforced with the help of triggers. This could include complex calculations, derived values, or workflow-related operations.

V. Data Transformation

Prior to being entered, modified, or deleted, data can be transformed using triggers. This can involve applying particular formatting standards, normalizing the data, or converting the data.

What is the role of Triggers? Write an SQL trigger to carry out the following action: On delete of an account, for each owner of the account, check if the owner has any remaining amount, and if she does not, delete her from the depositor relation. [2021 Fall]

Ans The role of a trigger is to enforce certain business rules, maintain data integrity, or perform other automated actions based on changes in the database.

```
DELIMITER //
CREATE TRIGGER delete_owner_trigger
AFTER DELETE ON account
FOR EACH ROW
BEGIN
    -- Declare a variable to hold the count of remaining
    -- accounts
    DECLARE remaining_accounts INT;
    --for the owner
    DECLARE remaining_accounts INT;
```

```
-- Check if the owner has any remaining amount
SELECT COUNT(*) INTO remaining_accounts FROM
depositor
WHERE account_number =
OLD.account_number and oldbalance=0;
```

```
-- If the owner has no remaining accounts, delete her from
--the depositor relation
IF remaining_accounts = 1 THEN
DELETE FROM depositor WHERE account_number
=OLD.account_number;
```

```
END IF;
END
//DELIMITER;
```

4. Illustrate redundancy and the problems that it can cause. [2020 Spring]

Ans Redundancy refers to the duplication or repetition of data within a database or system. It occurs when the same information is stored multiple times in different locations or

in multiple records within the same location. While a certain level of redundancy is sometimes necessary for efficient retrieval and system performance, excessive redundancy can lead to various problems and challenges. Here are some problems that redundancy can cause:

a. Data Inconsistency

The chance of data errors or inconsistencies rises with redundancy. Any update, insertion, or deletion of data which is stored in several locations must be carefully handled to keep all copies of the data synchronized. There may be discrepancies and faults in the system if one copy of the data is changed but others are not correctly updated.

b. Increased Storage and Maintenance Costs

Redundancy uses more storage space and raises the price of maintaining and storing data. The same data must be stored more than once, consuming additional resources and storage space. Higher operational costs are caused by the complexity and time required for updating and keeping redundant data.

c. Update Anomalies

Update anomalies—Inconsistencies that happen when redundant data is updated—can be introduced by record but not the others can cause inconsistencies and confusion if the same customer address is stored in several records and the address changes.

d. Decreased Data Integrity

Issues with data integrity are more likely when there is redundant data. Enforcing constraints, integrity rules, and referential integrity becomes more difficult when data is copied across several locations. As a result, the system's dependability and accuracy may suffer from inaccurate or inconsistent data.

e. Poor Performance and Efficiency

Excessive redundancy can affect the effectiveness and performance of a system. The complexity of searches and

transactions is increased by the additional processing needed to retrieve and update redundant data. This may result in a decrease in system performance and general efficiency.

Difficulties in Data Integration

When integrating data from several sources or systems, redundant data might provide problems. When redundant data is maintained independently by each source, combining and resolving the data is difficult and prone to mistakes. A unified and correct representation of the data may be difficult to achieve due to inconsistencies and conflicts.

Increased Risk of Data Anomalies

Redundancy can result in data anomalies, when the system stores false or misleading information. Redundant data can contain flaws and inconsistencies that can spread across the system and result in inaccurate analysis, judgments, and reporting.

5. Explain the role of Functional Dependency in the process of Normalization.

Functional dependency helps in identifying and eliminating data redundancy by ensuring that each attribute in a database table depends on the primary key of the table.

In a database table, if an attribute X is functionally dependent on an attribute Y, it means that for every value of Y, there is a unique value of X. In other words, knowing the value of Y can determine the value of X. Functional dependency helps in identifying redundant data because if two attributes are functionally dependent on each other, it means that they contain the same information and one of them can be removed without losing any information.

6. Illustrate the advantage of 4NF with suitable example.

Fourth Normal Form (4NF) is a level of database normalization that deals with multi-valued dependencies.

To illustrate the advantage of 4NF, let's consider an example of a database for a library. Suppose we have the following table of information about books:

Books(BookID, Title, Author, Genre, DatePublished)

In this table, the primary key is "BookID". There is no partial dependency or transitive dependency, so it satisfies the third normal form. However, it has a multi-valued dependency between the "Title" and "Author" attributes, as a book can have multiple authors and an author can have written multiple books. This means that the table violates the fourth normal form.

To convert this table into 4NF, we can decompose it into two tables:

Books(BookID, Title, Genre, Publisher, DatePublished)
Authors(BookID, Author)

In this case, the "Books" table contains information about books, including the book's title, genre, publisher, and date of publication. The "Authors" table contains information about the authors, including the author's name and the books they have written.

This decomposition eliminates the multi-valued dependency and ensures that the table is in 4NF. This is advantageous because it reduces redundancy and simplifies the process of updating the database. For example, if an author publishes a new book, we only need to add a new row to the "Books" table and a new row to the "Authors" table, rather than updating multiple rows in a single table. This makes it easier to maintain data consistency and avoid data anomalies.

7. Differentiate between Functional Dependency and Multi Valued Dependency.

ANS Functional Dependency refers to a relationship between two or more attributes in a database table, where the value of one attribute determines the value of another attribute. More specifically, if attribute A is functionally dependent on attribute B, it means that for any given value of A, there can only be one value of B.

For example, in a table that stores information about employees, the attribute "Salary" may be functionally dependent on the attribute "EmployeeID", since each employee has only one salary.

On the other hand, Multi-Valued Dependency refers to a relationship between three or more attributes in a database table, where the value of one or more attributes determines the value of another set of attributes, but not all of them. This means that for any given value of the determining attributes, there may be multiple possible combinations of values for the dependent attributes.

For example, in a table that stores information about movies and their directors, the director's name may be multi-valued dependent on the movie's title, since a movie may have multiple directors and a director may direct multiple movies.

5. Convert the following 2NF relation into 3NF(consider Name as primary key)

Name	Address	Phone	Salary	Post
Gill	KTM	456789	20000	Engineer
Van	BKT	654321	20000	Engineer
Robert/KTM		456789	20000	Engineer
Brown/BKT		654321	10000	Overseer
Albert/KTM		456789	10000	Officer

ANS To convert the given 2NF relation into 3NF, we need to identify and remove any transitive dependencies.

The given relation already satisfies the first normal form as each attribute contains atomic values, and it also satisfies the second normal form as there are no partial dependencies on any candidate keys (in this case, the primary key is "Name").

However, there is a transitive dependency between the "Name" and "Salary" attributes, as the "Salary" attribute is dependent on the "Name" attribute through the "Post" attribute. This means that the "Salary" attribute is directly dependent on the primary key "Name", and it violates the third normal form.

Also, there is a transitive dependency between the "Name" and "Address" attributes, as the "Address" attribute is dependent on the "Name" attribute through the "Phone" attribute. This means that the "Address" attribute is directly dependent on the primary key "Name", and it violates the third normal form.

To remove these transitive dependencies and convert the relation into 3NF, we can decompose the original relation into three separate relations:

- Relation 1: Name, Phone, Post
- Relation 2: Post, Salary
- Relation 3: Phone, Address

Relation 1 satisfies the third normal form as all the attributes are directly dependent on the primary key "Name". Relation 2 and Relation 3 also satisfy the third normal form as there are no transitive dependencies.

The final relations in 3NF will look like this:

Name	Phone	Post
Gill	456789	Engineer
Vao	654321	Engineer
Robert	456789	Engineer
Brown	654321	Overseer
Albert	454545	Officer

relation 2:

Post	Salary
Engineer	20000
Engineer	20000
Engineer	20000
Overseer	10000
Officer	10000

relation 3:

Phone	Address
456789	KTM
654321	BKT
454545	KTM

3. Suppose we are given Schema $R = \{A, B, C, G, H, I\}$ and set of functional dependencies $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, B \rightarrow H, CG \rightarrow I\}$. Find the closures of functional dependency F . [2017 Feb]

Q3 We can derive the closures of the given functional dependencies as follows:

- $A \rightarrow A$
- $A \rightarrow B$
- $A \rightarrow H(A \rightarrow B \rightarrow H)$
- $A \rightarrow C$
- $CG \rightarrow H$
- $CG \rightarrow I$ (Using Pseudotransitivity Rule i.e. $A \rightarrow CG \rightarrow I$ then $CG \rightarrow I$)
- $AG \rightarrow I$ (Using Pseudotransitivity Rule i.e. $A \rightarrow CG \rightarrow I$ then $AG \rightarrow I$)
- $B \rightarrow B$

B → H
C → C

G → C

H → H

I → I

Therefore, the closures of functional dependency F are:
 $F^+ = A \rightarrow B, B \rightarrow H, A \rightarrow C, CC \rightarrow H, AG \rightarrow H, CC \rightarrow I, AG \rightarrow J$

10. Compare and Contrast assertion & triggers.

Ans Assertions and triggers are two important database concepts that are used for ensuring data integrity and enforcing business rules. Though they serve similar purposes, there are some differences between them.

Assertions are declarative statements that specify a condition that must be true at all times in the database. They are used to enforce business rules and data integrity constraints that cannot be enforced using other database mechanisms such as primary keys, foreign keys, and check constraints. Assertions are evaluated whenever the database is modified, and if the condition specified in the assertion is not satisfied, the modification is rejected. Assertions are checked at the end of a transaction.

Triggers, on the other hand, are procedural code fragments that are executed in response to certain events, such as the insertion, deletion, or modification of data in a table. Triggers can be used to enforce complex business rules that cannot be enforced using other database mechanisms. They are written using SQL or other programming languages, and they can access the data in the database and modify it if necessary. Triggers can be executed before or after an event, and they can be defined to execute either once or multiple times. The main differences between assertions and triggers are:

- **Declaration:** Assertions are declarative statements that define a condition that must be satisfied, whereas triggers are procedural code fragments that execute in response to an event.

Purpose: Assertions are used to enforce data integrity constraints and business rules that cannot be enforced using other database mechanisms, while triggers are used to enforce complex business rules that cannot be enforced using other database mechanisms.

Evaluation time: Assertions are evaluated at the end of a transaction, while triggers are executed in response to an event.

Language: Assertions are written using SQL, while triggers can be written using SQL or other programming languages.

Consider following bank database:

If Branch-schema = (branch-name, branch-city, assets)

loan-schema = (loan-number, branch-name, amount)

Write an assertion for the bank database to ensure that the Assets value for the Koteshwar branch is equal to the sum of all the amounts lent by the Koteshwar branch.

Ans CREATE ASSERTION assets_equal TO sum of lent amounts

CHECK (

```
(SELECT SUM(amount) FROM loan WHERE branch_name = 'Koteshwar') = (SELECT assets FROM Branch WHERE branch_name = 'Koteshwar'));
```

12. Design relational database for the Dept. of Computer Engineering (DOCE) at Pokhara University. Your database should have at least three (3) relations. Describe referential integrity constraint based on the above database of DOCE.

Ans To design a relational database for the Department of Computer Engineering (DOCE) at Pokhara University, we can create following three relations:

Students: This relation will store information about the students enrolled in the department. It can include attributes such as Student ID, Name, Gender, Address, Email, and Year of Enrollment.

Courses: This relation will store information about the courses offered by the department. It can include attributes such as Course ID, Course Name, Course Code and Semester.

Enrollments: This relation will store information about the enrollment of students in courses. It can include attributes such as Student ID (foreign key referencing Students relation), Course ID (foreign key referencing Courses relation), and Enrollment Date.

Referential Integrity Constraints:

To maintain referential integrity in the above database, we can enforce the following constraints:

Foreign Key Constraint (Enrollments table):

The foreign key constraint on the Enrollments table ensures that the Student ID and Course ID values in the Enrollments table reference existing Student ID and Course ID values in the Students and Courses tables, respectively.

This constraint ensures that students can only be enrolled in existing courses offered by the department, and the information in the Enrollments table remains consistent.

Example of Foreign Key Constraint in Enrollments table:

```
CREATE TABLE Enrollments (  
    EnrollmentID INT PRIMARY KEY,  
    StudentID INT,  
    CourseID INT,  
    EnrollmentDate DATE,  
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),  
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID))
```

By enforcing this foreign key constraint, any insert or update operation on the Enrollments table will be checked against the referenced tables (Students and Courses) to ensure the existence of the corresponding Student ID and Course ID

values. If the referenced values don't exist, the constraint will prevent the operation from being performed, maintaining the referential integrity of the database.

15 Explain the roles of Assertions and Triggers in SQL. When Triggers are not appropriate to use? Give an example.

example. Database features like assertions and triggers give SQL data manipulation and validation more flexibility and control. Here is a description of the roles they play and an illustration of a situation in which triggers would not be suitable to use:

Assertions

Assertions serve as a means of expressing complex integrity rules that are difficult to enforce using other constraints, such as check constraints or key constraints. The formulation of conditions involving numerous tables and relationships is possible by them. For instance, an assertion can guarantee that every "Employee" record in the database has a corresponding "Department" record.

Triggers:

Triggers are used to enforce complicated business rules or carry out extra tasks that cannot be completed by constraints alone. They can be used to safeguard data integrity, set up auditing procedures, propagate changes to connected tables, or set off notifications in response to particular circumstances. When a "Employee" record is edited, a trigger can be set up to update a "LastModifiedDate" column.

When Triggers Are Not Appropriate to Use:

Although triggers provide flexibility and extensibility, there are several circumstances in which their use may not be suitable. When complex business logic or extensive processing is required, this is one instance. In general, triggers should be kept quick and light to prevent slowing down activities that modify data. It may be better to handle the logic at the application layer or via stored procedures if it becomes too complex or resource intensive.

For instance, consider a scenario where an e-commerce system needs to calculate and update order totals based on product prices and quantities whenever an order is placed or modified. If the calculation involves complex pricing rules, discounts, calculations, or involves interactions with external systems, implementing it solely through triggers may lead to performance issues and make the system harder to maintain. In such cases, it may be more appropriate to handle the calculations and updates in a separate stored procedure or application code, where it can be optimized and managed more effectively.



SECURITY

CHAPTER - 6

Needs of Security

6.1

Security refers to the protection from malicious attempts to steal or modify data. The need for database security is given below:

- In the case of shared data, multiple users try to access the data at the same time. In order to maintain the consistency of the data in the database, database security is needed.
- Due to the advancement of internet, data are accessed through World Wide Web, to protect the data against hackers, database security is needed.

- The plastic money (Credit card) is more popular. The money transaction has to be safe. More specialized software both to enter the system illegally to extract data and to analyze the information obtained is available.

Hence, it is necessary to protect the data/money

Security in different level are as follows

i. Database System Level

In database system, authentication and authorization mechanisms can be implemented to allow specific users access only to required data

ii. Operating System Level

Operating system super-users can do anything they want to the database so good operating system level security is required.

iii. Network Level

Network must use encryption to prevent eavesdropping (unauthorized reading of messages), Masquerading (pretending to be an authorized user or sending messages supposedly from authorized users)



CamScanner

7. Physical level

Physical access to computers allows destruction of data by intruders; traditional lock-and-key security is needed. Computers must also be protected from floods, fire, etc.

8. Human level

Users must be screened to ensure that an authorized user does not give access to intruders. Users should be trained in password selection and secrecy.

6.2. Security and Integrity Violations

Security and integrity violations in a database can occur for various reasons such as human error, software bugs, hardware failure, malicious attacks, or natural disasters. These violations can result in data breaches, data corruption, data loss, or unauthorized access to sensitive data.

Security violations can occur when unauthorized users gain access to sensitive data or when legitimate users perform unauthorized actions on the database. For example, a user might share their login credentials with someone else, or a hacker might exploit a vulnerability in the database software to gain access to sensitive data. To prevent security violations, access to the database should be controlled using strong authentication mechanisms, access control policies, and encryption of sensitive data.

Integrity violations can occur when data is modified or deleted in an unauthorized manner or when data accuracy is compromised due to software bugs or hardware failures. For example, a user might accidentally delete an important record from the database, or a software bug might cause data to become corrupted. To prevent integrity violations, access to the database should be controlled using access control policies, and data should be backed up regularly to ensure that it can be recovered in case of data loss or corruption.

To detect security and integrity violations, various techniques such as auditing, monitoring, and logging can be used. These techniques enable organizations to track and analyze [210]

activities performed on the database and detect any suspicious behavior or unusual activity that might indicate a security or integrity violation.

6.3 Access Control

The practice of regulating and limiting access to specific resources, such as data, information, physical areas, or digital systems, is referred to as access control. This is accomplished by establishing policies, processes, and technology that manage who is permitted access to particular resources, when they can access permitted, access to particular resources, when they can access, and what actions they are permitted to execute.

Access control can be implemented in a variety of ways, ranging from simple password-based systems to more complex, rigorous authentication and multi-factor authentication systems. Some common access control techniques include:

Discretionary Access Control (DAC)

A more flexible approach that allows users to determine who can access their resources. Typically, privilege granting and revocation are used to enforce discretionary access control in database systems. SQL supports discretionary access control through the GRANT and REVOKE commands. Granting a table or a view automatically grants all privileges on it, along with the right to view and modify it. In other words, the GRANT command grants, these privileges to other users. The syntax of this grant user's privileges in base tables and views. The syntax of this command is as follows:

```
GRANT privileges ON object TO users WITH GRANT OPTION;
```

For our purposes, object is either a base table or a view.

- **SELECT:** The right to access (read) all columns of the table specified as the object, including columns added later through ALTER TABLE commands.
- **INSERT (column name):** The right to insert rows into the table null or non-null values in the named column of the table cannot be object. If the right is not granted with respect to

all columns, including columns that might be added later, can simply use INSERT. The privileges UPDATE, DELETE, and INSERT are similar.

DELETE: The right to delete rows from the table or object.

REFERENCES (column-name) The right to define *foreign keys* (in other tables) that refer to the specified column of the table object. **REFERENCES** without a column name specifies the right with respect to all columns, including those that are added later.

If a user has a privilege with the grant option, he or she can pass it to another user (with or without the grant option) by using the GRANT command. A user who creates a view has practically the same privileges on the view that he or she has on every one of the views or base tables used to define the view. The user creating the view must have the SELECT privilege on each underlying table, if the view is to be queryable. The user creating the view must also have the SELECT privilege on each underlying table, if the view is to be updateable and so is always granted the SELECT privilege on the view. The creator of the view has the SELECT privilege with the grant option on every underlying table. In addition, if the view is updateable and the user holds INSERT, DELETE, or UPDATE privileges (with or without the grant option) on the (single) underlying table, the user automatically gets the same privileges on the view.

Only the owner of a schema can rewire the data definitions, segments CREATE, ALTER, and DROP on that schema. The right to execute these statements cannot be granted or revoked. Suppose that you want

Surgeon that year line has created the liability class, Rogers, and Sedus. Some examples of the CLART curriculum that line can now receive are listed below.

U24NT OPTIDN Receives 10 Years With

GRANT SELECT ON Sailors TO Michael
GRANT UPDATE (varies) ON Sailors TO
GRANT before

242 | Intelligent Database Management Systems

When a privilege is revoked from X, it is also revoked from all
 who got it solely from X. The revoke statement is used to
 revoke a privilege.

```
new <privilege list> on <relation name of new names> from
  <privilege list>
```

Example:

Example:
 ...at each branch from U1, U2, U3

Control (MAC)

A more rigid approach that is often used in high-security environments, such as military or government installations that have sensitive data and users based on security classes. This approach, called mandatory access control (MAC), would typically be used to control discretionary access control mechanisms.

Optical security classes are top secret (TS), secret (S), confidential (C), and unclassified (U), where TS is the highest level and U is the lowest. Other more complex security classification schemes exist, in which the security classes are organized in a lattice for simplicity. We will use the system with four security levels for simplicity, where $TS \geq S \geq C \geq U$ to differentiate our distinction levels, where TS is the highest security classification. The commonly used model for multilevel security, discussion as the Bell-LaPadula model, distinguishes each subject (user, program, principal, and object (database, data, column, view, etc.)) into one of the security classifications TS, S, C, or U. We will refer to the clearance (classification) of a subject S as $Sec(S)$ and refer to the clearance (classification) of an object O as $Sec(O)$.

Two restrictions are enforced on data access based on the subject's current classification:

1. **Simple Security Property:** A subject S is not allowed to read

*-Properly: A subject *S* is not allowed to write in object *O*

(CC'-properties)

If discretionary access controls are also specified, these rules represent additional restrictions. Thus, to read or write a database object, a user must have the necessary privileges (obtainable via

GRANT commands) and the security classes of the user and the object must satisfy the preceding restrictions.

6.4 Authorization and Authentication

6.4.1 Authorization

In a database, authorization is the process of approving or rejecting access to data depending on the rights and privileges given to a user or position. It is a crucial component of database security since it makes sure that only people with the proper permissions may access the information. Different forms of authorization on parts of the database are:

1. **Read Authorization**
Read authorization refers to the permission granted to a user or role to read or view data stored in a database. This permission allows the user to access and retrieve data from the database but does not allow them to modify or delete it.
2. **Insert Authorization**
Insert authorization refers to the permission granted to a user or role to insert new data into a database. This permission allows the user to add new records or data entries to the database but not modification of existing data.
3. **Update Authorization**
Update authorization refers to the permission granted to a user or role to modify existing data in a database. This permission allows the user to change or update the values of existing records or data entries in the database but not deletion of data.
4. **Delete Authorization**
Delete authorization refers to the permission granted to a user or role to remove existing data from a database. This permission allows the user to delete records or data entries from the database.

User can be given authorization on show without being given any authorization on the relations used in the view.

Definition: Ability of views to hide data serves both to simplify the user's view of the system and to enhance security by allowing users access only to data they need for their job. A combination of user-level security and view-level security can be used to restrict access to precisely the data that user needs.

6.4.2 Authentication

Authentication is the process of verifying the identity of a user before allowing access to a resource or service. It is an essential security measure used to protect sensitive information and ensure that only authorized individuals or entities can access it.

Authentication involves the use of credentials, such as usernames and passwords, security tokens, biometric data, or other authentication factors to verify the identity of the user or system. The authentication process can be performed by various methods, such as a login form, a security token, or a biometric scanner.

6.5 Security and Views

Views in a database can play a significant role in enhancing security. A view is a virtual table that is based on the result of a database query. It allows a user to see a specific subset of data without granting them access to the underlying tables or data.

Views can be used to implement security policies in a database by restricting access to sensitive data. For example, a view can be created that includes only the columns or rows that a user is authorized to see, while hiding the rest of the database. This can prevent unauthorized users from accessing sensitive data that they are not authorized to view.

Furthermore, views can be used to control the level of data access granted to different users or roles. Access can be restricted by creating views that limit the data that a user or role can see, while preventing them from accessing other data in the underlying tables. This approach can help prevent security breaches that result from unauthorized access to data.

Views can also be used to implement row-level security policies in a database. A view can be created that includes only rows of a table that a user is authorized to see, while hiding the rest of the rows. This approach is useful for implementing the granted access control policies that limit access to specific rows of data.

6.6 Encryption and Decryption

Despite being effective control measures, access and flow methods may not be able to completely safeguard databases from all dangers. Let's say we share data, but someone who shouldn't have access to it does. In this instance, by utilizing encryption, we may mask the message and prevent its disclosure even if the transmission is interrupted. Data is transformed during encryption into a ciphertext format that is difficult for unauthorized parties to understand. It enhances security and privacy when access controls are bypassed, because in cases of data loss or theft, encrypted data cannot be easily understood by unauthorized person.

The Data Encryption Standard (DES)

The American government created the DES system for use by the general people. It has received widespread acceptance in the United States as well as abroad as a cryptographic standard. On the communication channel between sender A and receiver B, DES can offer end-to-end encryption. The DES algorithm is a careful and complex combination of two of the fundamental building blocks of encryption: substitution and permutation (transposition). The algorithm's strength comes from repeatedly using these two methods over the course of 16 cycles. The message's original plaintext is encoded in blocks of 64 bits. Despite being 64 bits long, the key can actually be any 56-bit number.

The Advanced Encryption Standard (AES)

AES was introduced by the NIST after DES' suitability was questioned. In contrast to DES, which only supports keys with a size of 56 bits, this method can employ keys with a size of 128

bits, or 256 bits. AES introduces more key combinations than DES, which makes it more difficult to break.

Symmetric Key Algorithms

symmetric key is used for both encryption and decryption, this key is known as a symmetric key. For routine use with sensitive data in the database, quick encryption and decryption is achievable by applying a symmetric key. Only the same secret key will work to encrypt a message that has been encrypted using it. Secret-key algorithms are those used for symmetric key encryption. Secret-key algorithms are sometimes known as content encryption algorithms because they are frequently used to encrypt the contents of messages.

The necessity of sharing the secret key is the main drawback of secret-key algorithms. A password-based encryption technique is one approach that might be used to obtain the secret key from a user-supplied password string by applying the same function to the string at both the sender and receiver. The size of the key used affects how strong the symmetric key encryption is. For the same method, having a longer key makes encryption more difficult to crack than using a shorter key.

Public Key Encryption

In 1976, Diffie and Hellman proposed a cryptosystem, which they called public key encryption. Instead of relying on operations on bit patterns, public key algorithms use mathematical functions as their foundation. They address one issue with symmetric key encryption, namely the need for secure key exchange between sender and recipient. Two keys are utilized for encryption and decryption in public key systems. While the secret key is never communicated, the public key might be done so in an insecure manner. Asymmetric key encryption algorithms are those that carry out complementary operations (encryption and decryption) using two linked keys, a public key and a private key. Regarding key distribution, confidentiality, and authentication, the use of two keys can have significant repercussions. The two keys used for public key encryption are referred to as the public key and the private key. The private key is kept secret, but it is referred to as a

private key rather than a secret key (the key used in conventional encryption) to avoid confusion with conventional encryption, two keys are mathematically related, since one of the keys is used to perform encryption and the other to perform decryption. However, it is very difficult to derive the private key from the public key.

Six components make up a public key infrastructure or scheme:

1. Plaintext

This is the data or readable message that is fed into the algorithm as input.

2. Encryption Algorithm

This algorithm performs various transformations on the plaintext.

3. and 4. Public and Private Keys

There are a pair of keys that have been selected so that if one is used for encryption, the other is used for decryption. The exact transformations performed by the encryption algorithm depend on the public or private key that is provided as input. For example, if a message is encrypted using the public key, it can only be decrypted using the private key.

5. Ciphertext

This is the scramble message produced as output. It depends on the plaintext and the key. For a given message, two different keys will produce two different ciphertexts.

6. Decryption Algorithm

This algorithm accepts the ciphertext and the matching key and produces the original plaintext.

As the name suggests, the public key of the pair is made public for others to use, whereas the private key is known only to the owner. A general purpose public key encryption algorithm or one key scheme encryption and a different secret key for decryption. The essential steps are as follows:

1. X18 | Index or Indexes Message Encryption

Each user generates a pair of keys to be used for the encryption and decryption of messages.

1. Each user places one of the two keys in a public register or public accessible file. This is the public key. The companion key is kept private.
2. When a sender wishes to send a private message to a receiver, he or she encrypts the message using the receiver's public key.
3. The sender encrypts the message using the receiver's public key.
4. When the receiver receives the message, he or she decrypts it using the receiver's private key. No other recipient can decrypt the message because only the receiver knows his or her private key.

SOLUTION TO EXAMS' AND OTHER IMPORTANT QUESTIONS

1. Differentiate between authorization and authentication with brief examples.

[2021 Feb, 2017 Feb, 2014 Aug]

Ans Authentication is the process of verifying the identity of a user or system before allowing access to a resource or service. Authentication involves the use of credentials, such as usernames and passwords, security tokens, biometric data, or other authentication factors to verify the identity of the user or system.

Example:

When we log in to our email account, we are asked to provide our username and password, and the system verifies that the information we provided matches the stored credentials for your account.

Authorization, on the other hand, is the process of granting or denying access to a resource or service based on the authenticated user's permissions or privileges. Authorization determines what a user is allowed to access or perform after they have been authenticated. It involves setting up access controls, permissions, and policies to ensure that only authorized individuals or entities can access the resource or service.

Example:

After we log in to our email account, we may have authorization to read, compose, and send emails, but we may not have authorization to delete emails or change settings.

2. Explain the need of access control, Authorization and Authentication.

Ans Access control is necessary because not all users need the same level of access to a resource or system. For example, in an organization, employees may have different job roles and responsibilities, and therefore, they require different levels of access to company resources, such as files, folders, databases,

| 220 | Insights on Database Management Systems

and applications. Without the right access control procedures in place, unauthorized individuals could access confidential data, change or delete data, and seriously harm the firm.

Access control is essential for several reasons:

1. Protecting Sensitive Information

Access control ensures that sensitive information is only accessible to authorized users who have a legitimate need to know. This helps to prevent data breaches, theft, and other malicious activities.

ii. Maintaining Confidentiality

Access control mechanisms ensure that confidential information is kept confidential by preventing unauthorized users from accessing it.

iii. Enforcing Compliance

Access control mechanisms can help organizations comply with regulatory requirements by ensuring that only authorized users have access to sensitive data.

iv. Preventing Insider Threats

Access control mechanisms can help prevent insider threats by limiting the access of employees to only the resources they need to do their job.

v. Enhancing System Availability

Access control mechanisms can help prevent denial-of-service attacks by limiting access to system resources.

Authentication refers to the process of verifying the identity of a user, device, or application. It involves the use of various authentication methods, such as passwords, biometrics, smart cards, and tokens, to ensure that only authorized users are granted access to sensitive information and resources. Authentication is necessary because it ensures that only legitimate users can access a system, and unauthorized users are denied access.

Authorization refers to the process of determining what actions an authenticated user is allowed to perform. It involves granting or denying access to resources based on the

user's identity, privileges, and permissions. Authorization is necessary because not all users should have the same level of access to a resource or system, and unauthorized access can lead to data breaches, theft, and other malicious activities.

3. List all security issues you are familiar that can breach security. Encryption and Decryption are very important part in cryptography so justify this statement with suitable example.

Ans Security issues that can lead to a breach of security are:

I. Weak Passwords

Weak passwords or password reuse across multiple accounts can make it easy for attackers to gain unauthorized access.

II. Social Engineering

Social engineering involves tricking people into disclosing sensitive information or performing an action that can lead to a security breach.

III. Phishing

Phishing is a type of social engineering where attackers send fraudulent emails or messages to trick users into providing sensitive information.

IV. Malware

Malware, such as viruses, Trojans, and spyware, can infect a system and steal sensitive information or cause damage.

V. Denial-of-Service Attacks

Denial-of-service attacks involve overwhelming a system with traffic or requests to make it unavailable to legitimate users.

VI. Insider Threats

Insider threats involve employees or other trusted individuals who misuse their access to sensitive information or systems.

vii. Physical Security Breaches

Physical security breaches involve unauthorized access to physical assets, such as buildings, servers, or data centers.

viii. Unpatched Software Vulnerabilities

Unpatched software vulnerabilities can be exploited by attackers to gain unauthorized access to systems or steal sensitive information.

ix. Third-Party Risks

Third-party risks involve vulnerabilities in software or services provided by third-party vendors or partners.

x. Internet of Things (IoT) Devices

IoT devices can be vulnerable to attacks that compromise the security of a network or system.

The process of converting plaintext into ciphertext, a form of data that is unreadable and unintelligible to anyone who does not have the key to decrypt it, is known as encryption. Decryption is the reverse process of converting ciphertext back into plaintext.

For modern communication systems to guarantee confidentiality and privacy, encryption and decryption are necessary. Without encryption, unauthorized third parties might intercept and access sensitive information including credit card numbers, login credentials, and medical records. For example, let's say we want to send a sensitive email message to a friend. We could use encryption to protect the contents of the message from being intercepted and read by anyone who does not have the key to decrypt it.

Here's how the process might work:

- i. We use an encryption program to encrypt the plaintext message using a key.
- ii. The encryption program transforms the plaintext message into ciphertext, which is unreadable and unintelligible without the key.

iii. We send the ciphertext message to our friend via email or another communication channel.

iv. Our friend receives the ciphertext message and uses a decryption program to decrypt the message using the key.

v. The decryption program transforms the ciphertext message back into plaintext, which our friend can now read.

Without encryption, the plaintext message would be vulnerable to interception and reading by unauthorized third parties, which could compromise the confidentiality of the message and lead to various security issues. However, by using encryption and decryption, the message remains secure and private, even if it is intercepted by an attacker.

4. Why ACL technique is considered safe way for database security? How is any user allowed or prevented from accessing a certain resource?

[20/7/11]

Ans Due to its capacity to offer smooth control over resource access, the ACL (Access Control List) technique is regarded as a secure method for database security. Administrators can set and maintain access permissions for specific people or groups using ACLs, ensuring that only authorized parties can access a database's resources. Here is a more thorough description of how ACLs are used to permit or restrict user access to particular resources:

I. Resource Access Control

Every database resource, including tables, views, stored procedures, and columns, is accompanied by an access control list (ACL) that contains a list of access control entries (ACEs). The privileges or permissions assigned to specific users or groups for that particular resource are described in ACEs.

II. User Authentication

Users must identify themselves by supplying legitimate credentials, such as usernames and passwords, before they may access the database. By confirming their

identity, user authentication makes sure that only authorized users can continue.

III. ACL Evaluation

The corresponding ACL is consulted whenever a user tries to access a particular resource. If there is a match, the system checks the user's credentials against the ACEs in the ACL.

IV. Permission Verification

The system checks to see if the user has the required permissions to carry out the requested operation on the resource if the user's credentials match an ACE in the ACL. Read, write, update, delete, execute, and other special operations could be included in the permissions.

V. Access Granting

Access to the resource is granted if the user's credentials and the intended action comply with the permissions outlined in the ACE. The requested operation—such as retrieving data, editing records, or running queries—can be carried out by the user.

VI. Access Denial

Access to the resource is prohibited if the user's credentials do not match any ACE in the ACL, or if the intended action exceeds the permitted scope. The user is unable to use the resource or carry out the unlawful behavior.

VII. Access Control Administration

The management and upkeep of the ACLs is the duty of the database administrators or other authorized individuals. To make sure that access permissions appropriately represent the security policies and requirements of the organization, they create, alter, or remove ACEs as necessary. Database systems achieve a strong security foundation by using ACLs. Administrators can accurately define which individuals or groups have access to particular resources

and what actions they can take thanks to ACLs, which offer a granular level of control. The possibility of unauthorized access, data breaches, or harmful actions within the database is reduced thanks to this fine-grained access control. It lets companies to implement the principle of least privilege, limiting user access to sensitive information or crucial processes while providing users only the rights necessary to carry out their responsibilities.

3. What are the needs of cryptography?

ANS

Prat Singh

Using a format that is incomprehensible to unauthorized parties, cryptography is the process of safeguarding (scrambled) and then decrypted (unscrambled) using mathematical techniques and algorithms. Data confidentiality, integrity, authentication, and non-repudiation are all goals of cryptography.

Cryptography is needed for various reasons, including:

- I. **Confidentiality:** By encrypting data, it makes sure that private information stays that way and is inaccessible to unauthorized people.
- II. **Integrity:** Cryptography ensures the integrity and reliability of data by confirming that it was not changed or tampered with during transmission or storage.
- III. **Authentication:** It aids in establishing the legitimacy of people, systems, or equipment in order to avoid impersonation and unwanted access.
- IV. **Non-Repudiation:** Through the use of cryptography, parties can demonstrate their participation in a transaction or the validity of their digital signatures, preventing action denial.
- V. **Key Management:** Secure cryptographic key management is necessary for encryption, decryption, and other processes.

- vi. **Secure Communication:** For communication over potentially unsafe networks, cryptography offers secure channels that guard against interceptions or eavesdropping.
- vii. **Data Privacy and Protection:** It helps guard against unauthorized access to and exposure of private and sensitive information.
- viii. **Compliance and Regulations:** To ensure data security and privacy, cryptography is frequently required to meet regulatory and compliance requirements.



QUERY PROCESSING

CHAPTER 7

7.1 Introduction to Query Processing

Query processing is a translation of high-level queries into low level expressions. It refers to the range of activities that are involved in extracting data from the database which includes translation of queries in high level database languages into expression that can be implemented at the physical level of the system. Query processing is a stepwise process that can be used at the physical level of the file system, query optimization and actual execution of query to get the result and requires the basic concepts of relational algebra and file structure.

Basic Steps in Query Processing:

1. Parsing and translation
2. Optimization
3. Evaluation

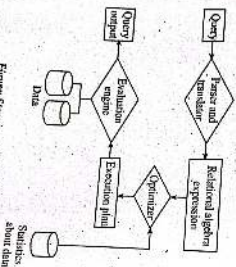


Figure: Steps in query processing

Query Parsing and Translation

1. In this phase a query is translated into SQL and into relational algebraic expression. The parser of the query processor module checks the syntax of the query, the user's privileges to execute the query, the table names and attribute names, etc. The correct table names, attribute names and the privilege of the users can be taken from the data dictionary. If the query is valid, then it is converted from high level language SQL to low level instruction in relational algebra which is represented as a query tree data structure. Different possible relational algebra expression may exist for a single query.

Example:

SELECT Ename FROM Employee WHERE Salary > 5000;

The query is then translated into Relational Algebra

Expression as:

$\sigma_{\text{salary} > 5000}(\Pi_{\text{Ename}}(\text{Employee}))$

OR

$\Pi_{\text{Ename}}(\sigma_{\text{salary} > 5000}(\text{Employee}))$

2. Query Optimization

Query optimization is the process of selecting the most efficient query evaluation plan among the many strategies usually possible for processing a given query specially if the query is complex. A relational algebra operation annotated with instructions on how to evaluate it is called an evaluation primitive. A sequence of primitive operations that can be used to evaluate a query is a query execution plan or query evaluation.



3. Query Evaluation

The query execution engine takes a query, evaluates, then specifies the join and returns the answers to the query. Specify which access path to follow, which algorithm to use to evaluate operators and how operators behave.

Output

The final result of the query is shown to the user.

7.2 Equivalence of Expressions

A query can be expressed in several different ways with different cost of evaluation. Rather than talking the relational expression as given, we consider alternative expressions.

Two relational algebra expressions are said to be equivalent if on every legal database instance, the two expressions generate the same set of tuples. In SQL, inputs and outputs are generated tuples, two expressions in the multiset version of the relational algebra are said to be equivalent if on every legal database instance two expressions generate the same number of tuples.

Equivalence Rules

An equivalence rule says that expressions of two forms are equivalent. By applying transformation rules, the expression can transform one relational algebra expression into an equivalent expression that is known to be more efficient. Following rules can be applied.

1. Conjunctive selection operations can be decomposed into a sequence of individual selections.
2. Selection operations are commutative.
3. Only the last in a sequence of projection operations is needed; the others can be omitted.

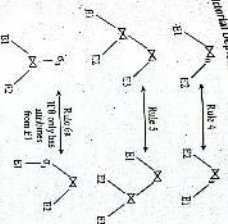
$$\pi_{A_1, A_2, \dots, A_n}(\pi_{B_1, B_2, \dots, B_m}(\pi_{C_1, C_2, \dots, C_p} R)) = \pi_{A_1, A_2, \dots, A_n} R$$

selections can be combined with cartesian products and then

$$\sigma_A(\sigma_B(X, Y)) = \sigma_{A \wedge B}(X, Y)$$

$$\sigma_A(\sigma_B(\sigma_C(E_1))) = \sigma_{A \wedge B \wedge C}(E_1)$$

potential Depiction of Equivalence Rules



1. Projection operations (and natural joins) are commutative

$$\text{Given } E_1 = E_2 \wedge E_3$$

2. Natural join operation is associative

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

3. These joins are associative in the following manner:

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

where E_1 involves attributes from only E_1 and E_2 .

4. The selection operation distributes over the join operation under the following two conditions:

a. When σ_A involves only the attributes of one of the operands (E_1), being joined.

$$\sigma_A(E_1 \bowtie E_2) = (\sigma_A(E_1)) \bowtie E_2$$

b. When σ_A involves only the attributes of E_1 and E_2 involves only the attributes of E_2 .

$$\sigma_A \wedge B(E_1 \bowtie E_2) = (\sigma_A(E_1) \bowtie \sigma_B(E_2))$$

8. The projections operation distributes over the theta join operation as follows:

a. If P involves only attributes from $L_1 \cup L_2$

$$\Pi_{L_1, L_2}(E_1 \bowtie E_2) = (\Pi_{L_1}(E_1)) \bowtie (\Pi_{L_2}(E_2))$$

b. Consider a join $E_1 \bowtie E_2$.

> Let L_1 and L_2 be sets of attributes from E_1 and E_2 , respectively.

> Let L_2 be attributes of E_1 that are involved in join condition θ , but are not in $L_1 \cup L_2$ and

> Let L_1 be attributes of E_2 that are involved in join condition θ , but are not in $L_1 \cup L_2$.

$$\Pi_{L_1, L_2}(E_1 \bowtie E_2) = \Pi_{L_1, L_2}((\Pi_{L_1 \cup L_2}(E_1)) \bowtie (\Pi_{L_1 \cup L_2}(E_2)))$$

9. The set operations union and intersection are commutative
 $E_1 \cup E_2 = E_2 \cup E_1$
 $E_1 \cap E_2 = E_2 \cap E_1$

(set difference is not commutative).

10. Set union and intersection are associative.

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$

$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

11. The selection operation distributes over $(\cup, \cap)$ and $(\sigma(E_1 - E_2) = \sigma(E_1) - \sigma(E_2))$ and similarly for $\cup$ and $\cap$ in place of $-$.

Also: $\sigma(E_1 - E_2) = \sigma(E_1) - E_2$ and similarly for $\cap$ in place of $-$, but not for $\cup$

12. The projection operation distributes over union

$$\Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$

Transformation Example:

Query: Find the names of all customers who have an account at some branch located in Brooklyn.

$\Pi_{\text{customer-name}}(\text{branch-city} \bowtie \text{branch} \bowtie \text{account} \bowtie \text{depositor}))$

Transformation using rule 7a.

$\Pi_{\text{customer-name}}((\text{branch-city} \bowtie \text{branch})) \bowtie (\text{account} \bowtie \text{depositor}))$
 performing the selection as early as possible reduces the size of the relation to be joined.

Example with Multiple Transformation

Query: Find the names of all customers with an account at a branch whose account balance is over \$1000.

$\Pi_{\text{customer-name}}(\text{branch-city} \bowtie \text{branch} \bowtie \text{account} \bowtie \text{depositor}))$
 (account > 1000; branch > 100; branch > 1000)

Transformation using join associativity (Rule 6a):

$\Pi_{\text{customer-name}}(\text{branch-city} \bowtie \text{branch} \bowtie \text{account} \bowtie \text{depositor}))$
 $\bowtie (\text{account}) \bowtie (\text{branch})$

Second form provides an opportunity to apply the "perform selections early" rule, resulting in the subexpression

$\text{branch-city} \bowtie \text{branch} \bowtie (\text{branch} \bowtie \text{account})$

Thus a sequence of transformations can be useful.

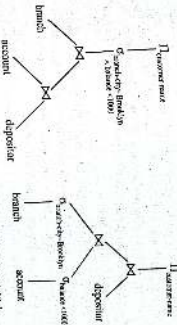


Figure 8: Initial Expression Tree

Figure 9: Tree After Multiple Transformations

When we compute $\Pi_{\text{customer-name}}(\text{branch-city} \bowtie \text{branch} \bowtie \text{account} \bowtie \text{depositor}))$ we obtain a relation whose schema is

(branch-name, branch-city, assets, account-number, balance)

Push projections using equivalence rules 8a and 8b, eliminate unneeded attributes from intermediate results to get

$\Pi_{\text{customer-name}}(\Pi_{\text{account}}(\text{branch-city} \bowtie \text{branch} \bowtie \text{account} \bowtie \text{depositor})) \bowtie \text{depositor}))$

Cost of (b) (2) block transfers plus one seek
 $b_2 + (b_1/2) + 1$

Since at most one record satisfies condition, scan can be terminated as soon as the required record is found. In the worst case, if blocks transfers are still required, linear search can be applied regardless of

- Selection condition or
- Ordering of records in the file, or
- Availability of indices

A2) Clustering B+-tree index, Equality on Key)

For an equality comparison on a key attribute with a clustering index, we can use the index to retrieve a single record that satisfies the corresponding equality condition.

Case

$$(b_1/2) + (1/2) b_2$$

Where b_1 is the height of the tree in 'B' level
 $(b_1/2)$ is given because it seeks in reverse the B+ tree of height b_1 and we need one seek to reverse the call a file where the record is stored.

A3) Clustering B+-tree index, Equality on Non-key)

We can retrieve multiple records by using a clustering index when the selection condition specifies an equality term, the previous case is a non-key attribute, A. The only difference is the method. However, the records must be stored consecutively in the file since the file is ordered by the search key.

Cost of the query becomes

$$b_1 + (1/2) b_2 + (1/2) b_2 + 1$$

One seek for each level of the tree, one seek for the B+ tree there b_1 is the number of blocks containing records with the searched search key, all of which are read. These blocks are then blocks assumed to be stored sequentially hence it is a clustering index) and then it requires additional seek.

136 | Page on Database Management Systems

4) Secondary B+-tree Index, Equality on Key)

Selections specifying an equality condition can use a secondary index. This strategy can require a single record if the equality condition is on a key; multiple records may be retrieved if the indexing field is not a key.

In the first case, only one record is retrieved. The time cost in this case is the same as that for a clustering index (see A2).

$$(b_1/2) + (1/2) b_2$$

Where b_1 is the height of the tree in 'B' level

In the second case, each record may be retrieved on a different block, which may result in one I/O operation per retrieved record, with each I/O operation requiring a seek and a block transfer.

The worst-case time cost in this case is

$$(b_1/2) + (1/2) b_2$$

where n is the number of records fetched

5) Selections Involving Comparison

We can implement selection of the form $A \geq B$ or $A < B$ by using a linear file scan or by using indices in the following ways:

A1) Clustering B+-tree index, Comparison)

A clustering ordered index (for example, a clustering B+-tree index) can be used when the selection condition is a comparison.

For comparison conditions of the form $A \geq V$ or $A \leq V$ a clustering index on A can be used to direct the retrieval of tuples, as follows: For $A \geq V$, we scan up the index V of A.X, index to find the first tuple in the file that is a value of A.X greater than or equal to V . The tuples starting from that tuple up to the end of the file return all tuples that satisfy the condition. For $A \leq V$, the first tuple that is greater than or equal to V is the first tuple that does not satisfy the condition.

For comparison conditions of the form $A < V$ or $A > V$, the index is used to find the first tuple that is greater than or equal to V . For comparison conditions of the form $A < V$ or $A > V$, the index is used to find the first tuple that is greater than or equal to V . For comparison conditions of the form $A < V$ or $A > V$, the index is used to find the first tuple that is greater than or equal to V .

Cost of Querying | 137 |

from the beginning of the file, and continuing up to $\text{floor}(v)$ including the first tuple with attribute $A = v$. The case of $A > v$ is similar, except that the scan continues up to $\text{floor}(v)$ including the first tuple with attribute $A > v$. In either case, the index is not useful.

Cost:

$$h_1 * (r_1 + c_1) + c_2 + b * c_2$$

Identical to the case of A3, equality on non-key.

A6(Secondary B+-tree Index, Comparison)

We can use a secondary ordered index to guide retrieval by comparison conditions involving $<, \leq, =$, or $>$. The lowest-level index blocks are scanned, either from the smallest value up by $\text{floor}(v)$ and 5), or from v up to the maximum value $\text{floor}(v) + 5$. The secondary index provides pointers to the records, but to get the actual records we have to fetch the records by using the pointers. This step may require an I/O operation for each record fetched, since consecutive records may be on different disk blocks; as before, each I/O operation requires a disk seek and a block transfer. If the number of retrieved records is large, using the secondary index may be even more expensive than using linear search. Therefore, the secondary index should be used only if very few records are selected.

Cost:

$$(h_1 + n) * (r_1 + c_1)$$

Identical to the case of A4, equality on non-key.

Implementation of Complex Selections

So far, we have considered only simple selection conditions of the form $A \text{ op } B$, where op is an equality or comparison operation. We now consider more complex selection predicates.

Conjunction

A conjunctive selection is a selection of the form:

$$\sigma_{A_1 \text{ op } 1 \wedge A_2 \text{ op } 2 \wedge \dots \wedge A_n \text{ op } n}(r)$$

Disjunction

A disjunctive selection is a selection of the form:

$$\sigma_{A_1 \text{ op } 1 \vee A_2 \text{ op } 2 \vee \dots \vee A_n \text{ op } n}(r)$$

A disjunctive condition is satisfied by the union of all records satisfying the individual, simple conditions B_i .

Negation

The result of a selection $\sigma_{\neg \theta}(r)$ is the set of tuples of r for which the condition θ evaluates to false. In the absence of nulls, this set is simply the set of tuples in r that are not in $\sigma_{\theta}(r)$.

We can implement a selection operation involving either a conjunction or a disjunction of simple conditions by using one of the following algorithms

A7 (Conjunctive Selection using One Index)

We first determine whether an access path is available for an attribute in one of the simple conditions. If one is, one of the selection algorithms A2 through A6 can retrieve records satisfying that condition.

To reduce the cost we choose a B_i and one of algorithms A1 through A6 for which the combination results in the least cost for $\sigma_{B_i}(r)$. The cost of algorithm A7 is given by the cost of the chosen algorithm.

A8 (Conjunctive Selection using Composite Index)

An appropriate composite index (that is, an index on multiple attributes) may be available for some conjunctive selection. If the selection specifies an equality condition on two or more attributes, and a composite index exists on these combined attribute fields, then the index can be searched directly. The type of index determines which of algorithms A2, A3, or A4 will be used.

A9 (Conjunctive Selection by Intersection of Identifiers)

Conjunctive selection operations involve the use of record pointers or record identifiers. This algorithm requires indices with record pointers, on the fields involved in the individual conditions. The algorithm scans each index for pointers to

tuples that satisfy an individual condition. The intersection of all the retrieved pointers is the set of pointers to tuples that satisfy the conjunctive condition. The algorithm to traverse the pointers to retrieve the actual records. If indices are available on all the individual conditions, then the algorithm tests the retrieved records against the remaining conditions. The cost of algorithm A9 is the sum of the costs of the individual index scans, plus the cost of retrieving the records in the intersection of the retrieved lists of pointers. This cost can be reduced by sorting the list of pointers and retrieving records in a block come together, hence all pointers in the block can be retrieved using a single I/O operation, and (2) blocks are read in sorted order, minimizing disk scan movement.

A10 Disjunctive Selection by Union of Identifiers

If access paths are available on all the conditions of a disjunctive selection, each index is scanned for pointers to tuples that satisfy the individual condition. The union of all the retrieved pointers yields the set of pointers to all tuples that satisfy the disjunctive condition. We then use the pointers to retrieve the actual records.

However, if even one of the conditions does not have an access path, we have to perform a linear scan of the relation to find tuples that satisfy the condition. Therefore, if there is even one such condition in the disjunct, the most efficient access method is a scan, with the disjunctive condition tested on each tuple during the scan.

7.3.2 Sorting

We can sort a relation by building an index on the sort key, and then using that index to read the relation in sorted order. However, such a process orders the relation in sorted order, through an index, rather than physically. Hence, the reading of tuples in the sorted order may lead to a disk access (disk seek plus block transfer) for each record, which can be very expensive, since the number of records can be much larger than the number of blocks. For this reason, it may be desirable to order the records physically.

For relations that fit in memory that is records are completely in main memory, techniques like quick sort can be used for relations that don't fit in memory, which cumulatively sorts a good choice. External sort-merge, which cumulatively sorts multiple runs of the data based on amount that fits in memory at one time.

External Sort-Merge Algorithm

Sorting of relations that do not fit in memory is called external sorting. The most commonly used technique for external sorting is the external sort-merge algorithm.

Let M denote the number of blocks in the main-memory buffer available for sorting, that is, the number of disk blocks whose contents can be buffered in available main memory.

1. In the first stage, a number of sorted runs are created, each run is sorted, but contains only some of the records of the relation.

$i = 0$;

repeat

 read M blocks of the relation, or the rest of the relation,

 whichever is smaller;

 sort the in-memory part of the relation;

 write the sorted data to run file R_i ;

$i = i + 1$;

until the end of the relation

2. In the second stage, the runs are merged. Suppose, for now, that the total number of runs, N , is less than M , so that we can allocate one block to each run and have space left to hold one block of output. The merge stage operates as follows:

 read one block of each of the N files R_i into a buffer block in memory;

 repeat

 choose the first tuple (in sort order) among all buffer blocks;

 write the tuple to the output, and delete it from the buffer block;

relation s the inner relation of the join, since the loop for s encloses the loop for t . The algorithm uses the notation t_i , where t_i and t_i are tuples $t_1, \dots, t_n$ denotes the tuple consisting of concatenating the attribute values of tuples t_1 and t_2 for each tuple t_i in s do begin

test pair (t_i, t_i) to see if they satisfy the join condition

if they do, add t_i, t_i to the result

end

The nested-loop join algorithm is expensive, since it examines every pair of tuples in the two relations. The number of pairs of tuples to be considered is $t_1 \times t_2$, where t_1 denotes the number of tuples in r , and t_2 denotes the number of tuples in s .

For each record in r , we have to perform a complete scan on s . In the worst case, the buffer can hold only one block of s . Required, where b_r and b_s denote the number of blocks containing tuples of r and s , respectively. We need only one slot for each scan on the inner relation s since it is first, then $t_1, \dots, t_n$ and t_i are added to r , leading to a total of $b_r + b_s$ seeks.

In the best case, there is enough space for each relation to fit simultaneously in memory, so each block would have to be read only once; hence, only $b_r + b_s$ block transfers would be required, along with 2 seeks.

Example:

Consider the natural join of student and takes. Assume for now that we have no buffers whatsoever in either relation, the second loop to compare the join assume that student is the outer relation and takes is the inner relation in the join. We will have to examine $50001 \times 10,000 = 50 \times 10^8$ pairs of tuples. In the worst case, the number of block transfers is $5000 \times 400 \times 100 = 2,000,000$, plus $5000 \times 400 = 2,000,000$ seeks.

244 | **Understanding Database Management Systems** | 245 |

best-case scenario, however, we can read both relations only once, and perform the comparison. This computation requires at most $100 \times 400 = 500$ block transfers, plus 2 seeks. A significant improvement over the worst-case scenario. If we had used blocks as the relation for the outer loop and student for the inner loop, the worst-case cost of our final strategy would have been $10,000 \times 100 \times 400 = 1,000,000$ block transfers, plus $10,000$ disk seeks. The number of block transfers is significantly less, and although the number of seeks is higher, the overall cost is reduced, assuming $1 \mu s = 1$ microsecond and $1 ms = 0.1$ milliseconds.

Block Nested-Loop Join

If the buffer is too small to hold either relation entirely in memory, we can still obtain a major saving in block accesses if we process the relations on a per-block basis, rather than on a per-tuple basis. Block nested-loop join is a variant of the nested-loop join, where every block of the inner relation is paired with every block of the outer relation. Within each pair of blocks, every tuple in one block is paired with every tuple in the other block, to generate all pairs of tuples. As before, all pairs of tuples that satisfy the join condition are added to the result.

for each block b_i of r do begin

for each block b_j of s do begin

for each tuple t_i in b_i do begin

test pair (t_i, t_j) to see if they satisfy

the join condition

if they do, add t_i, t_j to the result

end

end

end

end

end

244 | **Understanding Database Management Systems** | 245 |

In the worst case, each block in the inner relation s is tried only once for each block in the outer relation. Thus, in the worst case, there will be a total of $b \times b \times b$ block transfers when b and b denote the number of blocks containing records b and b , respectively. Each scan of the inner relation requires d per block, leading to a total of $2 \times b$ seeks. It is more efficient to use the smaller relation as the outer relation, in case neither of the relations fits in memory.

In the best case, where the inner relation fits in memory, there will be $b \times b$ block transfers and, just two seeks (we would choose the smaller relation as the inner relation in this case).

Now return to our example of computing student m takes, using the block nested-loop join algorithm. In the worst case, we have to read each block of takes once for each block of student.

Thus in the worst case, a total of $300 \times 400 \times 100 = 40,100$ block transfers plus $2 \times 100 = 200$ seeks are required.

This cost is a significant improvement over the $5000 \times 400 \times 100 = 2,000,000$ block transfers plus 5,100 seeks needed in the worst case for the basic nested-loop join.

The best case cost remains the same—namely, $100 \times 400 = 500$ block transfers and two seeks.

3. Indexed Nested-Loop Join

In a nested-loop join, if an index is available on the inner loop's join attribute, inner joinings can replace the scans. For each tuple t_i in the outer relation, the index is used to look up tuples in s that will satisfy the join condition with tuple t_i . This join method is called an indexed nested-loop join. It can be used with existing indices, as well as with temporary indices created for the sole purpose of evaluating the join.

Looking up tuples in s that will satisfy the join conditions with a given tuple t_i is essentially a selection on s . For example, consider student m takes. Suppose that we have a

[246] Index in Database Management Systems

primary index with ID "00128". Then the relevant tuples in s are those that satisfy the selection " $ID = 00128$ ". The use of an indexed nested-loop join can be compared as cost of an index tuple in the outer relation r , a lookup is performed on the index for s , and the relevant tuples are retrieved.

In the worst case, there is space in the buffer for only one block of r and one block of the index. Then, b I/O operations are needed to read relation r , where b denotes the number of blocks containing records of r ; each I/O requires a seek and a block transfer, since the disk head may have moved in between each I/O. For each tuple in r , we perform an index lookup on s .

Then, the cost of the join can be computed as: $b(r \times b) \times n_s \times c$, where n_r is the number of records in r , $b(r \times b)$ is the cost of a single selection on s using the relation r , and c is the cost of a single selection on s using the relation r . We have seen how to estimate the cost of a join condition. We have seen how to estimate the cost of a single selection algorithm (possibly using indices); this estimate gives us the value of c . The next formula indicates that, if indices are available on both relations r and s , it is generally most efficient to use the one with fewer tuples as the inner relation.

For example, consider an indexed nested-loop join of student m takes, with student as the outer relation. Suppose also that takes has a covering B+-tree index on the join attribute ID, which contains 20 entries on average in each index node.

Such index has:

10,000 tuples, the height of the tree is 4, and one entry across the entire index has 2500 tuples. Since n_r is 5000, the total cost is $100 \times 5000 \times 5 = 25,000$ disk accesses, each of which requires a seek and a block transfer. In contrast, a worst-case nested-loop join of 40,100 block transfers plus 200 seeks were needed for a block nested-loop join, although the number of block transfers has been reduced, the seek cost has actually increased. Increasing the total cost, since a seek is considerably more expensive than a block transfer reduces the

If we had a selection on the student relation that reduces the

Cost: 10,000 [247]

number of rows significantly, indexed nested-loop join could be significantly faster than block nested-loop join.

4. Merge Join

Assuming that b_b buffer blocks are allocated to each relation, the number of disk seeks required would be $\lfloor b_a/b_b \rfloor + \lfloor b_b/b_b \rfloor$, much more expensive than data transfer. It makes sense to allocate multiple buffer blocks to each relation, provided extra memory is available.

For example, with $b_r = 0.1$ milliseconds per 4-Kilobyte block, and $b_b = 4$ milliseconds, the buffer size is 400 blocks (or 16 megabytes), so the seek time would be 4 milliseconds for 16 time would be just 10 percent of the transfer time.

Suppose the merge-join scheme is applied to our example of student $\bowtie$ takes. The join attribute here is ID. Suppose that the relations are already sorted on the join attribute ID. In this case, the merge-join takes a total of $400+100 = 500$ block transfers. If we assume that in the worst case only one buffer block is allocated to each input relation (that is, $b_b = 1$), a total of $400 + 100 = 500$ seeks would also be required; in reality, it can be set much higher since we need to buffer blocks for only two relations, and the seek cost would be significantly less.

5. Hash Join

Like the merge-join algorithm, the hash-join algorithm can be used to implement natural joins and equi-joins. In the hash-join algorithm, a hash function h is used to partition tuples of both relations. The basic idea is to partition the tuples of the relations into sets that have the same hash values of join attributes. We assume that:

- h is a hash function mapping join attributes values to $\{0, 1, \dots, m\}$, where join attributes denotes the common attributes of r and s used in the natural join.

- $r_0, r_1, \dots, r_m, n_h$ denote partitions of r tuples, each initially empty. Each tuple $tr \in r$ is put in partition r_i , where $i = h(\text{joinAttr}(tr))$.
- $s_0, s_1, \dots, s_m, n_s$ denote partitions of s tuples, each initially empty. Each tuple $ts \in s$ is put in partition s_i , where $i = h(\text{joinAttr}(ts))$.
- Thus, a hash join is estimated to require:
 $3(b_r + b_s) + 4n_h$ block transfers.
 $2((b_r/b_b) + (b_s/b_b)) + 2n_h$ seeks.

7.4 Query Optimization

Query optimization is the process of selecting the most efficient query evaluation plan from among many strategies usually possible for processing a given query, especially query is complex. We do not expect users to write their queries so that they can be processed efficiently. Rather we expect the system to construct a query evaluation plan that minimizes the cost of query evaluation. This is where query optimization comes into play. An important aspect of query processing is query optimization. The aim of query optimization is to choose the one that minimizes resource usage.

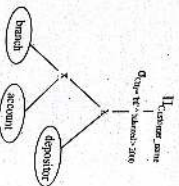
Aim of query optimization is to transform query into faster, equivalent query. Every method of query optimization depends on database statistics. The statistics cover information about relations, attribute, and indexes. Keeping the statistics current can be problematic. If the DBMS updates the statistics every time a tuple is inserted, updated, or deleted, this would have a significant impact on performance during peak period. An alternative approach is to update the statistics on a periodic basis, for example nightly, or whenever the system is idle. After a query has been scanned, parsed, and validated, the DBMS must choose from a set of possible execution strategies for the query, and this is known as query optimization. A relational algebra expression may have many equivalent expressions.

SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS

1. Make an operator tree for the following SQL expression
SELECT customer_name
FROM branch, account, depositor
WHERE branch_city = 'bri' AND balance > 100

$\Pi_{customer_name}$ ($\sigma_{(branch_city = 'bri' \wedge balance > 100)} (branch \bowtie account \bowtie depositor)$)

Equivalent relational algebra for given SQL expression is
 $\Pi_{customer_name} (\sigma_{(branch_city = 'bri' \wedge balance > 100)} (branch \bowtie account \bowtie depositor))$



2. Suppose we have the following relation: Employee
 (person_name, salary, city)
 Write the relational algebra expression for the query
 Find the names of all employees who live in Pothahar.
 Construct the initial operator tree and final efficient operator tree after applying transformation rules. [100 marks]

$\Pi_{person_name} (\sigma_{(city = 'Pothahar')} (Employee))$
 Here is the initial operator tree representation.

254 | Inquiry on Database Management Systems

Π_{person_name}
 $\sigma_{city = 'Pothahar'}$

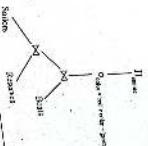
Employee

The initial expression, $\Pi_{person_name} (\sigma_{(city = 'Pothahar')} (Employee))$, directly applies the selection operation on the Employee relation to filter the employees who live in Pothahar and then projects the person_name attribute. This expression represents an efficient way to retrieve the names of employees who live in Pothahar. There is no need for further transformation or optimization in this case.

3. Considering the following schemas:
 Sailors (sid, sname, rating, age)
 Boats (bid, bname, color)
 Reserves (rid, bid, day)

Write the relational algebra expression for the query
 While the relational algebra expression for the query
 Find the names of sailors who have reserved a red or green boat. Construct the initial operator tree and final efficient operator tree after applying transformation rules. [100 marks]

To find the names of sailors who have reserved a red or green boat, we can use the following relational algebra expression:
 $\Pi_{sname} (\sigma_{(color = 'red' \vee color = 'green')} (Sailors \bowtie Reserves \bowtie Boats))$



Query Processing | 255 |

CHAPTER - 8

FILE ORGANIZATION AND INDEXING

8.1. Disks and Storage

Several types of data storage exist in most computer systems. These storage media are classified by the speed with which data can be accessed by the cost per unit of data to buy the medium and by the medium's reliability. Among the media typically available are these:

- i. Cache
- ii. Main Memory
- iii. Magnetic Disk Storage
- iv. Optical Storage
- v. Tape Storage

i. Cache

The cache is the fastest and most costly form of storage. Cache memory is small, its use is managed by the computer system hardware.

ii. Main Memory

The storage medium used for data that are available to be operated on by main memory. The general-purpose machine instructions operate on main memory. Although main memory may contain many megabytes of data or even gigabytes of data in large server systems, it is generally small (or too expensive) for storing the entire database. The contents of main memory are usually lost if power failure or system crash occurs; main memory is therefore said to be volatile.

iii. Flash Memory

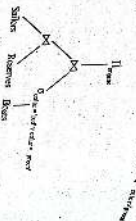
In flash memory data survives the power failure that is a serious drawback. Flash memory has a lower cost per byte than non-volatile, but a higher cost per byte than magnetic disks. Flash memory is widely used for data storage in devices such as cameras and cell phones. Flash memory is also used for storing data in "flash drives" also known as "pen drives".

When all the attributes in the involve only the attributes of the of the expressions(E1) being joined then following transformation is possible:

$$\sigma_{(E_1 \wedge E_2)}(R_1 \bowtie R_2) = (\sigma_{(E_1)}(R_1)) \bowtie R_2$$

Hence above query becomes:

$\Pi_{name} (Join (Sales \bowtie Reserves) \bowtie (\sigma_{(c \leq 100000)} (Sales)))$



4. Show how to derive the following equivalences by a sequence of transformations using the equivalence rules $\sigma_{(E_1 \wedge E_2)}(R) = \sigma_{(E_1)}(\sigma_{(E_2)}(R))$ and $\sigma_{(E_1)}(\sigma_{(E_2)}(R)) = \sigma_{(E_2)}(\sigma_{(E_1)}(R))$ [14.53H]

Applying the associative law of conjunction:

$$\sigma_{(c \leq 100000 \wedge E)}(R) = \sigma_{(c \leq 100000)}(\sigma_{(E)}(R))$$

Applying the selection-pushdown rule

$$\sigma_{(c \leq 100000 \wedge E)}(R) = \sigma_{(c \leq 100000)}(\sigma_{(E)}(R))$$

$$\sigma_{(c \leq 100000 \wedge E)}(R) = \sigma_{(c \leq 100000)}(\sigma_{(E)}(R))$$

drives," which can be plugged into the Universal Serial Bus (USB) slots of computing devices.

iv. Magnetic Disk

The primary medium for the long-term online storage of data is the magnetic disk drive, which is also referred to as hard disk drive (HDD). Magnetic disk, like flash memory, is non-volatile; that is, magnetic disk storage survives power failures and system crashes. Disks may sometimes fail and destroy data, but such failures are quite rare compared to system crashes or power failures.

v. Optical Storage

The digital video disk (DVD) is an optical storage medium with data written and read back using a laser light source. The Blu-ray DVD format has a capacity of 27 gigabytes to 128 gigabytes, depending on the number of layers supported. Although the original (and still main) use of DVDs was to store video data, they are capable of storing any type of digital data, including backups of database contents. DVDs are not suitable for storing active database contents. DVDs are required to access a given piece of data since the time compared to the time taken by a magnetic disk.

vi. Tape Storage

Tape storage is used primarily for backup and archival data. Archival data refers to data that must be stored safely for a long period of time, often for legal reasons. Magnetic tape is cheaper than disks and can safely store data for many years. However, access to data is much slower because the tape must be accessed sequentially from the beginning of the tape; tapes can be very long, requiring tens to hundreds of seconds to access data. For this reason, tape storage is referred to as sequential-access storage. In contrast, magnetic disk and SSD storage are referred to as direct-access storage because it is possible to read data from any location on disk.

4.3 Organization of Records into Blocks

A database is mapped into a number of different files that are organized by the underlying operating system. These files reside on disks. A file is organized logically as a sequence of records. These records are mapped onto disk blocks. Each file is logically partitioned into fixed-length storage units called blocks, which are the units of both storage allocation and data transfer.

Most databases use block sizes of 4 to 8 kilobytes by default. Not many databases allow the block size to be specified when a database instance is created. A block may contain several records, the exact set of records that a block contains is determined by the form of physical data organization being used.

There are two types of Records:

- Fixed length Records.
- Variable length Records.

4.3.1 Fixed Length Records

As an example, let us consider a file of instructor records for our university database. Each record of this file is defined (in pseudocode) as:

```
type instructor = record
    id varchar(5);
    name varchar(20);
    dept name varchar(20);
    salary numeric(8,2);
end
```

Assume that each character occupies 1 byte and that numeric(8,2) occupies 8 bytes. Then, the instructor record is 53 bytes long. A simple approach is to use the first 53 bytes for the first record, the next 53 bytes for the second record, and so on.

However, there are two problems with this simple approach. Unless the block size happens to be a multiple of 53 (which is unlikely), some records will cross block boundaries. That is,

part of the record will be stored in one block and put in another. It would thus require two block accesses to read or write such a record.

It is difficult to delete a record from this structure. The space occupied by the record to be deleted must be filled with some other record of the file, or we must have a way of marking deleted records so that they can be ignored.

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califert	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Figure: File containing instructor records

To avoid the first problem, we allocate only as many records to a block as would fit entirely in the block (this number can be computed easily by dividing the block size by the record size, and are left unused. When a record is deleted, we could move the record that came after it into the space formerly occupied by the deleted record, and so on, until every record following the deleted record had been moved ahead.

Such approach requires moving a large number of records, it might be easier simply to move the final record of the file into the space occupied by the deleted record. It is undesirable to move

records to occupy the space freed by a deleted record, since doing so requires additional block accesses. Since insertions tend to be so frequent than deletions, it is acceptable to leave open the space occupied by the deleted record, and to wait for a subsequent insertion before reusing the space.

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	32343	El said	History	60000
record 4	33456	Gold	Physics	87000
record 5	34565	Katz	Comp. Sci.	75000
record 6	45665	Katz	Comp. Sci.	75000
record 7	58583	Califert	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Figure: Instructor file, with record 3 deleted and all records moved

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	15151	Mozart	Music	40000
record 4	32343	El said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califert	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

Figure: Instructor file with record 3 deleted and final record moved

At the beginning of the file, we allocate a certain number of bytes as a **file header**. The header will contain a variety of information about the file. For now, all we need to store there is the address of the first record whose contents are deleted.

We use this first record to store the address of the second available record, and so on. We can think of these addresses as pointers, since they point to the location of a record. The deleted records thus form a linked list, which is often referred to as a **free list**. The figure below the file of instructor, with the free list, after records 1, 4, and 6 have been deleted.

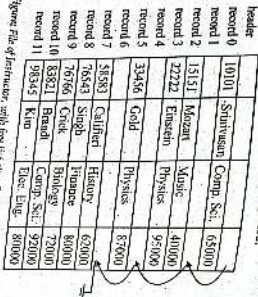


Figure: File of instructor, with free list after deletion of records 1, 4, and 6

On insertion of a new record, we use the record pointed to by the header. We change the header pointer to point to the next available record. If no space is available, we add the new record to the end of the file. Insertion and deletion for files of fixed-length records are simple to implement, because the space made available by a deleted record is exactly the space needed to insert a record.

8.2.2 Variable Length Records

ways:

1. Storage of multiple record types in a file.

262 | Insights on Database Management Systems

record types that allow variable lengths for one or more fields.
record types that allow repeating fields, such as arrays or multisets.
different techniques for implementing variable-length pointers exist. Two different problems must be solved by any such technique:
how to represent a single record in such a way that individual attributes can be extracted easily.
How to store variable-length records within a block, such that records in a block can be extracted easily.

The representation of a record with variable-length attributes typically has two parts: an initial part with fixed-length attributes, followed by data for variable-length attributes. Fixed-length attributes, such as numeric values, dates, or fixed-length character strings are allocated as many bytes as required to store their value.

Variable-length attributes, such as varchar types, are represented in the initial part of the record by a pair (offset, length), where offset denotes where the data for that attribute begins within the record, and length is the length in bytes of the variable-sized attribute. The values for these attributes are stored consecutively, after the initial fixed-length part of the record. Thus, the initial part of the record stores a fixed size of information about each attribute, whether it is fixed-length or variable-length. An example of such a record representation is shown in figure:

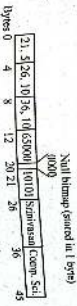


Figure: Representation of variable-length record

The figure shows an instructor record, whose first three attributes (ID, name, and dept name) are variable-length strings, and whose fourth attribute (salary) is a fixed-sized number. We

assume that the offset and length values are stored in two bytes each, for a total of 4 bytes per attribute. The salary attribute is assumed to be stored in 8 bytes, and each string takes as many bytes as it has characters.

The figure also illustrates the use of a null bitmap, which indicates which attributes of the record have a null value. In this particular record, if the salary were null, the fourth bit of the bitmap would be set to 1, and the salary value stored in bytes 12 through 19 would be ignored. The slotted-page structure is commonly used for organizing records within a block, and is shown in Figure. There is a header at the beginning of each block, and is containing the following information:

1. The number of record entries in the header.
2. The end of free space in the block.
3. An array whose entries contain the location and size of each record.

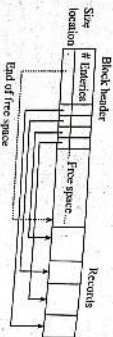


Figure: Slotted page structure

The actual records are allocated contiguously in the block, starting from the end of the block. The free space in the block is contiguous between the final entry in the header array, and the first record. If a record is inserted, space is allocated for it at the end of free space, and an entry containing its size and location is added to the header.

If a record is deleted, the space that it occupies is freed, and its entry is set to deleted (its size is set to -1, for example). Further, the records in the block before the deleted record are moved, so that the free space created by the deletion gets occupied, and all free space is again between the final entry in the

page array and the first record. The end-of-free-space pointer in the header is appropriately updated as well.

Records can be grown or shrunk by similar techniques, as long as there is space in the block. The cost of moving the records is not too high, since the size of a block is limited: typical values are around 4 to 8 kilobytes.

The slotted-page structure requires that there be no pointers to records directly to records. Instead, pointers must point to the first entry in the header that contains the actual location of the record. This level of indirection allows records to be moved to prevent fragmentation of space inside a block, while supporting indirect pointers to the record.

Byte String Representation

In byte string representation we attach a special end of record (-) symbol to the end of record. Each record is stored as a string of successive bytes.

	0	1	2	3	4	5
Perryridge	A-102	400	A-201	900	A-218	700
Round Hill	A-305	350	-	-	-	-
Mianus	A-215	700	-	-	-	-
Downtown	A-101	500	A-110	600	-	-
Redwood	A-222	700	-	-	-	-
Brighon	A-217	750	-	-	-	-

Figure: Byte string representation

Byte string notation has several disadvantages:

1. It is not easy to reuse space left by a deleted record.
2. In general there is no space for records to grow longer. If the records become longer it must be moved.

Fixed Length Representation with Reserved Space

Can use fixed length records of a known maximum length. Unused space in shorter records filled with a null or end of record symbol.

0	Perryridge	A-102	400	A-201	900	A-218	750
1	Round Hill	A-305	350	—	—	—	—
2	Milsons	A-215	700	—	—	—	—
3	Downtown	A-101	500	A-110	600	—	—
4	Redwood	A-222	700	—	—	—	—
5	Brighton	A-217	750	—	—	—	—

Figure: Fixed length representation with overflow block

Fixed length Representation with Pointer

A variable length record is represented by a list of fixed length records that are chained together by pointers. Can be used even if the maximum record length is not known.



Figure: Variable length representation with pointer

The main disadvantage of the pointer structure is that records are stored in all records, except for the file in chain. Alternation

two different blocks of blocks in a file. Overflow block contains the first record of each chain. Other than that the first records of chains.

Overflow block contains the first record of each chain. Other than that the first records of chains.

Perryridge	A-102	400
Round Hill	A-305	350
Milsons	A-215	700
Downtown	A-101	500
Redwood	A-222	700
Brighton	A-217	750

A-201	900
A-218	700
A-110	600

Figure: Fixed length representation with pointer using overflow block and overflow block

6.3 File Organizations

The way data is organized and stored within a file or group of files in a computer system is referred to as the organization. It governs how information is stored in the file(s), accessed, and retrieved. The type of data access patterns, performance requirements, and simplicity of maintenance all play a role in the choice of the organization. There are different ways to logically organize records in a file. They are:

1. **Heap file organization**
A record can be placed anywhere in the file where there is space; there is no ordering in the file.
2. **Sequential file organization**
Store records in sequential order based on the value of the search key of each record.
3. **Hashing file organization**
A hash function is computed on some attribute of each record. The result of the hash function specifies in which block of the file the record is placed.
4. **Multistage clustering file organization**
Records of several different relations can be stored in the same file. Allocation of clustering the organization is to store related records on the same block or cylinder I/O.

5. B+-tree file organization.

The B+-tree file organization is related to the B+-tree indexing structure and can provide efficient ordered access to records even if there are a large number of insert, delete, or update operations. Further, it supports very efficient access to specific records based on the search key.

8.3.1 Sequential File Organization

A sequential file organization is designed for efficient processing of records in sorted order based on some key. A search key is any attribute or set of attributes; it need not be the primary key, or even super key. Sequential file organization is suitable for applications that require sequential processing of the entire file.

To permit fast retrieval of records in search key order, we chain together records by pointers. The pointer in each record points to the next record in search key order. To minimize the number of block accesses in sequential file processing, we store records physically in search key order or as close to the search key order as possible.

10101	Srinivasa	Comp. Sci.	65000
12121	Vin	Finance	90000
13151	Mozart	Music	40000
22222	Einstein	Physics	95000
33456	Gold	Library	60000
45665	Koz	Physics	87000
58583	Calderin	Comp. Sci.	75000
76543	Smith	Finance	61000
76766	Chek	Biology	80000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Eng. Eng.	100000

Figure: Sequential File Organization

10101	Srinivasa	Comp. Sci.	65000
12121	Vin	Finance	90000
13151	Mozart	Music	40000
22222	Einstein	Physics	95000
33456	Gold	Library	60000
33456	Gold	Physics	87000
45665	Koz	Comp. Sci.	75000
58583	Calderin	History	62000
76543	Smith	Finance	80000
76766	Chek	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Eng. Eng.	100000
72222	Venti	Music	48000

Figure Sequential file after an insertion

The sequential file organization allows records to be read in sorted order; that can be useful for display purposes as well as for certain query processing algorithms. It is difficult, however, to maintain physical sequential order as records are inserted and deleted, since it is costly to move many records as a result of a single insertion or deletion.

We manage deletion by using pointer chains as we saw previously. For illustration we apply the following rules:

1. Locate the record in the file that comes before the record to be removed in sorted key order.

2. If there is a next record (that is, space left after a deletion) work in the same block as this record, insert new record there. Otherwise insert the new record in an earlier block. In either case, adjust the pointers so as to chain together the records in search key order.

8.3.2 Indexed Sequential File Organization

Indexed Sequential File Organization is a method of organizing data as a composite of files where the data is sorted in a sequential order, but an index is used to access the data quickly.

In Indexed Sequential File Organization, the data is kept in fixed length blocks or pages, and each block is given a specific index that points to the block's first record. The index is usually maintained in a separate file or data structure, and it is sorted based on the primary key of the data.

To access a specific record in an indexed sequential file, the system first searches the index for the appropriate block or page. Once the block is located, the system can then access the record sequentially within the block. This allows for fast random access to specific records while also providing efficient sequential processing of data.

The indexed sequential file organization is commonly used in large databases where data needs to be accessed quickly and efficiently. It is also useful for applications that require a combination of sequential and random access to data, such as financial systems or inventory management systems.

Example:

Let's look at an illustration of an indexed sequential file arrangement in the context of a student database.

Suppose we have a table that contains the following fields:

Student ID (primary key)
Name
Age
Gender
Address

The data would be kept in fixed-length blocks or pages with a specific number of records on each block. In an indexed sequential file organization, consider the case when there are 50 records in each block.

Each index entry in the index file points to the starting block of a series of data blocks containing records with key values within a certain range. The index entry would typically contain the highest key value in that range and the block number where those records begin. A sample of the index file might be as follows:

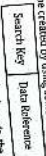
Student ID	Block Number
1050	1
1100	2
...	...
1250	5
1300	6
...	...
1500	10

Suppose we want to access the record for student ID 1275. The system would first search the index file to find the appropriate block number (in this case block number 6). Once it has located the correct block, the system can then access the records sequentially within the block until it finds the record for student ID 1275. This allows for fast random access to specific records while also providing efficient sequential processing of data.

Indexing is used to optimize the performance of a database by minimizing the number of disk access required when a query is processed. The index is a type of data structure. It is used to locate and access the data in database quickly.

Index structure

Index can be created by using some database columns



The first column of the database is the search key that contains a copy of the primary key or candidate key of the table. The values of the primary key are stored in sorted order so that the corresponding data can be accessed easily. The second column of the database is the data reference. It contains a set of pointers holding the address of the disk block where the value of the particular key can be found.

Index Evaluation Metrics

Index evaluation metrics are the metrics which are used to determine the efficiency of index. Different factors determine how efficient an index is but followings are the metrics which play a role in determining the efficiency of an index:

i. Access type

Access type is finding records with specified attribute value. Finding records whose attribute value falls in a specified range.

ii. Access time

Access time is time to find particular value.

iii. Insertion time

Insertion time is time to insert new value as well as to update the index structure.

iv. Deletion time

Deletion time is time to delete data item as well as to update the index structure.

v. Space overhead

Space overhead is additional space occupied by an index structure.

There are two basic kinds of indices:

1. Ordered indices
2. Hash indices

• Ordered indices

In an ordered index, the indices are usually sorted to make searching faster. The indices which are sorted and associates with each search key the records that contain it are known as ordered indices.

Example:

Suppose we have an employee table with thousands of record and each of which is 10 bytes long. If their IDS start with 1,2,3... and so on and we have to search student with ID-543, in case of database with no index, we have to search the disk block from starting till it reaches 543. The DBMS will read the record after reading 543*10=5430 bytes. In case of an index,

we will search using indexes and the DBMS will read the record after reading $543 \times 2 = 1084$ bytes which are very less compared to previous case.

Dense index

The dense index contains an index record for every search key value in the data file. It makes searching faster. The key value in the index table is the same as the number of records in the main table. It needs more space to store index record itself. The index records have the search key and a pointer to the actual record on the disk.

Pakistan	→	Pakistan	Lahore	1604300
USA	→	USA	Chicago	2789378
Nepal	→	Nepal	Katmandu	1456634
UK	→	UK	Cambridge	1360364

Figure: Dense Index

h. Sparse index

In sparse index, instead of pointing to each record in the main table the index points to the records in the main table in eggs, each item pointing to a block.

Pakistan	USA	Pakistan	Lahore	1604300
Nepal	USA	Chicago	2789378	
UK	Nepal	Katmandu	1456634	
	UK	Cambridge	1360364	

Figure: Sparse index

In this method of indexing, range of index columns store the same data block address, and when data is to be retrieved, the block address will be fetched linearly till we get the requested data. The main goal of this method should be more efficient search with less memory space.

Student-ID	Student-name	Age	Course	Index address
100	Joseph	20	200	
103	Patty	22	205	
106	James	19	200	

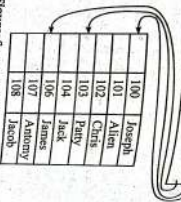


Figure: Sparse index

Following are the differences between dense index and sparse index:

Dense Index	Sparse Index
Index size is larger	Index size is smaller
Records in the data file does not need clustering	Records in the data file needs clustering
Data can be located in less time	Locating of data takes more time
Consumes less computing time in RAM	Consumes more computing time in RAM
The overhead for the insertion and deletion is more	The overhead for the insertion and deletion is less
Pointers point to each record in the data file	Pointers point to fewer records in data file

c. Clustering Index

A clustered index can be defined as an ordered data file. Sometimes the index is created on non-primary key columns which may not be unique for each record. In this case, to

identify the record faster, we will group two or more columns to get the unique value and create index out of them. This method is called a clustering index. The records which have similar characteristics are grouped, and indexes are created for these group.

Example:

Suppose a company contains several employees in each department. Suppose we use a clustering index, where all employees which belong to the same Dept_ID are considered within a single cluster, and index pointers point to the cluster as a whole. Here Dept_ID is a non-unique key.

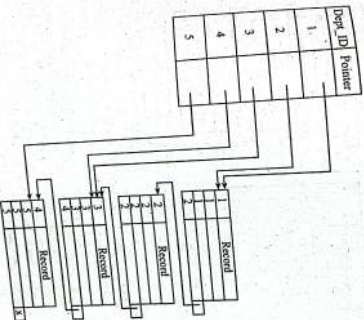


Figure: Clustering index

The previous schema is little convincing because now the block is shared by records which belong to the different files. We use separate disk block for separate clusters, then it is better technique.

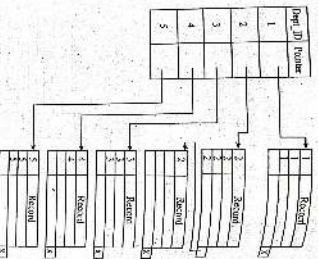


Figure Clustering index using separate disk block for separate clusters.

d. Primary Index

If the index is created on the basis of the primary key of the table, then it is known as primary indexing. These primary keys are unique to each record contain 1:1 relation between the performance of the searching operation is quite efficient.

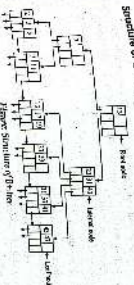
e. Secondary Index

Indices whose search key specifies an order different from the sequential order of the file are called non clustering indices or secondary indices.

B+ Tree Index

B+ tree is a balanced binary search tree that uses a tree like structure to store records in file. B+ tree has three levels: the root, internal level and the leaves and also ensures that all leaf nodes remain at the same height, so it is balanced. The leaf nodes are linked using a linked list, so B+ tree represents the medium are linked access. Order (n) of B+ tree represents the maximum no. of children a node can have. Degree of B+ tree represents maximum possible no of children a node can have. $n \geq 3$ is preferred. Degree $\leftarrow$ Order.

Structure of B+ tree



Root Node

Every leaf node is at equal distance from the root node and degree is 2.

Internal Node

Internal (non-leaf) nodes contain at least $\lceil n/2 \rceil$ pointers except the root node. A root, an internal node can contain n pointers where 'n' represents order of B-tree.

Leaf Node

Leaf nodes contain at least $\lceil (n-1)/2 \rceil$ record pointers and $\lceil (n-1)/2 \rceil$ key values. A root, a leaf node can contain one block pointer and $(n-1)$ key values. Every leaf node contains one block pointer to point to next leaf node and forms a linked list.

B+ tree Node Structure

Unlike binary tree, Node of B+ tree can have more than 1 key.
Typical Node has following structure.

P_1	K_1	P_2		P_{n-1}	K_n	P_n
						?

K are the search key values and P are the pointers to either pointers to records (in case of leaf nodes). The search keys in the node are ordered as:

$$K_1 < K_2 < \dots < K_n$$

Advantages of B+ Tree

a. Efficient Search

B+ trees provide efficient search with $O(\log n)$ complexity for both insertion and retrieval operations. This logarithmic search time remains constant even as data size increases, making them suitable for large data sets.

b. Range Queries

B+ trees support efficient range queries. They allow us to efficiently traverse through tree nodes to search all records within a given area, making them suitable for applications where point-based or bucket are common.

c. Balanced Structure

B+ trees are balanced data structures. A balanced tree structure is automatically maintained for insertions and deletions, ensuring that the tree remains relatively shallow and the search time remains the same.

d. Sorted Order

B+ trees maintain data in a structured order, which can be useful for applications that require structured data retrieval or produce structured results without requiring additional processing.

e. Large Datasets Support

B+ trees can handle large amounts of data more efficiently because they keep the tree height short, reducing the number of disk I/O operations required during recovery.

Effective Range Scan

B+ trees work particularly well for range scans or scans in selective parts of a data set. Their leaf nodes are often connected in overlapping lines, allowing for better sequential access.

Insertion and Deletion Support

B+ trees can handle insertions and deletions more efficiently, without significant loss of performance. They maintain values and avoid data fragmentation.

disk I/O Reduction

B+ trees reduce disk I/O by using large buffer factors and managing data appropriately to reduce the number of disk blocks accessed during queries.

disadvantage of B+ Tree is that it is inefficient for exact matches.

8.5 Hash Index

In a hash index, the search key is mapped to a location in the index structure using a hash function. The index structure typically consists of an array of pointers, or buckets, where each bucket holds a list of pointers to data entries with the same hash value.

Formally, let K denote the set of all search-key values, and let B denote the set of all bucket addresses. A hash function H is a function from K to B . Let h denote a hash function, with n memory bucket locations, the set of buckets is simply an array of pointers, with the j bucket at offset j from pointer store the head of a linked list containing the entries in that bucket.

To insert a record with search key K , we compute $H(K)$, which gives the address of the bucket for that record. We add the index entry for the record in the list at offset K . That index handles the case of multiple records in a bucket using another chaining. If the bucket does not have enough space, a bucket overflow is said to occur. We handle bucket overflows by using overflow buckets. If a record must be inserted into an overflow bucket is already full, the system provides an overflow bucket. It inserts the record into the overflow bucket.

is also full, the system provides another overflow bucket, and so on. All the overflow buckets of a given bucket are chained together in a linked list, as in Figure below. With overflow chaining, given search key k , the lookup algorithm must then search not only bucket $h(k)$, but also the overflow buckets linked from bucket $h(k)$.

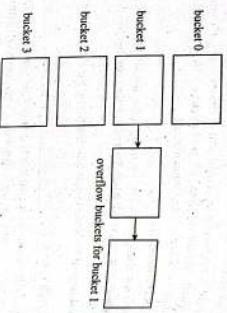


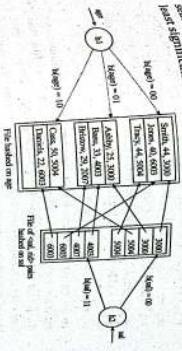
Figure: Overflow chaining

Hash indexing using overflow chaining is also called closed addressing (or, less commonly, closed hashing). An alternative hashing scheme called open addressing is used in some applications, but is not suitable for most database indexing applications since open addressing does not support deletes efficiently.

To search for a record with a given search key value, we apply the hash function to identify the bucket to which such records belong and look at all pages in that bucket. If we do not have the search key value for the record, for example, the index is based on sal and we want records with a given age value, we have to scan all pages in the file.

Hash indexing is illustrated in Figure, where the data is stored in a file that is hashed on age; the data entries in this first index file are the actual data records. Applying the hash function

on the age field identifies the page that the record belongs to. The hash function h for this example is quite simple; it converts the search key value to its binary representation and uses the two least significant bits as the bucket identifier.



Example of file organization using hash indices

Suppose we have a database of students, and the table 'Students' has the following attributes:

StudentID (primary key)

Name

Age

Major

We want to organize the file using the Major attribute as the search key.

Hash Function

Let's define a simple hash function that takes the Major attribute as input and output

i.e., $h(k)=k$

Hence, for CSE, ENG and BIO the hash values are CSE, ENG and BIO respectively

Buckets or Blocks

We divide the file into fixed-sized buckets or blocks, where each bucket corresponds to a unique major. For simplicity, let's assume we have three majors: CSE (Computer Science

and Engineering), "ENG" (English), and "Bio" (Biology). We create three buckets: one for each major.

Insertion

When inserting a new student record into the file, we apply the hash function to the Major attribute to determine the appropriate bucket. We then place the record in that bucket. For example, if a new student with Major "CSE" is added, the record will be placed in the CSE bucket.

Searching

To search for a student based on the Major attribute, we apply the hash function to the search key. The hash function will provide us with the bucket where the records with that major are stored. We only need to search within that bucket, making the search process more efficient.

Collision Handling

In case of collisions, where multiple records hash to the same bucket due to the limitations of the hash function or similar major names, collision handling techniques can be employed. For instance, separate chaining can be used, where each bucket maintains a linked list of records with the same hash value.

Example Data:

StudentID	Name	Age	Major
1	Alice	20	CSE
2	Bob	21	ENG
3	Charlie	19	CSE
4	David	20	BIO
5	Emma	22	CSE

Suppose we apply the hash function and divide the records into buckets based on the major. The resulting organization might look like this:

CSE Bucket:

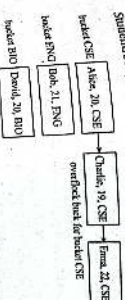
studentID: 1, Name: Alice, Age: 20, Major: CSE
studentID: 3, Name: Charlie, Age: 19, Major: CSE
studentID: 5, Name: Emma, Age: 22, Major: CSE

ENG Bucket:

studentID: 2, Name: Bob, Age: 21, Major: ENG

BIO Bucket:

studentID: 4, Name: David, Age: 20, Major: BIO



When searching for a student with a specific major, such as "CSE," we can directly access the CSE bucket and retrieve the relevant records without having to search through the entire dataset. This provides efficient and quick access to the desired data based on the search key.

SOLUTION TO EXAMS' AND OTHER IMPORTANT QUESTIONS

1. How do you evaluate the performance of magnetic disk block access?

Ans The performance of a magnetic disk can be evaluated using several metrics, including:

- i. **Transfer Rate:** It measures how quickly information may be read from or written to a disk. It usually refers to the speed at which data may be moved to and from the disk and is represented in megabytes per second (MB/s).
 - ii. **Seek Time:** It calculates how long it takes the read/write head of the disk to move between various tracks. The seek time contributes to the overall latency of accessing data on the disk and is often expressed in milliseconds (ms).
 - iii. **Latency:** It shows the interval of time between a data request and the beginning of a data transfer. It comprises of time it takes for the rotational delay (the amount of time it takes for the read/write head to revolve around the required data sector).
 - iv. **Input/Output Operations Per Second (IOPS):** It counts how many read/write operations the disk can handle in a single second. In general, better performance is indicated by more IOPS.
- To optimize disk block access and reduce the number of disk block accesses, the following techniques can be employed:
- i. **Caching:** By keeping frequently used data in faster storage (such as RAM or solid-state drives) closer to the processor, a cache can greatly boost performance. Data served straight from the cache through caching decreases the need for disk block access.
 - ii. **Disk Partitioning:** Performance can be enhanced by partitioning a big disk into several smaller ones. We can

cut down on the amount of time needed to seek out specific data blocks by grouping data according to usage patterns or access frequency.

- iii. **Defragmentation:** Defragmenting the disk on a regular basis keeps files and data blocks organized and sequential, cutting down on search times and enhancing overall disk performance.

iv. **File System Optimization:** Performance on disks can be improved by selecting an effective file system. Clustering, journaling, and complex data structures are some of the methods used by file systems like NTFS (New Technology File System) and ext4 (Fourth Extended File System) to optimize disk access.

v. **RAID (Redundant Array of Independent Disks):** RAID implementation can increase performance and offer fault tolerance. For instance, disk striping is used in RAID 0 to distribute data over many disks, enabling simultaneous disk block access and improved performance.

vi. Intelligent Disk Scheduling

vii. Data Compression

viii. Prefetching and Read-Ahead

2. In terms of file organization, define indexing. Elevator Algorithm, Log disk. How does mechanical hard disk work?

Ans Indexing

Indexing is a file organization technique that improves data retrieval efficiency by creating additional pairs. The key is called indexes. An index contains key-value pairs in the data file and the usually a field or combination of fields in the data file and the value is a pointer to the corresponding record. Instead of search and retrieve specific records faster based on indexed key values instead of scanning the entire file.

Elevator Algorithm

The elevator algorithm is a disk scheduling algorithm that works by maintaining a queue of pending disk requests, processing them in a particular direction. First, the head starts at one end of the disk and moves in one direction, servicing all requests in that direction until it reaches the end. Then change direction and process the remaining requests in the opposite direction.

Log Disk

Log disk is a technique used by some file systems to improve the efficiency and reliability of disk operations. It keeps file data in a separate area of the disk that is not changed by the system before they are applied to the main data structures. A log file serves as a continuous record of all disk operations, such as the modifications, deletions, and metadata updates. A log disk enables fast and consistent file system operations because the log can be used to restore the file system to a consistent state in the event of a system failure or crash.

Working of Mechanical Hard Disk

A mechanical hard drive or hard disk drive (HDD) is a non-volatile storage device that uses spinning disks or platters to store and retrieve data. Here's an overview of how mechanical hard drives work:

Platters

A hard drive consists of several circular platters coated with a magnetic material. These platters are stacked one on top of the other and spin at high speeds (typically thousands of revolutions per minute) on spindles.

Read/Write Heads

Each platter has its own read/write head. Floating just above its surface, the read/write head is attached to an actuator arm assembly so it can move across the surface of the platter.

Open Access

Open access is a disk scheduling algorithm that moves the read/write head to the platter. When reading data, the head is positioned over the track on the disk surface and converts it to an electrical signal. When writing data, the head magnetically encodes the data onto the platters.

Sectors and Tracks

The disk surface is divided into concentric tracks and sectors. Tracks are concentric circles on the platter and sectors are small pie-shaped areas within the track. Data is stored in a fixed amount of data.

Spindle Motor

The platter is rotated by a spindle motor that rotates the disk at a constant speed. Rotation speed is measured in revolutions per minute (RPM).

Seek Time

The time it takes for the read/write head to reach the desired track is called the seek time. Faster search times mean faster data access.

Latency

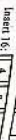
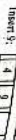
The time it takes for the read/write head to be positioned over the desired sector to spin underneath it is called latency. Affected by disk rotation speed.

3. Create a B+ tree of order 3 with following data:

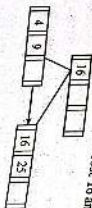
4, 9, 15, 25, 120, 125, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437, 438, 439, 440, 441, 442, 443, 444, 445, 446, 447, 448, 449, 450, 451, 452, 453, 454, 455, 456, 457, 458, 459, 460, 461, 462, 463, 464, 465, 466, 467, 468, 469, 470, 471, 472, 473, 474, 475, 476, 477, 478, 479, 480, 481, 482, 483, 484, 485, 486, 487, 488, 489, 490, 491, 492, 493, 494, 495, 496, 497, 498, 499, 500, 501, 502, 503, 504, 505, 506, 507, 508, 509, 510, 511, 512, 513, 514, 515, 516, 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 528, 529, 530, 531, 532, 533, 534, 535, 536, 537, 538, 539, 540, 541, 542, 543, 544, 545, 546, 547, 548, 549, 550, 551, 552, 553, 554, 555, 556, 557, 558, 559, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689, 690, 691, 692, 693, 694, 695, 696, 697, 698, 699, 700, 701, 702, 703, 704, 705, 706, 707, 708, 709, 710, 711, 712, 713, 714, 715, 716, 717, 718, 719, 720, 721, 722, 723, 724, 725, 726, 727, 728, 729, 730, 731, 732, 733, 734, 735, 736, 737, 738, 739, 740, 741, 742, 743, 744, 745, 746, 747, 748, 749, 750, 751, 752, 753, 754, 755, 756, 757, 758, 759, 760, 761, 762, 763, 764, 765, 766, 767, 768, 769, 770, 771, 772, 773, 774, 775, 776, 777, 778, 779, 780, 781, 782, 783, 784, 785, 786, 787, 788, 789, 790, 791, 792, 793, 794, 795, 796, 797, 798, 799, 800, 801, 802, 803, 804, 805, 806, 807, 808, 809, 810, 811, 812, 813, 814, 815, 816, 817, 818, 819, 820, 821, 822, 823, 824, 825, 826, 827, 828, 829, 830, 831, 832, 833, 834, 835, 836, 837, 838, 839, 840, 841, 842, 843, 844, 845, 846, 847, 848, 849, 850, 851, 852, 853, 854, 855, 856, 857, 858, 859, 860, 861, 862, 863, 864, 865, 866, 867, 868, 869, 870, 871, 872, 873, 874, 875, 876, 877, 878, 879, 880, 881, 882, 883, 884, 885, 886, 887, 888, 889, 890, 891, 892, 893, 894, 895, 896, 897, 898, 899, 900, 901, 902, 903, 904, 905, 906, 907, 908, 909, 910, 911, 912, 913, 914, 915, 916, 917, 918, 919, 920, 921, 922, 923, 924, 925, 926, 927, 928, 929, 930, 931, 932, 933, 934, 935, 936, 937, 938, 939, 940, 941, 942, 943, 944, 945, 946, 947, 948, 949, 950, 951, 952, 953, 954, 955, 956, 957, 958, 959, 960, 961, 962, 963, 964, 965, 966, 967, 968, 969, 970, 971, 972, 973, 974, 975, 976, 977, 978, 979, 980, 981, 982, 983, 984, 985, 986, 987, 988, 989, 990, 991, 992, 993, 994, 995, 996, 997, 998, 999, 1000.

4. Create a B+ tree of order 4 with the given data

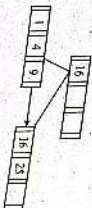
10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150, 160, 170, 180, 190, 200, 210, 220, 230, 240, 250, 260, 270, 280, 290, 300, 310, 320, 330, 340, 350, 360, 370, 380, 390, 400, 410, 420, 430, 440, 450, 460, 470, 480, 490, 500, 510, 520, 530, 540, 550, 560, 570, 580, 590, 600, 610, 620, 630, 640, 650, 660, 670, 680, 690, 700, 710, 720, 730, 740, 750, 760, 770, 780, 790, 800, 810, 820, 830, 840, 850, 860, 870, 880, 890, 900, 910, 920, 930, 940, 950, 960, 970, 980, 990, 1000.



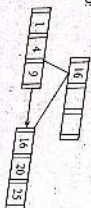
Split from 9 or 16. Here we choose 16 and move 16 up.



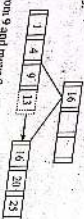
Insert 1:



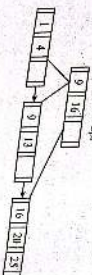
Insert 20:



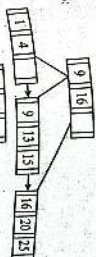
Insert 13:



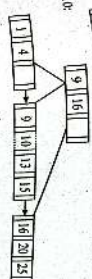
Split from 9 and move 9 up.



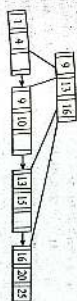
Insert 15:



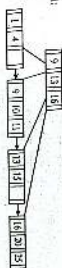
Insert 10:



Split from 13 and move 13 up



Insert 11:



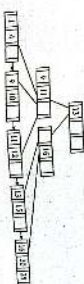
Insert 12:



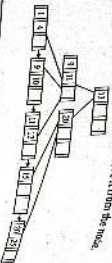
Split from 11 and move 11 up



Split from 13 and move 13 up



To delete 16 from the B+ tree, we need to follow these steps.
Search for the key 16 in the tree.
Since 16 is located in a leaf node, remove it from the node.



4. Construct a B+ tree for the following set of key values (2,3,5,7,11,17,19,23,29,31). Assume that the tree is initially empty and values are added in ascending order where the pointer number is four.

2019

Insert 2:



Insert 3 & 5:



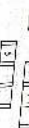
Insert 7:



Split leaf 5 and move 5 up.



Insert 11:

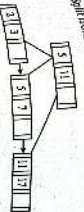


Insert 17:

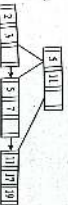


280 | Single on Overlap Management Systems

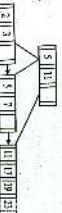
split from 11 and move 11 up.



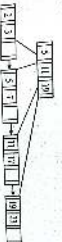
Insert 19:



Insert 23:



Split from 19 and move 19 up.



Insert 29:



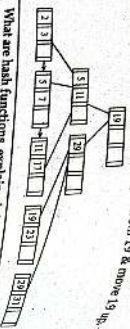
Insert 31:

31 will go to the leaf containing 19, 23 & 29 but as full so split from 29 & move 29 up.



the operation of Insert 1201

Also root 15, 10, 19 & 29 is full so split from 19 & move 19 up.



5. What are hash functions, explain giving example.

A hash function, also known as a hash value or hash code, is a mathematical operation that accepts an input (or "key") and outputs a fixed-length string of characters. A hash function's main goal is to identify data uniquely and make efficient retrieval and storage activities possible.

Let's consider a simplified hash function that calculates the hash value based on the length of a word. The steps for this example are as follows:

1. Take the input word: For instance, let's use the word "apple".

2. Calculate the length of the word: The length of "apple" is 5 characters.

3. Assign a fixed value to each possible word length: Let's assume we have a predefined mapping where the hash characters for word lengths are as follows: 3 characters = 10, 4 characters = 20, 5 characters = 30, and so on.

4. Generate the hash value: Based on the length of "apple" (5 characters), the hash value would be 30 according to the predefined mapping.

6. When is it preferred to use dense index rather than sparse index? Explain with suitable example.

When there are a lot of range-based queries that need to be completed quickly and the data distribution in the indexed column is rather consistent, a dense index is preferable over a sparse index.

Example:
Imagine a database table that stores information about books in a library. One of the indexed columns is "Publication Year," which indicates the year each book was published.

If the distribution of publication years in the database is relatively uniform, a dense index would be a suitable choice.

If there is a dense index on the "Publication Year" column, an entry would be created for each book record in the index table. This means that every book, regardless of its publication year, would have an entry in the index.

Now, let's suppose we need to execute a query to find all books published between the years 2000 and 2010. In this case, a dense index is advantageous because it covers the entire range of publication years. The index entries allow for efficient retrieval of the books falling within the specified range, as each entry in the index points directly to the corresponding book record.

On the other hand, if the distribution of publication years is uneven, with some years having a large number of books and others having very few, a sparse index may be more appropriate.

For instance, consider a scenario where the majority of books in the database were published in the last five years, but there are only a few books from earlier years. In this case, a sparse index on the "Publication Year" column would be useful. The sparse index would have entries only for the years with a significant number of books, skipping the entries for years with fewer books.

When executing a query to retrieve books from a specific year, the sparse index allows for efficient access to the relevant records by using the index entry for that particular year. This helps reduce the size of the index and improve query performance, as the index focuses on the years that have a higher density of data.



9.1 Failure Classification

To find where the problem has occurred, we generalize a failure into following categories:

- i. Transaction Failure
- ii. System Crash
- iii. Disk Failure

Transaction Failure

The transaction failure occurs when it fails to execute or when it reaches a point it cannot go any further if few transactions or process is hurt, then this is called transaction failure.

Reasons for transaction failure could be:

a. Logical Errors

If a transaction cannot complete due to some code error or an internal error or an internal error condition then the logical error occurs.

b. System Error

System error occurs where the DBMS itself terminates an active transaction because the database system is not able to execute it.

Example:

The system aborts an active transaction in case of deadlock or resource unavailability.

System Crash

System crash can occur due to power failure or other hardware or software failure.

Example:

Operating System Error.

Database systems have numerous integrity checks to prevent corruption of disks data.

Disk Failure

Disk failure occurs where hard disk drives or storage drives fail frequently. It was a common problem in the early days of technology evolution. Disk failure occurs due to the formation of bad sectors, disk head crash and unreachability to the disk or any other failure, which destroy all or part of the disk storage.

9.2 Concept of Log Based Recovery and Shadow Paging

9.2.1 Log Based Recovery

The log is a sequence of records. Log of each transaction is maintained in some stable storage so that if any failure occurs, then it can be recovered from there. If any operation is performed on the database, then it will be recorded in the log. But the process of storing the logs should be done before the actual transaction is applied in the database.

When transaction T₁ starts, it registers itself by writing a <T₁, Start> log record. Before T₁ executes Write (X), a log record <T₁, X, V1, V2> is written where V1 is the value of X before the write and V2 is the value to be written to X. Log record notes that T₁ has performed a write on data item X. X had value V1 before the write and will have value V2 after the write. When T₁ finishes its last statement, the log record <T₁, Commit> is written. We assume that log records are written directly to stable storage (that is they are not buffered).

Let us assume that there is transaction to modify the city of a student. The following logs are written for this transaction.

When the transaction is initiated, then it writes 'start' log

<T₁, Start>

When the transaction modifies the city from 'Krm' to 'Pohbar' then another log is written to the file

<T₁, city, 'Krm', 'Pohbar'>

When the transaction is finished then it writes another log to indicate the end of the transaction.

<T_n, commit>

- There are two approaches to use logs:
1. Deferred database modification.
 - ii. Immediate database Modification.

Deferred Database Modification

The deferred modification technique occurs if the transaction does not modify the database until it has committed. The method all the logs are created and stored in the stable storage and the database is updated when the transaction commits. In this deferred database modification scheme records are stored in the log but defers all the writes to partial commit.

If the system crashes before the transaction completes its execution or if the transaction aborts, then the information on the log is simply ignored. Deferred database modification technique ensures transaction atomicity by recording all database modification in the log. Assume that transactions execute serially.

(X) operation starts by writing <T₁, Start> record to the log. A Write where V is the new value of X.

Note: Old value is not needed for this scheme.

The write is not performed on X at this time but is deferred.

When T₁ partially commits <T₁, Commit> is written to the log. Finally, the log records are read and used to actually execute the failure that results in information loss. The recovery schemes use the following recovery procedures.

Redo (T₁) sets the value of all data items updated by transaction T₁ to the new values. New value can be found in the log. The redo operation must be idempotent to executing it several times must be equivalent to executing it once. This is required if we are to generate correct behavior even if a failure occurs during the recovery process. During the recovery after a crash, a transaction needs to be redone if and only if both <T₁,

<T₁, Commit> are there in the log. Redoing a log record (Redo T₁) sets the value of all data items updated by transaction to the new value. Crashes can occur while the transaction is executing the original updates or while recovery is being taken.

<T ₀ , start>	<T ₀ , start>	<T ₀ , start>
<T ₀ , A,950>	<T ₀ , A,950>	<T ₀ , A,950>
<T ₀ , B,2050>	<T ₀ , B,2050>	<T ₀ , B,2050>
	<T ₀ , Commit>	<T ₀ , Commit>
	<T ₁ , Start>	<T ₁ , Start>
	<T ₁ , C,600>	<T ₁ , C,600>
		<T ₁ , Commit>
A	B	C

If log on stable storage at the time of crash is given as above (i.e. a, b, c). No redo actions need to be taken for case 'a'. Redo (T₀) must be performed for case 'b' since <T₀, Commit> is present. Redo (T₀) must be performed followed by redo (T₁) since <T₀, Commit> and <T₁, Commit> are present for case 'c'.

Immediate Database Modification

The immediate update technique allows database modification in output the database while the transaction is still in the active state. Data modification written by active transaction are called uncommitted modification. Since undoing may be needed. Update logs must have both old and new value. Recovery procedure has two operations instead of one.

1. **Undo (T₁)** restores the value of all data items updated by T₁ to their old values, going backwards from the last record of T₁.

ii. **Redo (T₁)** sets the value of all data items updated by T₁ to the new values going forward from the first log record of T₁.

Both operations must be idempotent i.e. even if the operation is executed multiple times the effect is the same as if it was executed once. When recovering after failure:

i. Transaction T1 needs to be undone if the log contains the record <T1, Start> but does not contain the record <T1, Commit>

ii. Transaction T1 needs to be redone if the log contains the record <T1, Start> and the record <T1, Commit>

Undo operations are performed first then redo operations.

<T0, Start>

<T0,A,1000,950>

Undo(T0);

<T0,B,2000,2050>

<T0, Start>

B is redone to 2000 and A is to 1000

<T0,A,1000,950>

<T0,B,2000,2050>

Undo(T1) and redo(T0);

<T0, Commit>

C is redone to 700 and then A and B are set to 550 and 2050 respectively

<T1, Start>

<T1,C,700,600>

<T0, Start>

<T0,A,1000,950>

<T0,B,2000,2050>

<T0, Commit>

Both (T0) and redo (T1). A and B are set to 950 and 2050 respectively. Then C is set to 600

<T1, Start>

<T1,C,700,600>

<T1, Commit>

Check Pointing

Check pointing is the mechanism to mark the entries in the log file that those changes are permanently updated into database and if there is any failure log files need not traversed beyond that point. Only those entries after checkpoint are not written to DB or as per the schedule. This is done at periodic intervals

Checkpointing is done as follows:

1. Output all log records currently residing in main memory, into stable storage.
2. Output all modified buffer blocks to the disk.
3. Output a log record of the form <checkpoint L>, where L is a list of transactions active at the time of the checkpoint onto stable storage.

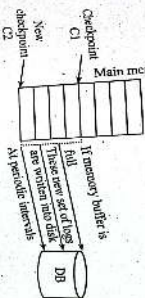


Figure: Checkpointing

Recovery using Checkpoints

Following problems occur in recovery procedure as discussed earlier:

- i. Searching the entire log is time consuming
- ii. We might unnecessarily redo transactions which have already output their updates to the database.

During recovery we need to consider only the most recent transaction T, that started before the checkpoint

- i. Scan backward from the end of the log to find the most recent <Checkpoint> record
- ii. Continue scanning backwards till a record <T_i, Start> is found.
- iii. Need only to consider the part of the log following above start record. Earlier part of the log is can be ignored during the recovery and can be erased whenever desired.

after identified T_1 , the redo and undo operations is to be applied to the T_1 and all T_1 has started execution after T_1 transaction. For all transactions (starting from T_1 or later) with no commit, execute undo (T_1) (Done only in case of immediate modification. Scanning forward in the log for all transactions from T_1 , or later with a $c < 1$, commit, execute redo (T_1)).

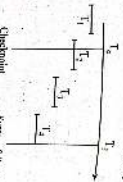


Figure 9.20: System failure

T_1 can be ignored (if updates already output to disk or a checkpoint). T_2 and T_3 are before T_4 is ending.

Fuzzy checkpointing

Fuzzy checkpointing, in contrast to normal, checks pointing allows transaction to execute while logs are being applied to disks. In fuzzy checkpointing it follows below steps:

- 1. Temporarily stops the transactions to major data C blocks to be copied
- 2. Writes for new checkpoint: i.e. the log
- 3. Creates a list M for all P logs between the last checkpoint to new checkpoint. M is the list of log records which are yet to be written to disk
- 4. Once all M is listed, it allows the transaction to execute. Now the transaction will start entering the logs after the case, the checkpoint. It should enter the logs into the blocks that are in X or old checkpoints
- 5. The buffer blocks in list M are written to the disk or archive storage. No transactions should update these blocks. If the disk fails, and then the block is updated to the disk.

It should have pointers to the last checkpoint. Last checkpoint in the main memory at fixed location. This will help to read the blocks for the next update and maintain new list M.

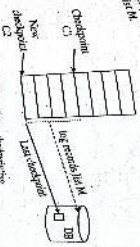


Figure 9.22: Fuzzy checkpointing

Whereas there is a recovery from failure then logs are read from the last checkpoint stored in DB. Logs is because logs before last checkpoint are already updated to DB and those after these points have to be written. These logs are recovered as described in above methods.

9.22 Shadow Paging

Shadow paging is an alternative to log-based recovery. This state of the database prior to transaction execution may be saved. Shadow page table is now modified during execution. To start with, both the page tables are identical. After the transaction, when the transaction completes, the current page table becomes the shadow page table. Whenever any page is updated to be written for the first time

- i. A copy of this page is made into an empty page
- ii. The current page table is then made to point to the copy

iii. The update is performed on the copy

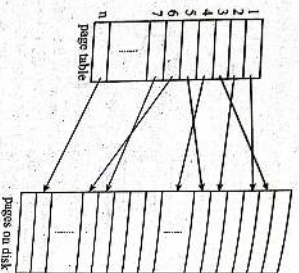


Figure: Paging

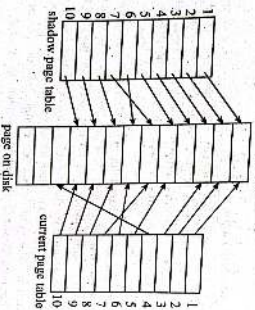


Figure: Shadow paging

to **commit** a transaction we must do the following:

- i. Ensure that all buffer pages in main memory that have been changed by the transactions are output to the disk.
- ii. Output the current page table to the disk. Note that we must output the shadow page table, since we may need it for recovery from a crash.
- iii. Output the disk address of the current page table to the fixed location in stable storage containing the address of the shadow page table.

This action overwrites the address of the old shadow page table. Therefore, the current page table has become the shadow page table and the transaction is committed. If a crash occurs prior to the completion of step iii, the effects of the transaction will be preserved; no redo operations need to be invoked.

Following are the advantages of Shadow paging over log-based schemas:

- i. No overhead of writing log to the records.
- ii. Recovery from crashes is significantly faster (since no undo or redo operations are needed)

Following are the disadvantages:

- i. Commit overhead

Requires multiple blocks to be output: the actual data blocks, the current page table and the disk address of the current page table (log-based schemas need to output only the log records). can be reduced by using a page table structured like a B+ tree. No need to copy entire tree, only to copy paths in the tree that lead to updated leaf nodes. Commit overhead is high even with above extension. Need to flush every updated page and page table

- ii. Data fragmentation

Shadow paging cause database pages to change location when they are updated. Data gets fragmented (related pages get separated on disk)

III. Garbage collection

After every transaction completion, the database containing old versions of modified data need to be collected. Hard to extend algorithm to allow transactions to run concurrently. Easier to extend log based schemes.

9.3 Data Backup/Recovery

9.3.1 Data Backup

When we back up our data, copies of our data are made to ensure availability and protect against data loss. Important files, databases, or entire systems are duplicated and stored in separate locations or storage media. Data backup is essential for following purpose:

I. Data Protection

Backups are designed to protect against accidental deletion, hardware failure, software corruption, cyberattacks, natural disasters, or other unforeseen events that can lead to data loss.

II. Business Continuity

In the event of data loss or system failure, backups enable organizations to recover critical data and resume operations quickly, minimizing downtime and reducing impact on business continuity.

III. Data Recovery

Backup provides the ability to recover lost or corrupted data. By restoring data from backups, organizations can restore previous versions of files or restore systems to a known good state.

IV. Disaster Recovery

In the event of a catastrophic incident such as a fire, flood, or cyberattack, offsite backups ensure that our data is safe and can be restored even if the primary location is inaccessible or compromised.

Compliance and Legal Requirements

Many industries and jurisdictions have specific data retention and compliance regulations. Regular backups help organizations meet these requirements and provide proof of data integrity.

Following aspects should be considered while implementing data backups:

Backup Strategy

Right backup strategy should be determined based on the data size, frequency of changes, recovery time objectives (RTOs), and recovery point objectives (RPOs) of the organization. Common backup strategies include full backup, incremental backup, differential backup, and continuous data protection.

II. Backup Frequency

Backup schedule should be set up that suits business needs and the speed at which data changes. Important data may require more frequent backups than less important data.

III. Storage Media

Appropriate storage medium should be chosen for backup, such as an external hard drive, network attached storage (NAS), tape drive, or cloud storage. Multiple backups can be created and stored on different media for redundancy.

IV. Off-Site Storage

Store backups in a physical location should be separated from primary data to protect against disasters that could impact the primary location.

V. Automation and Monitoring

Backup software or tools that automate the backup process and provide monitoring to ensure successful backups and detect errors or problems should be used.

VI. Testing and Verification

Regular test of the integrity and recoverability of backups should be done by performing restoration tests. This ensures

that the backup data is valid and can be successfully restored when needed.

Other aspects such as data security, access controls, and data retention policies should be considered to ensure the overall protection and effective management of data.

Types of Backup

There are several types of backups that organizations can use to protect their data. The choice of backup type depends on factors such as data importance, recovery time objective (RTO), storage capacity, and available resources. It is important to consider the specific requirements by the organization while choosing the appropriate backup types. Typically, businesses use a combination of backup types to balance data protection needs, recovery time, and storage efficiency. Here are some common types of backups:

I. Full Backup

A full backup involves making a copy of all selected data at a specific point in time. It includes all files, databases or system components. Full backups capture all data but can be time consuming and require significant storage space.

II. Incremental Backup

Incremental backups only record changes made since the last backup, whether it's a full backup or an incremental backup. It backs up new or changed files since the last backup, reducing backup time and storage requirements. To restore data, the last full backup and all subsequent incremental backups must be applied.

III. Differential Backup

Differential backup only records the changes made since the last full backup. Unlike incremental backups, they do not take data, just the latest full backup and the latest differential backup are required.

IV. Synthetic Full Backup

Synthetic full backups are created by combining a previous composite full backup with subsequent incremental backups. This full backup with subsequent incremental backups, this approach eliminates the need to perform regular full backups while maintaining a full backup set. This can help reduce backup time and storage space requirements.

V. Mirror Backup

Mirror backup creates an exact copy or mirror image of the source data in real time or with minimal latency. Changes to the source data are immediately reflected in the mirror backup. Cloned backups offer high availability and fast recovery, but may require significant storage space.

VI. Snapshot Backup

Snapshot backup captures the state of data at a specific point in time, allowing for quick backup and recovery. They use copy-on-write or redirect-on-write techniques to create a virtual copy of point in time. Snapshots can be used as the basis for other types of backups.

VII. Cloud Backup

Cloud backup involves storing backup copies of data in offsite cloud storage. It provides scalability, ease of management, and protection against physical damage or loss. Cloud backup can use different backup types, such as full, incremental, or differential backup.

VIII. Continuous Data Protection (CDP)

CDP captures every change made to data in real-time or near real-time, creating a continuous stream of backups. It enables granular recovery at any point in time, providing minimal data loss in case of failures or data corruption.

9.3.2 Data Recovery

Data recovery is the process of retrieving and restoring lost, deleted, corrupted or inaccessible data from storage media or the system. It involves the use of specialized techniques, tools and

expertise to recover data that is unavailable due to various reasons, such as accidental deletion, hardware or software failure, file system corruption, virus attack or malware, physical damage to storage media or other causes unforeseen events.

The data recovery process normally involves the following steps:

1. Evaluation and Assessment

The first step is to assess the nature and extent of data loss.

Data recovery specialists or software tools analyze the affected storage media or system to determine the cause of data loss and the likelihood of successful recovery.

2. Data Recovery Method Selection

Based on the evaluation, suitable data recovery methods and techniques are selected. This may include file-level recovery, partition recovery, disk image creation and recovery, RAID data recovery, or other specialized methods depending on the specific situation.

3. Data Recovery Execution

The selected data recovery methods are performed to recover lost or inaccessible data. This may involve the use of professional hardware and software tools, or the involvement of professional data recovery services.

4. Data Reconstruction and Repair

In the event of data corruption or damaged file system, data recovery may include repairing or rebuilding damaged components to restore the integrity and accessibility of the data.

5. Data Verification and Validation

Once the data recovery is complete, the recovered data will be validated and verified to ensure its integrity and accuracy. This may involve verifying file and directory structures, performing checksums or validating hashes, and comparing the recovered data with known good copies or backups.

6. Data Restoration

After successful data recovery and validation, the recovered data will be restored to its original location or other storage medium. This may involve transferring data to a new disk or system, rebuilding the RAID array, or restoring files and folders to their original directories.

9.4 Remote Backup Systems

Remote backup provides a sense of security in case the primary location where the database is located gets destroyed. Remote backup can be offline or real-time or online. In case it is offline, it is maintained manually.

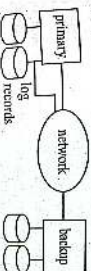


Figure: Remote backup systems

Online backup systems are more real-time and lifelines for database administrators and investors. An online backup system is a mechanism where every bit of the real-time data is backed up simultaneously at two distinct places, one of them is directly connected to the system and the other one is kept at a remote place as backup.

As soon as the primary database storage fails, the backup system scans the failure and switches the user system to the remote storage. Sometimes this is so instant that the users can't even realize a failure.

Detection of Failure

Backup size must detect when primary size has failed to distinguish primary site failure from link failure, maintain several communication links between the primary and the remote backup.

Transfer of Control

To take over control backup site first perform recovery using its copy of the database and all the log records it has received from the primary. Thus, completed transactions are rolled back. Incomplete transactions are rolled back.

When the backup site takes over processing it becomes the new primary. To transfer control back to old primary when it recovers, old primary must receive redo logs from the old backup and apply all updates locally.

Time to Recover

To reduce delay in takeover, backup site periodically processes the redo log records (in effect, performing recovery from previous database state), performs a checkpoint, and can then delete earlier parts of the log.

Hot-Spare configuration permits very fast takeover. Backup updates redo log record as they arrive, applying the process new transactions. When failure of the primary is detected the distributed database with replicated data. Remote backup is faster and cheaper, but less tolerant to failure. Remote backup is faster updates by delaying transaction commit until update is faster backup; avoid this delay by permitting lower degrees of durability.

Time to Commit

To ensure that the updates of a committed transaction are durable, a transaction must not be declared committed until its log records have reached the backup site. This delay can result in permit lower degrees of durability, and some systems therefore be classified as follows.

One-safe

One-safe configuration commits as soon as transaction's commit log record is written at primary. Problem in one safe over.

1.310 | Backup on Database Management System

Two-very-safe

Two very safe commits when transaction's commit log record is written at primary and backup. It reduces availability since transactions cannot commit if either site fails.

Two-safe

Two safe proceed as in two-very-safe if both primary and backup are active. If only the primary is active, the transaction commits as soon as is commit log record is written at the primary. Better availability than two-very-safe, avoids problem of lost transactions in one-safe.

SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS

1. What happens for a log based recovery.

<T0 Start>
<T0.A,1000,880>
<T0.B,2000,2050>
<T0.Commit>
<T1 Start>
<T1.C,700,600>

In log-based recovery, the transaction logs are used to recover the database to a consistent state after a failure. Here,

we assume that the system has crashed after T1 started and before T1 committed. Therefore, T1 must be rolled back and T0 must be redone.

Here is the step-by-step recovery process using the transaction logs:

First, we identify the most recent checkpoint, which records the state of the database and the transaction table at a certain point in time. Since there is no checkpoint in the sequence given, we assume that the checkpoint is at the beginning of the log.

i. Redo Pass

In this pass, we redo all the committed transactions after the checkpoint in the log. In this case, T0 is the only committed transaction after the checkpoint, and it updates records A and B. Therefore, we apply the updates of T0 to the database, and the values of A and B are set to 950 and 2050, respectively.

ii. Undo Pass

In this pass, we undo the incomplete transactions that started after the checkpoint in the log. Here, we need to undo the changes made by T1, which updated record C. Since T1 has not committed, we simply discard the changes made by T1 and leave the value of C at 700.

After the recovery process is completed, the database is in a consistent state, and the changes made by the committed transaction T0 have been saved, while the changes made by transaction T1 have been discarded.

2. Discuss several issues that must be addressed while designing remote backup system. [2019 Sept]

To maintain the integrity, dependability, and security of data, numerous elements must be carefully taken into account while designing a remote backup system. The following are a few critical challenges that must be handled during the design process:

i. Data Transfer Efficiency: Transferring enormous amounts of data across the network with as little impact as possible on network capacity and latency is one of the main issues in remote backup. The amount of data transported can be optimized and decreased via effective data compression, incremental backup methods, and intelligent data deduplication.

ii. Data Encryption and Security: Since sensitive information is carried through networks that may not be trusted, encryption is essential to prevent unauthorized access to or interception of the data. To secure data during transit, robust encryption techniques like SSL/TLS should be deployed. To ensure that only authorized people may access and restore the backed-up data, access controls and authentication measures should also be in place.

iii. Redundancy and Reliability: In order to guarantee data availability and resilience against hardware or network failures, remote backup systems should use redundancy methods. Utilizing redundant network lines or RAID setups can assist maintain data integrity and reduce downtime by implementing redundancy at both the storage and network levels.

iv. Data Integrity and Verification: Mechanisms for checking the reliability of backed-up data should be

Included in backup systems. Checksums, signatures, and hash algorithms may be used in the procedure to identify and fix any data corruption or manipulation that occurred while the backup was in progress. To ensure that data can be recovered when needed, routine integrity checks and validation procedures should be carried out.

v. Scalability and Performance: The backup system must

be built to manage growing data volumes and support future expansion. This necessitates scaling factors like load balancing, distributed storage structures, and support data transfer. Speeds for backup and restore can also be increased through performance optimizations like efficient indexing and metadata management. The

vi. Disaster Recovery Planning

vii. Compliance and Regulatory Requirements

ix. Data Retention and Archiving

x. Cost Considerations

3. Compare the Shadow Paging recovery scheme and Log based recovery scheme.

Ans The difficulties with log-based approach are as follows: [2016 Feb, 2019, Fall]

- The search process is time consuming.
 - Most of the transactions according to log-based algorithm, need to be redone have already written their updates into the database. Although redoing them will cause no harm, it will nevertheless cause recovery to take longer.
- In shadow paging system, the overhead of log-record output is eliminated, and recovery from crashes is significantly faster since no undo or redo operations are needed. The commit of a single transaction using shadow paging requires multiple blocks to be output.
- The actual data blocks
 - The current page tables
 - The disk address of the current page table.

log-based schemes need to output only the log records, which for typical small transactions, fit within one block.

shadow paging causes database pages to change location when they are updated. Hence, the locality property of the pages is lost. This results in the usage of complex, higher overhead schemes for physical storage management in shadow paging as compared to log-based recovery.

Also, garbage collection becomes necessary for shadow paging recovery system. Because, each time a transaction commits, the database pages containing the old version of data changed by the transaction become inaccessible. Such pages are considered garbage, since they are not part of free space and do not contain usable information. Garbage may be created also as a side effect of crashes. Periodically, it is necessary to find all the garbage pages, and to add them to the list of free pages. Thus, garbage collection in shadow paging recovery system results in additional overhead and complexity on the system.

4. In a log based recovery, how does deferred modifications scheme differ with immediate modification scheme? [2016 Spring]

Ans Deferred modifications and immediate modifications are two methods for handling database updates and their corresponding logging schemes in a log-based recovery scheme. These two schemes differ in the following ways:

Deferred Modifications Scheme:

The database updates are initially made directly to the database in the deferred modifications method without immediately generating corresponding log records. This implies that during the actual update operation, the modifications are not instantly logged. Instead, the system waits until a later checkpoint or commit point before logging these modifications.

The database is kept updated and consistent under the deferred modifications method, but the corresponding log

records are created after the actual updates have been made. Since it avoids the overhead of writing log records for each individual modification, this technique offers better efficiency during the update process.

The deferred modifications technique introduces the danger that the system could lose the changes made since the last checkpoint or commit point. If a failure happens before the updates are recorded, a checkpoint operation is periodically carried out to force the writing of all pending alterations to the log, minimizing this danger and assuring their persistence.

Immediate Modifications Scheme:

The immediate modifications approach, on the other hand, makes sure that each database update is promptly followed by a corresponding log entry. The system first creates a log record that contains information about the modification (such as the kind of operation, the affected data, and the old and new values) whenever a modification operation takes place. Before the actual database alteration is made, this log record is written to the log.

In comparison to the deferred modifications scheme, the immediate modifications scheme provides higher durability and recovery guarantees. The log records capture all the information required to restore the database to a consistent state in the event of a failure because they are generated before the changes are made. The recovery procedure uses the log data to undo or repeat the modifications as necessary in the event of a failure.

The necessity to produce log records for each update operation, however, adds extra overhead to the immediate modifications scheme. The update process performance may be slightly impacted by this, particularly in high-volume transactional systems.



TRANSACTION PROCESSING AND CONCURRENCY CONTROL

10.1 Introduction to Transactions

An action, or series of actions, carried out by a single user or application program, which reads or updates the contents of the database is called transaction. A transaction is a logical unit of work on the database. During transaction execution the database may be inconsistent but when the transaction is committed, the database must be consistent.

There are two main issues to deal with:

1. Failures of various kinds, such as hardware failures and system crashes.
- ii. Concurrent execution of multiple transactions.

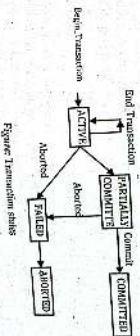


Figure: Transaction states

Transaction State

1. **Active**
The initial state; the transaction stays in this state while it is executing.
2. **Partially committed**
When a transaction executes its final operation it is said to be in a partially committed state.

3. Failed

Which occurs if the transaction can't be committed or transaction is aborted while in active state.

4. Aborted

If any of the checks fails and the transaction has reached a failed state, then the recovery manager rolls back all its write operations on the database to bring the database back to its original state where it was prior to the execution of the transaction. Transactions in this state are called aborted. The database recovery module can select one of the two operations after transaction aborts:

- > restart the transaction
- > kill the transaction

5. Committed

If a transaction executes all its operations successfully it is said to be committed. All its effects are now permanently established on the database system

10.2 ACID Properties of Transaction

To preserve integrity of data, the database system must ensure four important properties of transaction called ACID properties.

1. Atomicity

The "all or nothing" property. A transaction is an invisible unit that is either performed in its entirety or is not performed at all. It is the responsibility of the recovery subsystem of the DBMS to ensure atomicity. It states that all operations of the transaction take place at once if not the transaction is aborted.

There is no midway, i.e., transaction cannot occur partially. Each transaction is treated as one unit and either run to completion or is not executed at all. Atomicity involves the following two operations:

1. Abort

If a transaction aborts, then all the changes made are not visible

II. Commit

If a transaction commits, then all the changes made are visible

Example:

Let's assume that following transaction T consisting of T1 and T2. A consists of Rs600 and B consists of Rs300. Transfer Rs 100 from account A to account B

T1	T2
Read(A)	Read(B)
A:A-100	B:B+100
Write(A)	Write(B)

After the completion of transaction, A consists of Rs 500 and B consists of Rs 400

If the transaction T fails after the completion of Transaction T1 but before completion of T2, then the amount will be deducted from A but not added to B. This shows the inconsistent database state. In order to ensure correctness of database state, the transaction must be executed entirely.

2. Consistency

The integrity constraints are maintained so that the database is consistent before and after the transaction. The execution of transaction will leave a database in either its prior stable state or a new stable state. The consistency property of the database states that every transaction sees a consistent database instance. The transaction is used to transform the database from one consistent state to another consistent state

Example:

The total amount must be maintained before or after the transaction.

Total before T occurs
 $600 + 300 = 900$

Total after T occurs
 $500 + 400 = 900$

Therefore, the database is consistent. In the case when T1 is completed but T2 fails, then inconsistency will occur.

3. Isolation

It shows that the data which is used at the time of execution of a transaction cannot be used by the second transaction until the first one is completed.

In isolation, if the transaction T1 is being executed and using the data item X, then the data item can't be accessed by any other transaction until the transaction T1 ends. The concurrency control subsystem of the DBMS enforces the isolation property.

4. Durability

The durability property is used to indicate the performance of the database's inconsistent state. It states that the transaction made the permanent changes.

The effects of a successfully completed (committed) transaction are permanently recorded in the database and must not be lost, because of subsequent failure.

Concurrent Execution

Multiple transactions are allowed to run concurrently in the system in concurrent execution. The process of managing simultaneous operations on the database without having them interfere with one another is called concurrency control. The operations of transactions are interleaved to achieve concurrent execution.

Advantages of concurrent execution are:

1. Increased processor and disk utilization, leading to better transaction throughput: one transaction can be using the CPU while another is reading from or writing to the disk.

Reduced average response time for transactions; short transactions need not wait behind long ones.

concurrent control schemas

Concurrent control schemas are mechanisms to achieve isolation, i.e., to control the interaction among the concurrent transactions in order to prevent them from destroying the consistency of the database.

10.3 Schedules and Serializability

10.3.1 Schedule

A sequence of the operations by a set of concurrent transactions that preserves the order of the operations in each for the individual transactions is called schedule. A schedule for a set of transactions must consist of all instructions of those transactions. A schedule must preserve the order in which the instructions appear in each individual transaction.

A transaction that successfully completes its execution will have a commit instruction as the last statement. A transaction that fails to successfully complete its execution will have an abort instruction as the last statement.

Types of schedule:

1. Serial schedule
2. Non serial schedule

Serial Schedule

Serial schedule is a schedule where the operations of each transaction are executed consecutively without any interleaved operations from other transactions. There is no interference between transactions.

Example:

Schedule 1

T1	T2
Read(A) A:=A-50 Write(A) Read(B) B:=B+50 Write(B)	Read(A) Temp:=A*0.1 A:=A-temp Write(A) Read(B) B:=B+temp Write(B)

Figure: A serial schedule where T1 is followed by T2.

Schedule 2

T1	T2
Read(A) A:=A-50 Write(A) Read(B) B:=B+50 Write(B)	Read(A) Temp:=A*0.1 A:=A-temp Write(A) Read(B) B:=B+temp Write(B)

Figure: A serial schedule where T2 is followed by T1.

Non serial schedule

A schedule where the operations from a set of concurrent transactions are interleaved

Let T1 and T2 be the transactions defined previously. The following schedule is not serial schedule but it is equivalent to schedule 1. In schedules 1,2 and 3 the sum A+B is preserved.

Schedule 3

T1	T2
Read(A) A:=A-50 Write(A)	Read(A) Temp:=A*0.1 A:=A-temp Write(A)
Read(B) B:=B+50 Write(B)	Read(B) B:=B+temp Write(B)

Figure: A non-serial schedule equivalent to schedule 1

The following concurrent schedule does not preserve the value of A+B:

Schedule 4

T1	T2
Read(A) A:=A-50	Read(A) Temp:=A*0.1 A:=Temp Write(A)
Write(A) Read(B) B:=B+50 Write(B)	Read(B) B:=B+Temp Write(B)

Figure: Non-serial schedule that does not preserve A+B

10.3.2 Serializability

Serializability is a concurrency scheme where the concurrent transaction is equivalent to one that executes the concurrent serially. A schedule is list of transactions. A schedule S of n equivalent of same n transactions. Two schedules are called serial equivalent if they produce the same final state of the database. A schedule s is serial if, for every transaction T_i participating in the schedule, All the operations of T_i are executed consecutively in the

A schedule is serializable if it is equivalent to a serial schedule. Different forms of schedule equivalence give rise to notions of:

- > Conflict serializability
- > View serializability

The main objective of serializability is to find non-serial schedules that allow transactions to execute concurrently without interference and produce a database state that could be produced by a serial execution

Conflict Serializability

Two actions A_i and A_j executed on the same data object by T_i and T_j conflicts if either of them is write operation. Conflict serializability defines two instructions of different transactions accessing the same data item to perform a read/write operation.

Let A_i and A_j are consecutive non conflicting actions that belong to different transactions, we can swap A_i and A_j without changing the result. If two transactions are both read operation then they are not in conflict.

If one transaction wants to perform a read operation and other transaction wants to perform a write operation then they are in conflict and cannot be swapped. If both the transactions are for write operation, then they are in conflict but can be allowed to take place in any order, because the transaction do not read value updated by each other.

If a schedule s can be transformed into schedule s' by a series of swaps of non-conflicting instructions we say that s and s' are conflict equivalent. We can say that a schedule s is conflict equivalent to a serial schedule Actions I₁ and I₂ of transactions T₁ and T₂ respectively conflict if and only if there exist some item Q accessed by both I₁ and I₂ and at least one of those instructions wrote Q.

I ₁ =read(Q), I ₂ =read(Q)	I ₁ and I ₂ don't conflict
I ₁ =read(Q), I ₂ =write(Q)	They conflict
I ₁ =write(Q), I ₂ =read(Q)	They conflict
I ₁ =write(Q), I ₂ =write(Q)	They conflict

Intuitively a conflict between I₁ and I₂ forces a logical temporal order between them. That is replicating their order will change the result. If I₁ and I₂ are consecutive in a schedule and

they do not conflict, their results would remain same even if they had been interchanged in the schedule

Example:

Schedule 5 can be transformed into schedule 6, a serial schedule where T2 follows T1 by series of swaps of non-conflicting instructions. Schedule 5 is conflict serializable.

T1	T2	T1	T2
Read(A)	Read(A)	Read(A)	Read(A)
Write(A)	Write(A)	Write(A)	Write(A)
Read(B)	Read(B)	Read(B)	Read(B)
Write(B)	Write(B)	Write(B)	Write(B)

Schedule 5

Schedule 6

Figure Transformation of non-serial schedule into serial schedule by swapping non-conflicting instructions.

Example of a schedule that is not conflict serializable.

T3	T4
Read(Q)	Write(Q)
Write(Q)	

Figure Schedule that is not conflict serializable

We are not able to swap instructions in the above schedule to obtain either serial schedule <T3, T4> or the serial schedule <T4, T3>. Schedule S1 is conflict serializable schedule with S3 but not with S2.

S1	S2	S3
T1	T1	T1
T2	T2	T2
Read(A)	Write(A)	Read(A)
Write(A)	Write(B)	Read(B)
Read(B)	Read(A)	Write(A)
Write(B)	Read(B)	Write(B)

Figure Schedule S1 is conflict serializable schedule with S3 but not with S2

Precedence graph for testing conflict serializability

Precedence graph or serialization graph is used commonly to test Conflict Serializability of a schedule. It is a directed Graph (V, E) consisting of a set of nodes $V = \{T_1, T_2, T_3, \dots, T_n\}$ and a set of directed edges $E = \{e_1, e_2, e_3, \dots, e_m\}$. The graph contains one node for each Transaction T_i . An edge e_i is of the form $T_i \rightarrow T_j$ where T_i is the starting node of e_i and T_j is the ending node of e_i . An edge e_i is constructed between nodes T_i to T_j if one of the operations in T_i appears in the schedule before some conflicting operation in T_j .

Algorithm

1. Create a node T in the graph for each participating transaction in the schedule.
2. For the conflicting operation $read_item(X)$ and $write_item(X)$ - If a Transaction T_i executes a read item (X) after T_j executes a write item (X), draw an edge from T_j to T_i in the graph.
3. For the conflicting operation $write_item(X)$ and $read_item(X)$ - If a Transaction T_i executes a write item (X) after T_j in the executes a read item (X), draw an edge from T_j to T_i in the graph.

4.

For the conflicting operation write_item(X) and write_item(X) - If a Transaction T_1 executes a write_item(X) after T_1 graph. executes a write_item(X), draw an edge from T_1 to T_1 in the graph.

5. The Schedule S is serializable if there is no cycle in the precedence graph.

> If there is no cycle in the precedence graph, it means we can construct a serial schedule S which is conflict equivalent to schedule S.

> The serial schedule S can be found by Topological Sorting of the acyclic precedence graph. Such schedules can be more than 1.

Example:

Consider the schedule S:

$S: r_1(x) \ r_1(y) \ w_2(x) \ w_1(x) \ r_2(y)$

Creating tabular representation of above schedule.

T1	T2
$r_1(x)$	
$r_1(y)$	$w(x)$
$w(x)$	$r(y)$

Creating Precedence graph

I. Make two nodes corresponding to Transaction T1 and T2.



II. For the conflicting pair $r_1(x) \ w_2(x)$, where $r_1(x)$ happens before $w_2(x)$, draw an edge from T1 to T2.



III. For the conflicting pair $w_2(x) \ w_1(x)$, where $w_2(x)$ happens before $w_1(x)$, draw an edge from T2 to T1.



Since the graph is cyclic, we can conclude that it is not conflict serializable to any schedule serial schedule.

Let us try to infer a serial schedule from this graph using topological ordering.

The edge $T_1 \rightarrow T_2$ tells that T_1 should come before T_2 in the linear ordering.

The edge $T_2 \rightarrow T_1$ tells that T_2 should come before T_1 in the linear ordering.

So, we cannot predict any particular order (when the graph is cyclic). Therefore, no serial schedule can be obtained from this graph.

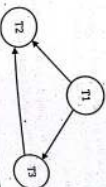
Consider the another schedule S1:

$S1: r_1(x) \ r_3(y) \ w_1(x) \ w_2(y) \ r_3(x) \ w_2(x)$

Creating tabular representation of above schedule.

T1	T2	T3
$r_1(x)$		$r(y)$
$w(x)$	$w(y)$	
$w(x)$		$r(x)$

The graph for above schedule is:



Since the graph is acyclic, the schedule is conflict serializable. Performing Topological Sort on this graph would give us a possible serial schedule that is conflict equivalent to schedule S1.

In Topological Sort, we first select the node with in-degree 0, which is T1. This would be followed by T3 and T2. So, S1 is conflict serializable since it is conflict equivalent to the serial schedule T1 T3 T2.

View Serializability

A schedule will be view serializable if it is view equivalent to a serial schedule. If a schedule is conflict serializable it will be view serializable. The view serializable which does not conflict View Equivalent

Two schedule S1 and S2 are said to be view equivalent if they satisfy the following conditions.

1. Initial Read

An Initial read of both schedules must be the same. Suppose two schedule S1 and S2. In schedule S1, if a transaction T1 is also read A, then in S2, transaction T1 should also read A.

T1	T2	T1	T2
Read(A)	Write(A)	Read(A)	Write(A)

Schedule 1

Schedule 2

Figure: Initial read

Above two schedules are view equivalent because initial read operation in S1 is done by T1 and in S2 it is also done by T1.

2. Updated Read

In schedule S1, if T1 is reading A which is updated by T2 then in S2 also T1 should read A which is updated by T2.

T1	T2	T3	T1	T2	T3
Write(A)	Write(A)	Read(A)	Write(A)		Read(A)

Schedule S1

Schedule S2

Figure: Updated read

Above two schedules are not view equivalent because in S1, T3 is reading A updated by T2. In S2, T3 is reading A updated by T1.

1. Final Write

A final write must be the same between both the schedules. A final write operation should also be done by T1 in schedule S1, if a transaction T1 updates A at last then in S2, final write operation should also be done by T1.

T1	T2	T3	T1	T2	T3
Write(A)	Read(A)	Write(A)	Write(A)	Read(A)	Write(A)

Schedule S1

Schedule S2

Figure: Final write

Above two schedules are view equivalent because final write operation in S1 is done by T3 and in S2, then final write operation is also done by T3.

Example:

Schedule S

T1	T2	T3
Read(A)	Write(A)	
Write(A)		Write(A)

With 3 transactions the total number of possible schedule is 3!=6.

$S1 < T1, T2, T3 >$
 $S2 < T1, T3, T2 >$
 $S3 < T2, T3, T1 >$
 $S4 < T2, T1, T3 >$
 $S5 < T3, T1, T2 >$
 $S6 < T3, T2, T1 >$

Taking first schedule S1

T1	T2	T3
Read(A)		
Write(A)		
	Write(A)	
		Write(A)

Step 1: Updated Read

In both schedules S and S1, there is no read except the initial read that's why we don't need to check that condition.

Step 2: Initial Read

The initial read operation in S is done by T1 and in S1, it is also done by T1.

Step 3: Final Write

The final write operation in S is done by T3 and in S1, it is also done by T3. So, S and S1 are View Equivalent.

The first schedule S1 satisfies all three conditions, so we don't need to check another schedule.

Hence, view equivalent serial schedule is:
 $T1 \rightarrow T2 \rightarrow T3$

preferences between conflict serializability and view serializability:

- > Conflict serializability is easy to achieve but view serializability is difficult to achieve.
- > Every conflict serializable schedule is view serializable but the reverse is not true.
- > It is easy to test conflict serializability but expensive to test view serializability.
- > Most of the concurrency control schemes used in practice are conflict serializability.

10.4 Concepts of Locking for Concurrency Control

The management and coordination of many concurrent transactions that access and modify the same data simultaneously is referred to as concurrency control in DBMS. By controlling concurrency, transactions are prevented from interfering with one another or yielding inaccurate results, and the database is kept in a consistent state.

In this type of protocol, any transaction cannot read or write data until it acquires an appropriate lock on it. There are two types of lock:

- i. Shared lock
- ii. Exclusive lock

1. Shared Lock

It is also known as a Read-only lock. In a shared lock, the data item can only be read by the transaction. It can be shared between the transactions because when the transaction holds a lock, then it can't update the data on the data item. If a transaction T1 has obtained a shared lock on item Q, then T1 can read but cannot write Q.

II. Exclusive Lock

In the exclusive lock, the data item can be both reads as well as written by the transaction. If a transaction T1 has obtained an exclusive lock on item Q, then T1 can both read and write Q. Lock requests are made to concurrency control manager. Transaction can proceed only after request is granted.

Lock compatibility matrix

Current State of Locking of Data Items		Unlocked		Shared(S)		Exclusive(X)	
		Unlocked	Shared(S)	Exclusive(X)	Unlocked	Shared(S)	Exclusive(X)
Lock Mode of Request	Unlocked		Yes	Yes		Yes	Yes
	Shared(S)	Yes	Yes	No	Yes	Yes	No
	Exclusive(X)	Yes	No	No	Yes	No	No

Figure: Lock compatibility matrix

A transaction may be granted a lock on an item if the request lock is compatible with locks already held on the item by the requesting transaction. Any number of transactions can hold shared lock on an item, but if any transaction holds an exclusive lock on the item, no other transaction may hold any lock on the item. If a lock cannot be granted, the requesting transaction is made to wait till all incompatible locks held by other transactions have been released. The lock is then granted.

10.4.1 Two-Phase Locking (2PL)

This is a protocol which ensures conflict serializable schedules. The two-phase locking protocol divides the execution phase of the transaction into three parts. In the first part, when the execution of the transaction starts, it seeks permission for the lock it requires. In the second part, the transaction acquires all the locks. The third phase is started as soon as the transaction releases its first lock. In the third phase, the transaction cannot demand any new locks. It only releases the acquired locks.

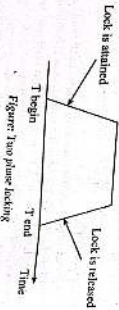


Figure: Two phase locking

There are two phases of 2PL:

- I. Growing phase
- II. Shrinking phase

Growing Phase

In the growing phase, a new lock on the data item may be acquired by the transaction, but none can be released.

Shrinking Phase

In the shrinking phase, existing lock held by the transaction may be released, but no new locks can be acquired. In the below example, if lock conversion is allowed then the following phase can happen:

1. Upgrading of lock (from S(a) to X (a)) is allowed in growing phase.
2. Downgrading of lock (from X(a) to S(a)) must be done in shrinking phase.

Example:

	T1	T2
0	Lock-S(A)	
1		Lock-S(A)
2	Lock-X(B)	
3		
4	Unlock(A)	
5		Lock-X(C)
6	Unlock(B)	
7		Unlock(A)
8		Unlock(C)
9		

The following way shows how unlocking and locking work with 2-PL.

Transaction T1:

- Growing phase
- From step 0-2
- Shrinking phase
- From step 4-6
- Lock point
- At 3

Transaction T2:

- Growing phase
- From step 1-5
- Shrinking phase
- From step 7-8
- Lock point
- At 6

10.4.2 Strict Two-Phase Locking (Strict-2PL)

The first phase of Strict-2PL is similar to 2PL. In the first phase, after acquiring all the locks, the transaction continues to execute normally. The only difference between 2PL and strict 2PL is that Strict-2PL does not release a lock after using it. Strict-2PL waits until the whole transaction to commit, and then it releases all the locks at a time. Strict-2PL protocol does not have shrinking phase of lock release. It does not have cascading abort as 2PL does.



Figure: Strict two-phase locking

Lock conversion

Two phase locking with lock conversions.

first phase:

- > Can acquire a Lock-S on item.
- > Can acquire a Lock-X on item.
- > Can convert a Lock-S to a Lock-X (upgrade)

second phase

- > Can release a Lock-S
- > Can release a Lock-X
- > Can convert a Lock-X to Lock-S (downgrade)
- > This protocol assures serializability. But still relies on the programmer to insert the various locking instructions.

pitfalls of lock based protocol

Consider the partial schedule

T3	T4
lock-x(B)	
Read(B)	
B:=B-50	
Write(B)	
	lock-S(A)
	Read(A)
	lock-S(B)

Neither T3 nor T4 can make progress. Executing lock-S(B) causes T4 to wait for T3 to release its lock on B, while executing Lock-X(A) causes T3 to wait T4 to release its lock on A. Such situation is called a **Deadlock**.

To handle a deadlock one of T3 or T4 must be rolled back and its locks released. The potential for deadlock exists in most locking protocols. Deadlocks are necessary evil. Starvation is also possible if concurrency control manager is badly designed.

Example:

A transaction may be waiting for an X-Lock on an item, while a sequence of other transactions request and are granted on S-Lock on the same item. The same transaction is repeatedly rolled back due to deadlocks. Concurrency control manager can be designed to prevent starvation.

10.5 Deadlock

A deadlock is a condition where two or more transactions are waiting indefinitely for one another to give up locks. Deadlock is said to be one of the most feared complications in DBMS as it brings the whole system to a halt.

Example:

In the student table, Transaction T1 holds a lock on some rows in the Students table and needs to update some rows in the Grades table. Simultaneously, Transaction T2 holds locks on those very rows (Which T1 needs to update) in the Grades table but needs to update the rows in the Student table held by Transaction T1.

Now, Transaction T1 will wait for transaction T2 to give up the lock, and similarly, transaction T2 will wait for transaction T1 to give up the lock. As a consequence, All activity comes to a halt and remains at a standstill forever unless the DBMS detects the deadlock and aborts one of the transactions.

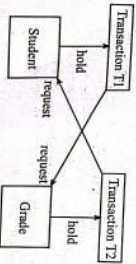


Figure: Deadlock

In multi process system deadlock is an unwanted situation that arises in a shared resource environment, where a process

indefinitely waits for a resource that is held by another process. System is deadlocked if there is a set of transactions such that every transaction in the set is waiting for another transaction in the set. Following schedule is a schedule with deadlock:

T1	T2
Lock-x on X	Lock-x on Y
Write(X)	Write(Y)
Wait for Lock-x on Y	Wait for Lock-x on X
Write(Y)	Write(X)

To prevent any deadlock situation in the system the DBMS aggressively inspects all the operations, where transactions are about to execute. The DBMS inspects the operations and analyzes if they can create a deadlock situation. If they can create a deadlock situation might occur, then that transaction is never allowed to be executed. Deadlock prevention protocols ensure that the system will never enter into a deadlock state.

10.5.1 Deadlock Avoidance

When a database is stuck in a deadlock state, then it is better to avoid the database rather than aborting or restarting the database. This is a waste of resource. Deadlock avoidance mechanism is used to detect any deadlock situation in advance. A method like "wait for graph" is used for detecting the deadlock situation but this method is suitable only for the smaller database. For the larger database, deadlock prevention method can be used.

10.5.2 Deadlock Detection

In a database, when a transaction waits indefinitely to obtain a lock, then the DBMS should detect whether the transaction is

involved in a deadlock or not. The lock manager maintains a wait graph to detect the deadlock cycle in the graph.

Wait for graph

This is suitable method for deadlock detection. In this method, a graph is created based on the transaction and their lock. If the created graph has a cycle or closed loop then there is a deadlock. The wait for graph is maintained by the system for every transaction which is waiting for some data held by the others. The system keeps checking the graph if there is any cycle in the graph. The wait for graph for the above scenario is shown below:

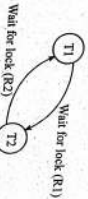


Figure: Wait for graph

Deadlocks can be described as a wait for graph which consists of a pair $G=(V, E)$. V is a set of vertices (all the transactions in the system). E is a set of edges; each element is an ordered pair $T_i \rightarrow T_j$. If $T_i \rightarrow T_j$ is in E , then there is a directed edge from T_i to T_j , implying that T_i is waiting for T_j to release a data item. When T_i requests a data item currently being held by T_j , then edge $T_i \rightarrow T_j$ is inserted in the wait for graph. This edge is removed only when T_j is no longer holding data item needed by T_i .

10.5.3 Deadlock Prevention

Deadlock prevention is suitable for a large database. If the resources are allocated in such a way that deadlock never occurs, then deadlock can be prevented. The Database Management System analyzes the operations of the transactions whether they can create a deadlock situation or not. If they do, then the DBMS never allow that transaction to be executed. Two approach to deadlock prevention are:

1. One approach ensures that no cyclic waits can occur by ordering the requests for locks, requiring all locks to be acquired together.

2. Another approach performs transaction rolled back instead of waiting for a lock, whenever the wait could potentially result in a deadlock.

The first approach requires that each transaction locks all its data items before execution in one step or none are locked. There are two main disadvantages:

1. It is often hard to predict, before the transaction begins, what data items need to be locked.

2. Data item utilization maybe very slow. Since many of the data items may be locked but unused for a long time

The second approach for preventing deadlocks is to use preemption and transaction rollback. These schemas use timestamps just for deadlock prevention

Wait-Die Schema

In this schema, if a transaction requests for a resource which is already held with a conflicting lock by another transaction, then the DBMS simply checks the timestamp of both transactions. It allows older transaction to wait until the resource is available for execution

When Transaction T_i requests a data item currently held by T_j , T_i is allowed to wait only if it has a timestamp smaller than that of T_j . Otherwise, T_i is rolled back (dies). Let's assume there are two transactions T_1 and T_2 and let $TS(T)$ is a timestamp of any transaction T . If T_2 holds a lock by some other transaction and T_1 is requesting for resources held by T_2 then the following actions are performed by DBMS:

1. Check if $TS(T_1) < TS(T_2)$. If T_1 is older transaction and T_2 has held some resource, then T_1 is allowed to wait until the data item is available for execution. This means if the older transaction is waiting for a resource which is locked by the younger transaction, then older transaction is allowed to wait for resource until its available.

ii. Check if $TS(T_i) < TS(T_j)$. If T_i is older transaction and has held some resource and if T_j is waiting for it then T_j is killed and restarted later with the random delay but with the same timestamp.

Example:

Suppose that transaction T_1 , T_2 and T_3 have timestamps 20, 30 and 40 respectively. If T_1 requests a data item held by T_2 then T_1 will wait. If T_3 requests a data item held by T_2 then T_3 will be rolled back.

Wound Wait Scheme

In wound wait scheme, if the older transaction requests for a resource which is held by the younger transaction, then older transaction forces younger one to kill the transaction and release the resource. After the minute delay, the younger transaction is restarted but with the same timestamp.

If the older transaction has held a resource which is requested by the younger transaction, then the younger transaction is asked to wait until older release it. When transaction T_1 requests a data item currently held by T_3 , T_1 is allowed to wait only if it has timestamp larger than that of T_3 otherwise T_1 is rolled back.

Example:

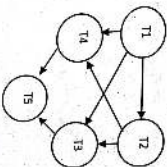
Suppose that transaction T_1 , T_2 and T_3 have timestamps 20, 30 and 40 respectively. If T_1 requests a data item held by T_2 , then the data item will be preempted from T_2 and T_2 will be rolled back. If T_3 requests a data item held by T_2 , then T_3 will wait. Both in wait die and in wound wait schemes a rolled back transaction is restarted with its original timestamp. Older transactions thus have precedence over newer ones and starvation is hence avoided.

Timeout-Based schemes

A transaction waits for a lock only for specified amount of time. After that the wait times out and transaction is rolled back. Thus, deadlocks are not possible. Simple to implement, but timeout interval. Also, difficult to determine good value of the timeout interval.

SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS

Since every conflict serializable schedule is view serializable, why do we emphasize conflict serializability rather than view serializability? Consider the precedence graph given below. Is the corresponding schedule conflict serializable? Explain your answer.



Since every conflict serializable schedule is view serializable, why do we emphasize conflict serializability rather than view serializability? Because conflict-serializability needs in simple algorithms for its checking, while checking of view-serializability belongs to NP-complete problems. NP-complete problems are any of a class of computational problems for which no efficient solution algorithm has been found.

Since there are no efficient solution, we emphasize conflict serializability. Also, every conflict serializable schedule is view serializable we could only concentrate on view serializability if a schedule is not conflict serializable.

Since the graph is acyclic, the schedule is conflict serializable. Performing Topological Sort on this graph would give us a possible serial schedule that is conflict equivalent to given schedule.

In Topological Sort, we first select the node with in-degree 0, which is T_1 . So T_1 comes at first in order.

ADVANCED DATABASE CONCEPTS

11.1 Object Oriented Model

This data model is a method of representing real world objects. Object oriented model considers each object in the world as objects and isolates it from each other. Object oriented model groups its related functionalities together and allows inheriting its functionality to others related sub groups.

Element of Object Oriented Model

- **Objects**
The real world entities and situations are represented as objects in the object oriented database model.
- **Attributes and Method**
Every object has certain characteristics. These are represented using attributes. The behavior of the object is represented using methods.
- **Class**
Similar attributes and methods are grouped together using a class. An object can be called as an instance of the class.
- **Inheritance**
A new class can be derived from the original class. The derived class contains attributes and methods of the original class as its own.

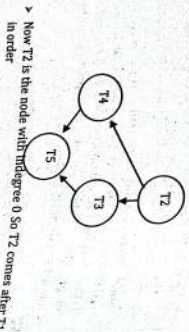
Example:

Shape, circle, Rectangle and Triangle are all objects in this model.

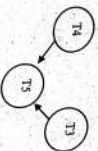
Circle has the attributes center and radius. Rectangle has the attributes length and breadth. Triangle has the attributes base and height. The objects circle, rectangle and triangle inherit from the object shape

11.2 Object Relational Model

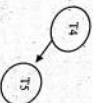
A data modeling and manipulation technique that connects object-oriented programming and conventional relational databases is called the Object-Relational Model (ORM).



> Now T2 is the node with indegree 0 So T2 comes after T1 in order



> Now T4 and T3 are node with indegree 0 So T3 or T4 can come after T2 in order



> Now T4 is the node with indegree 0 So T4 can come after T3 in order



> Hence given schedule is conflict serializable since it is conflict equivalent to the serial schedule T1 T2 T3 T4 T5.



A programmer can specify a mapping between objects in the programming language and tuples in database relations using object-relational mapping technologies.

A selection condition on an attribute of an object, or set of conditions, relevant data are then retrieved from the selection database, and one or more objects are created from the underlying data according to the predetermined mapping between the retrieved and relations.

The mapping from objects to relations is then used to appropriately update, insert, or delete tuples in the database. The software can change retrieved objects, create new objects, or declare that an item is to be removed, and then issue a save command.

Object-relational mapping systems' main objective is to make it easier for programmers to construct applications by giving them access to an object model while yet keeping the advantages using systems can also offer: significant speed improvements over direct access to the underlying database when working with objects cached in memory.

Developers can map database tables to classes, database columns to object properties, and database relationships to object associations using the tools and libraries offered by ORM frameworks. Metadata, which details the connections between database tables, columns, and rows, is often used for this mapping. Some popular ORM frameworks include Hibernate for Java, Entity Framework for .NET, SQLAlchemy for Python, and Doctrine for PHP.

Advantages:

I. Object-oriented approach

ORM provides a more object-oriented approach to database access, allowing developers to work with database data as objects rather than writing low-level SQL queries. This simplifies database access and makes it easier to write maintainable code.

II. Reduced development time

ORM can reduce development time by automating the process of mapping database tables to classes, database columns to object properties, and database relationships to object associations. This allows developers to focus on writing business logic and application behavior rather than writing low-level database access code.

III. Portability

ORM can make applications more portable by abstracting the underlying database technology. This makes it easier to switch between different database vendors and even different database types, such as SQL and NoSQL databases.

IV. Improved security

ORM can improve security by providing built-in protection against SQL injection attacks, a common security vulnerability in database-driven applications.

V. Scalability

ORM can improve scalability by providing support for caching, lazy loading, and other performance optimization techniques. This can help to improve application performance and reduce the load on the database server.

VI. Easier testing

ORM can make it easier to write unit tests and integration tests by providing a more object-oriented approach to database access. This can simplify the testing process and help to ensure that the application behaves correctly under different scenarios.

Disadvantages

I. Performance overhead

ORM introduces a performance overhead as it needs to translate between object-oriented code and relational databases. This can slow down database access and impact application performance, especially when working with large datasets.

II. Complex configurations

ORM can be complex to configure and set up, especially when working with complex data models or non-standard database schemas. This can require additional time and effort during the development process.

III. Limited control

ORM abstracts the underlying database, which can limit the level of control that developers have over database access and optimization. This can make it harder to optimize database queries for specific use cases or to use advanced database features.

IV. Lack of flexibility

ORM can be less flexible than writing custom SQL queries, especially when working with complex data models or non-standard database schemas. This can limit the ability to optimize database access for specific use cases.

V. Learning curve

ORM can require additional learning for developers who are not familiar with object-oriented programming or relational databases. This can slow down the development process and increase the learning curve for new developers.

11.3 Distributed Databases

A distributed database is a collection of multiple interconnected databases which are spread physically across various locations that communicate via a computer network. Transactions may access data at one or more sites. The main data reside in on single location where as in the later the data reside in a several locations. A distributed database management system (DDBMS) is a centralized software system that manages a distributed database in a manner as if it were all stored in a single location.

General Architecture of Pure Distributed Databases

The general schema architecture of a DDB is shown in Figure below. The enterprise is given a consistent, unified view that reveals the logical structure of the underlying data across all nodes. The global conceptual schema (GCS), which offers network 'sides', is a representation of this view. Each node is transparent, is having its own local internal schema (LIS) based on physical organizational details at that particular location in order to account for potential heterogeneity in the DDB. The local to conceptual schema (LCS) describes how data are logically organized at each site. The fragmentation and replication transparency are provided by the GCS, LCS, and their underlying mappings.

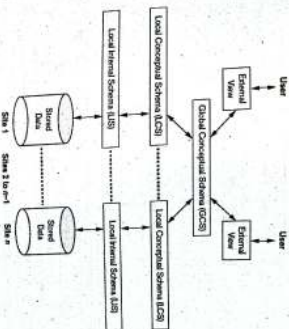


Figure: Schema architecture of distributed databases.

The component architecture of a DDB is shown in the figure below. In order to validate and impose defined constraints, the global query compiler refers to the global conceptual schema from the global system catalog. The global query optimizer creates optimal local queries from global queries by referring to both

global and local conceptual schemas. Using a cost function that calculates costs based on response times (CPU, I/O, and network latencies) and estimated sizes of intermediate outcomes, it assesses all potential solutions. The optimizer chooses the candidate with the lowest cost for execution after computing the costs for each candidate. Each local DBMS would have a local optimizer, transaction manager, and execution engines. Together with the local transaction manager at each of those sites, the global transaction manager is in charge of coordinating the execution across many locations.

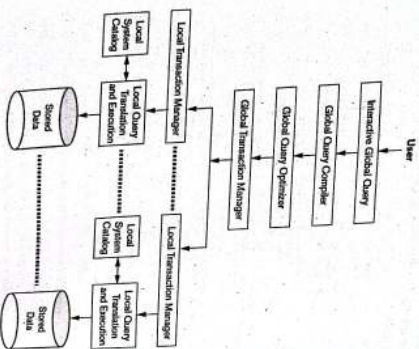


Figure: Component architecture of distributed databases.

Types:

- I. Homogeneous Database
- II. Heterogeneous Database

11.3.1 Homogeneous Distributed Database

In homogeneous database, all sites have identical software and are aware of each other and agree to co-operate in processing user requests. Each site surrenders part of its autonomy in terms of right to change schemas or software. Database appears to user as a single system.

11.3.2 Heterogeneous Distributed Database

In heterogeneous distributed database different sites may use different schemas and software. Difference in schema is a major problem for query processing. Difference in software is a major problem for transaction processing. Sites may not be aware of each other and may provide only limited facilities for co-operation in transaction processing.

11.3.3 Distributed Database Storage

In distributed data storage information is stored on more than one node, often in a replicated fashion. Assume relational data model stored in the database. Two approaches to store relation in distributed database are:

I. Replication

System maintains multiple copies of data stored in different sites for faster retrieval and fault tolerance.

II. Fragmentation

Relation is partitioned into several fragments stored in distinct sites. Replication and fragmentation can be combined. Relation is partitioned into several fragments, system maintains several identical replicas of each fragments

Data Replication

A relation or fragment of a relation is replicated if it is stored redundantly in two or more sites. Full replication of a relation is

the case where the relation is stored at all sites. Fully redundant databases are those in which every sites contains a copy of the entire database.

Advantages of Replication

i. Availability

Failure of site containing relation r does not result in unavailability of r if replicas exist.

ii. Parallelism

Queries on r may be processed by several nodes in parallel.

iii. Reduced data transfer

Relation r is available locally at each site containing a replica of r .

Disadvantages of Replication

i. Increased cost of updates

Each replica of relation r must be updated.

ii. Increased complexity of concurrency control

Concurrent updates to distinct replicas may lead to inconsistent data unless special concurrency control mechanisms are implemented. One solution may be to choose one copy and apply concurrency control operations on primary copy.

Data Fragmentation

Data fragmentation is division of relation r into fragments $r_1, r_2, r_3, \dots, r_n$ which contains sufficient information to reconstruct relation r . These fragments may be stored at different locations. The data fragmentation process should be carried out in such a way that the reconstruction of original database from the fragments is possible.

Types of Data Fragmentation

1. Horizontal Data Fragmentation

Each tuple of r is assigned to one or more fragments

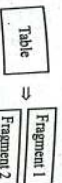


Figure: Horizontal data fragmentation

Consider the relation:

Customer_Id	Name	Area	Payment Type	Sex
1	Bob	London	Credit Card	M
2	Mike	Manchester	Cash	M
3	Ruby	London	Cash	F

Horizontal fragmentation are subsets of tuple (rows)

Fragment 1

Customer_Id	Name	Area	Payment Type	Sex
1	Bob	London	Credit Card	M
2	Mike	Manchester	Cash	M

Fragment 2

Customer_Id	Name	Area	Payment Type	Sex
3	Ruby	London	Cash	F

Advantages of horizontal fragmentation

- Allows parallel processing on fragments of a relation
- Allows a relation to be split so that tuples are located where they most frequently accessed

2. Vertical Data Fragmentation

The schema for relation r is split into several smaller schemas. This is the vertical subset of a relation that means a relation/table is fragmented by considering the columns of it.

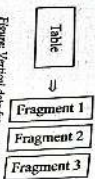


Figure: Vertical data fragmentation

Vertical fragmentation are subset of attributes

Customer Id	Name	Area	Sex
1	Bob	London	M
2	Mike	Manchester	M
3	Ruby	London	F

Customer Id	Payment Type
1	Credit Card
2	Cash
3	Cash

Advantages of vertical fragmentation

- Allows tuples to be split so that each part of the tuple is stored where it is most frequently accessed.
- Tuple id attribute allows efficient joining of vertical fragments.
- Allows parallel processing on a relation

3. Hybrid Data Fragmentation

This is the combination of horizontal as well as vertical fragmentation. This type of fragmentation will have distributed over the DB, and vertical fragmentation to have subset of columns of the table.

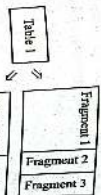


Figure: Hybrid data fragmentation

As we observe in above diagram this type of fragmentation can be done in any order. It does not have any particular order. It is solely based on the user requirements. But it should satisfy fragmentation conditions.

Reasons for Distributed Database

Distributed database is used because of:

- Business unit autonomy and distribution.
- Data sharing
- Data communication costs.
- Data communication reliability and costs.
- Multiple application vendors.
- Database recovery.
- Transaction and analytic processing

11.4 Concept of Data Warehouse

A data warehouse is a collection of corporate information and data derived from operational systems and external data sources. A data warehouse is designed to support business decisions by allowing data, consolidated analysis and reporting at different aggregate levels. Data is populated into data warehouse through the process of extraction transformation loading.

A single complete and consistent store of data obtained from a variety of different sources made available to end users in what they can understand and use in a business context is called warehouse. A data warehouse is subjected oriented integrated.

time-variant nonvolatile collection of data in support of management's decision-making process. Following are the characteristics of data warehouse:

- i. Subject oriented
- ii. Integrated
- iii. Non-Volatile
- iv. Time-Variant
- v. Summarized
- vi. Large Volume
- vii. Non normalized
- viii. Metadata
- ix. Data Sources

Subject Oriented

A data warehouse is subject oriented because it provides information around a subject rather than the organization's ongoing operations. These subjects can be product, customers, suppliers, sales, revenue, etc. A data warehouse does not focus on the ongoing operations, rather it focuses on modelling and analysis of data for decision making. Excludes data not useful in decision support process.

Integrated

A data warehouse is constructed by integrating data from heterogeneous sources such as relational databases, flat files, etc. This integration enhances the effective analysis of data. Data preprocessing are applied to ensure consistency. The data warehouse is a centralized consolidated database that integrates data from entire organization. Multiple sources, diverse sources and diverse formats

Time Variant

The data collected in a data warehouse is identified with a particular time period. The data in a data warehouse provides information from the historical point of view. The data warehouse represents the flow of data through time. Every key structure contains either implicitly or explicitly an element of time.

Non-volatile

Non-volatile means the previous data is not erased when new data is added to it. A data warehouse is kept separate from the operational database and therefore frequent changes in operational database is not reflected in the data warehouse.

Note: A data warehouse does not require transaction processing, recovery, and concurrency controls, because it is physically stored and separate from the operational database.

Data warehouse represents company's entire history. Near term history is continually added to it. It is always growing. Most of the warehouse support terabyte databases and multiprocessors. Read only database is used for data analysis and query processing. Data warehouse requires two operations in data accessing

- > Initial loading of data
- > Access of data.

11.4.1 Rules of Data Warehouse

- > Data warehouse and operational Environments are separated.
- > Data is integrated.
- > Contains historical data over a long period of time.
- > Data is a snapshot data captured at a given point in time.
- > Data is subject oriented.
- > Mainly read only with periodic batch updates.
- > Development lifecycle has a data driven approach versus traditional process driven approach.
- > Data contains several levels of detail. Current, old, lightly summarized, highly summarized.
- > Environment is characterized by read only transactions to very datasets.
- > System that traces data sources transformations and storage.
- > Metadata is a critical component. Source, transformation, integration, storage, relationships, history etc.

11.4.2 Need for Data Warehousing

- > Industry has huge amount of operational data. Knowledge worker wants this data into useful information. This information is used by them to support strategic decision making.
- > It is a platform for consolidated historical data for analysis.
- > It stores data for good quality so that knowledge worker can make correct decisions.
- > From business perspective:
 - It is latest marketing weapon
 - Helps to keep customers by learning more about their needs
 - Valuable tool in today's competitive fast evolving world

11.4.3 Data Warehouse Architecture

Data warehouse server

A data warehouse server is the physical storage used by a data warehouse system. The physical storage consists of almost always a relational DBMS and rarely flat files. Various processed data and other relevant information that comes from several applications and sources are stored in a data warehouse server where it is organized for future business analysis and user query purposes.

Online Analysis Processing Servers

Online analysis processing servers are used to support and operate on multi-dimensional data structures. It allows managers, and analysts to get an insight of the information through fast, consistent, and interactive access to information.

Clients

- > Query and reporting tools
- > Analysis tools
- > Data mining tools

11.4.4 Application of Data Warehouse

- > OLAP (Online Analytical Processing) is a term used to describe the analysis of complex data from data warehouse
- > DSS (Decision support systems) also known as EIS (Executive Information Systems) supports organizations leading decision makers for making complex and important decisions
- > Data mining is used for knowledge discovery the process of searching data for unanticipated new knowledge
- > Data warehouses are widely used in the following fields:
 - Financial services
 - Banking services
 - Consumer goods
 - Retail sectors
 - Controlled manufacturing

SOLUTION TO EXAMS AND OTHER IMPORTANT QUESTIONS

1. Explain the advantage and disadvantage of object oriented database over relational database. [2+2 Spring]

Ans Advantages of Object-Oriented Database (OODB) over Relational Database (RDB):

i. Enhanced Modeling Capabilities

In comparison to RDB, OODB offers improved modeling capabilities since it enables more accurate depiction of complex real-world objects, relationships, and behaviors. This makes it ideal for systems like multimedia systems, CAD/CAM programs, and scientific simulations where the object structure is a key factor.

ii. Native Object Persistence

Objects can be stored directly in the database without the requirement for object-relational mapping or laborious normalization because OODB inherently supports object persistence. This streamlines development and lessens the impedance mismatch between the database and the object-oriented programming language.

iii. Complex Data Structures

With the aid of OODB, complex data structures, such as those involving object association, composition, and inheritance, can be stored. This encourages more accurate depictions of actual relationships, the encapsulation of data and activity, and more organic modeling of complex domain concepts.

iv. Improved Performance

For some applications, OODB delivers better performance. Compared to RDB's complicated join processes, direct access to objects enables quicker retrieval and navigation. Additionally, OODB does away with the requirement for pricey normalization procedures, which may enhance performance for specific query patterns.

Disadvantages of Object-Oriented Database (OODB) over Relational Database (RDB):

1. Lack of Standardization

Like SQL in RDB, OODB does not have a standardized query language. As a result, working with various OODB systems may be more difficult, and system compatibility may be constrained.

ii. Limited Industry Support

Compared to RDB, OODB has not experienced extensive industrial adoption. There is an absence of reliable and developed OODB solutions because the vast majority of database systems and tools are created for relational databases. This may present difficulties in terms of support, compatibility, and availability with current software ecosystems.

iii. Complexity and Learning Curve

In comparison to RDB, designing, implementing, and maintaining OODB systems can be more difficult. For developers and administrators, mastering the complex nature of mapping objects to a database schema as well as object-oriented concepts like inheritance and polymorphism can be challenging.

iv. Data Integrity and Consistency

Due to the absence of standardized procedures like constraints and triggers found in RDB, OODB may have trouble assuring data consistency and integrity. In OODB, it may be more challenging to ensure data integrity in complicated object interactions and to enforce business rules, requiring customized implementation.

2. Under which situations will be beneficial to have replication or fragmentation of data? Explain with suitable example. [2+1+1 Fall]

Ans Depending on the unique requirements and characteristics of the system, both data replication and data fragmentation

might be advantageous in various circumstances. The benefits of each strategy are explained below, along with appropriate examples:

i. Data Replication

Creating and maintaining numerous copies of the same data across several locations or nodes in a distributed system is known as data replication. It has a number of advantages, including as increased availability, fault tolerance, and decreased network latency.

ii. Improved Availability

By providing numerous independent copies of the data, replication can increase system availability. Data can still be accessed from other replicas, ensuring continuous service even if one node or location fails. This is especially beneficial in situations where high availability is essential, such as online banking systems or e-commerce platforms.

iii. Load Balancing

The task can be split among several nodes using replication. The system can balance read and write activities by replicating data and letting clients access the closest replica, which lessens the burden on individual nodes and boosts speed. Data replication is used by content delivery networks (CDNs) to cache material closer to end consumers, lowering latency and enhancing user experience.

iv. Geographic Distribution

Data replication makes it possible to store data across several different places, allowing for localized access and lowering network latency. In worldwide systems with users scattered across many regions, this is advantageous. A multinational corporation might, for instance, duplicate its customer data in regional databases to offer quick and responsive services to clients in several locations around the world.

v. Data Fragmentation

Data fragmentation refers to the splitting of a database into smaller pieces or partitions in a distributed system, data fragmentation refers to the splitting of a database into smaller pieces or partitions that can be stored on several nodes or servers. A portion of the data is present in every piece. Improvements in efficiency, scalability, and data localization are just a few advantages of fragmentation.

vi. Performance

By dispersing the data among several nodes, fragmentation might enhance query performance. Faster response times can be achieved by parallelizing the execution of queries that only need a portion of the data. For instance, client data may be dispersed based on geographic regions in a sizable e-commerce system. The amount of data that has to be processed is decreased when processing orders since the system may query the pertinent fragment comprising client data from the particular region.

vii. Scalability

Data fragmentation enables the addition of new nodes to the system as the data volume increases, supporting horizontal scalability. The task can be distributed across each new node, allowing the system to manage bigger datasets. Data fragmentation is frequently used by social media platforms to divide user data across various servers and handle the increasing number of users and the data they generate.

viii. Data Localization

When sensitive data must be stored in specific locations due to legal or privacy restrictions, fragmentation might be advantageous. For instance, due to data protection laws, patient data in a healthcare system may need to be held only inside the borders of a specific nation or region. Data fragmentation and localized database storage assure adherence to these specifications.

3. Briefly explain on the requirement of data fragmentation and replication in the context of distributed databases.

ANS

Data fragmentation is required if improving at least one of the following is our goal:

- > Single-user response time
- > Concurrency
- > Availability
- > Backup-and-restore characteristics
- > Loading of data

Data replication is required in context of distributed databases because of the following advantages:

- i. **Availability:** Failure of site containing relation r does not result in unavailability of r if replicas exist.
- ii. **Parallelism:** Queries on r may be processed by several nodes in parallel.
- iii. **Reduced data transfer:** Relation r is available locally at each site containing a replica of r .



'Silence' and 'smile' are two powerful words. 'Smile' is the way to solve many problems and 'silence' is the way to avoid many problems.



Er. Bishon Lamichhane

Visiting Faculty, Kathford International College
of Engineering and Management, IoE, TU
Former Lecturer, Kathmandu Engineering
College, IoE, TU



Er. Ujjwal Thapa

System Engineer, National Information
Technology Center

Publisher: SYSTEM INCEPTION

Price Rs 500/-

Distributor: SUGAM STATIONERY SUPPLIERS

Koteshwor, Kathmandu

Contact No.: 9841599592



CamScanner